Architectural Design and Evaluation of a Peer-to-Peer Solar Energy Trading System via Smart Contracts on a Simulated Solana-compatible Consortium Network

Chanthawat Kiriyadee
Computer Engineering and Artificial Intelligence
University of the Thai Chamber of Commerce
Bangkok, Thailand
2410717302003@live4.utcc.ac.th

Abstract — The growing adoption of rooftop solar generation leaves many electricity users with surplus energy that they wish to trade directly. Yet direct Peer-to-Peer (P2P) electricity trading remains difficult, owing to grid constraints and the lack of a transparent price-formation mechanism. This paper presents the architectural design and evaluation of a P2P energy-trading system on a permissioned, governable consortium blockchain with predictable transaction cost. The core of the design is a clear separation of data verification from transaction settlement: off-chain verification is performed by an Aggregator Bridge that checks the Ed25519 signatures of meter readings before orders are matched by a Continuous Double Auction (CDA), while on-chain settlement records only verified matches and enforces correctness conditions at every step. The value of the work lies not in any single component but in the integration of all of them. Evaluation on a simulated system is consistent with this design: the ingest path sustains 80 meters at the design rate of 5.33 readings/s with no data loss, and under a step load up to 640 meters it keeps the loss ratio below 0.03%, with the measured ingest rate being a lower bound of a single sender client. The matching engine processes about 3.1×10^4 order pairs per second, and the single-pair on-chain settlement instruction costs about 97,000 compute units (80,000–103,000 across token-leg variants), roughly 48% of the default per-instruction budget. The complementary generation-mint path that credits verified surplus on-chain is thinner still — its associated-token-account creation and Token-2022 mint cost only a few thousand compute units combined, keeping the whole mint an order of magnitude under the per-instruction budget and bound by neither compute nor transaction packet size. Meanwhile the settlement path is global-write-bound by design — every settlement must serialize on a central reconciliation write — which on a single validator yields a sub-1 transaction-per-second rate (about 0.5 TPS in a single recorded run, pending repeated measurement); even this conservative bound still supports about 450 settlements per 900-second clearing window, roughly an order of magnitude above the ≤ 40 matched pairs the tested 80-meter workload generates. These results support using the blockchain as a thin settlement layer. Alongside the measurements, we report the design's guarantees through a compiled registry of named invariants that separates status from enforcement mechanism and requires a disclosed gap for anything short of full enforcement; applied to the settlement currency's payment leg it reports two of nine invariants as unmet, including the platform's own non-custody claim. This is an architectural evaluation on a simulation, not a measurement on a real power system.

Index Terms — Consortium Blockchain, Peer-to-Peer, Continuous Double Auction, Proof of Authority, Microservices

I. INTRODUCTION

Rooftop-solar prosumers produce surplus they want to trade, yet direct P2P stays blocked by grid constraints and the absence of transparent price discovery — this paper contributes an architectural design and simulation evaluation of a governable, cost-predictable P2P market.

The expansion of Renewable Energy (RE) has led many electricity users to shift from being purely consumers to being producers and consumers of energy at the same time (Prosumers). This change creates challenges for the traditional power grid, which was designed primarily for centralized distribution to end users. Research on Peer-to-Peer energy markets therefore focuses on trading mechanisms that preserve both the constraints of the grid and the reliability of the power system [1], [2], [3].

The transition toward Smart Grid systems in the Thai context, following the master plan for the development of Thailand's power grid, employs Advanced Metering Infrastructure (AMI), making energy production and consumption data more granular. Integrating Distributed Energy Resources (DER) such as rooftop solar photovoltaic systems, electric vehicle (EV) charging stations, and Battery Energy Storage Systems (BESS) must account for requirements on DER interconnection, Microgrid controllers, Islanding, and the constraints of the low-voltage grid [4], [5], [6].

This work makes four main contributions. First, a decoupled architecture design that clearly separates off-chain data verification through the Aggregator Bridge — namely Ed25519 signature verifi-

cation of meter readings following the DLMS/COSEM standard and evaluation of grid stability conditions together with order matching via Continuous Double Auction (CDA) — from on-chain settlement. Second, the design of a trust boundary for settlement on an Anchor/Solana-compatible Proof of Authority (PoA) consortium network, where the blockchain verifies the Ed25519 signatures of both buyer and seller, prevents replay through an order nullifier and escrow state, while the energy-quantity conditions and oracle attestation are enforced in the off-chain layer, making the blockchain act clearly as a settlement and audit layer (see Section IV). Third, the development of an AMI simulation suite that relies on grid modeling with pandapower and a Smart Meter Simulator to generate deterministic meter readings, together with a preliminary measurement of the ingest rate of the telemetry ingest path. Fourth, a method for reporting what such a design actually guarantees: a compiled, served, and test-asserted registry of named invariants in which status and enforcement mechanism are independent axes, every entry short of full enforcement must carry a disclosed gap, and the enforcement axis ranks a guarantee provided by the blockchain runtime above one provided by a program check and that above an off-chain check. Applied to the settlement currency's payment leg (Section IX), the registry reports two of its nine invariants as unmet — including the platform's own non-custody claim — which is the point of the construction rather than an artifact of it. The main distinction from prior work (see Section II) is not in any single component (consortium blockchain, CDA, or off-chain oracle, all of which exist in prior work), but in integrating these components through a clearly defined trust boundary: a zone-partitioned CDA over a consortium PoA network that clearly separates the off-chain telemetry verification layer (Ed25519/DLMS) from the settlement layer. The on-chain settlement mechanism at the core of this work is atomic delivery-versus-payment (DvP) that combines the Ed25519 signature verification of both parties, partial-fill replay prevention through an order nullifier, and a cross-zone flow cap into a single set of correctness conditions enforced within a single transaction, which prior work has not specified systematically. The scope of this work is an architectural design and evaluation on a simulation system, not a measurement from a field power grid or a production Solana network. The empirical contribution of this work is therefore a reproducible single-system characterization — namely the ingest rate, compute-unit cost, and throughput of the settlement path — rather than a head-to-head quantitative comparison with other platforms, which is future work.

On the economic-mechanism side, the system uses an energy token issued from actual production and certified by a Renewable Energy Certificate (REC), where one REC token represents 1 MWh of certified green generation. In the Thai regulatory context, RECs are issued solely by the Energy Regulatory Commission (ERC), the government authority; the on-chain programs accordingly name the issuance instruction and certificate account with the *erc* identifier (*issue_erc*, *ErcCertificate*). This is combined with a stablecoin pegged to the Thai baht (THBC) for pricing and settlement, and the GRX token for staking and governance. The details of the pricing model are described in Section VI, and the details of the relevant programs are described in Section VII.F.

This paper is organized as follows. Section II reviews related research on blockchain and Peer-to-Peer energy markets. Section III defines the system, trust, and attacker models. Section IV describes the system's settlement model. Section VI describes the CDA pricing mechanism and the settlement computation. Section VII describes the architecture, connectivity, and key implementation details. Section VIII describes how the device and wallet identities that the settlement model presupposes are constructed at onboarding time. Section IX describes the off-chain payment leg and the disclosed-invariant registry through which its guarantees are reported. Section X specifies the experimental details and workload. Section XI summarizes the architectural evaluation, and Section XII discusses results, limitations, and future development directions for the system. The acronyms frequently used in this paper are summarized in Table I.

Table I
Abbreviations used in this paper.

Acronym	Meaning
AMI	Advanced Metering Infrastructure
BESS	Battery Energy Storage System
BFT	Byzantine Fault Tolerance
CDA	Continuous Double Auction
CPI	Cross-Program Invocation
CU	Compute Unit
DAO	Decentralized Autonomous Organization
DER	Distributed Energy Resource
DLMS/COSEM	Device Language Message Specification / COSEM (IEC 62056)
DSO	Distribution System Operator
DvP	Delivery-versus-Payment
GRX / THBC	Governance token / THB-pegged stablecoin
IAM	Identity and Access Management
mTLS	Mutual TLS
OPF	Optimal Power Flow
PDA	Program Derived Address
PoA	Proof of Authority
PoH	Proof of History
RBAC	Role-Based Access Control
REC	Renewable Energy Certificate
SPIFFE	Secure Production Identity Framework for Everyone
SPL	Solana Program Library (Token)
SVM	Solana Virtual Machine
VPP	Virtual Power Plant

II. RELATED WORK

Prior P2P-market and blockchain-settlement work leaves one gap: none cleanly separate data verification from settlement while keeping on-chain cost bounded and governable.

Blockchain technology originated with Bitcoin [7] as a decentralized ledger system that requires no intermediary, and was extended to Smart Contract execution with Ethereum [8], [9]. An overview of the technology and the classification of networks into permissionless and permissioned types was summarized by NIST [10], which serves as a reference framework for selecting a network architecture suited to governance requirements.

In the context of Peer-to-Peer energy markets, a large body of research has studied market mechanisms and auction design. Mengelkamp et al. [11] compared local energy market designs and bidding strategies, while Munsing et al. [12] proposed using blockchain for decentralized optimization of energy resources in microgrid networks. These works highlight the potential of blockchain to support direct energy trading between users, but most still evaluate on public networks that are constrained by transaction cost and block-confirmation latency. A recent concrete example is the decentralized trading platform of Esmat et al. [13], which develops a market mechanism and settlement on a public blockchain (Ethereum) but does not clearly separate the off-chain data-verification layer from on-chain settlement.

To address governance and cost predictability, a number of studies have turned to consortium or permissioned networks. Kang et al. [14] used a consortium blockchain for Peer-to-Peer electricity trading among plug-in electric vehicles, and Hyperledger Fabric [15] is an example of a distributed operating system for permissioned networks with clearly defined participant permissions. On the consensus side, the Byzantine Generals problem [16] and the Practical Byzantine Fault Tolerance algorithm [17] form the theoretical foundation for transaction confirmation in networks with a limited number of nodes, consistent with the permissioned-network assumption that governs validator admission via Proof of Authority (PoA) in this work (here PoA is an admission/governance layer, while Layer 1 consensus still relies on PoH together with Tower BFT).

The systematic review of Andoni et al. [18] comprehensively surveys blockchain applications in the energy sector and identifies scalability, cost, and governance as the main challenges, providing a foundational reference frame for this body of research. Recent literature reviews further reflect that this remains an open research area. Tanis et al. [19] comprehensively reviewed the market structure, operational layers, and multi-energy systems of Peer-to-Peer trading, while Bhavana et al. [20] surveyed blockchain applications in energy markets and green hydrogen supply chains, pointing out that scalability, cost, and regulatory compliance remain the main challenges. In addition, a comparative evaluation of consensus mechanisms for Peer-to-Peer energy trading in microgrids [21] supports the choice of a permissioned network with controlled permissions, consistent with the PoA-based admission/governance assumption used in this work.

Unlike the works above, this article focuses on the design and evaluation of the architecture of a simulation system that clearly separates off-chain verification through the Aggregator Bridge from the settlement layer on the blockchain. The scope of this work is therefore architectural design and evaluation, not the reporting of measurements from a field electrical grid or a Solana production network; the energy data used in the prototype comes from an AMI Simulator, and what is recorded on the blockchain is a settlement event that has already been verified by the Backend/Aggregator Bridge, consistent with the preliminary guidance for testing microgrid control systems [22]. The positioning of this work relative to works that use blockchain for Peer-to-Peer energy markets is summarized in Table II, where the main distinction is the integration of zone-partitioned CDA on a PoA-based consortium network with the clear separation of the off-chain meter-telemetry verification layer (Ed25519/DLMS) from the settlement layer. Note that Table II is a qualitative positioning, since these works run on different networks, datasets, and assumptions, and therefore share no quantitative metric that is directly comparable.

Table II
Positioning of this work vs. prior blockchain-based P2P energy-trading studies.

Work	Network	Consensus	Market mechanism	Off/on-chain separation
Mengelkamp [11]	Public (LEM)	Public	Local market / bidding	Not clearly separated
Munsing [12]	Public	Public	Decentralized optimization	Not clearly separated
Kang [14]	Consortium	Consortium	Iterative double auction	Partial
Hyperledger Fabric [15]	Permissioned	Pluggable (Raft/ BFT)	Platform (not market-specific)	—
Esmat [13]	Public (Ethereum)	Public	Double auction	Partial
This work	**Consortium (Solana-compatible)**	**PoH + Tower BFT (PoA admission/ governance)**	**CDA price-time zone-partitioned**	**Clearly separated + Ed25519/DLMS telemetry**

III. SYSTEM MODEL AND THREAT MODEL

Actors, trust assumptions, and adversaries — forged meter readings, replay, and unauthorized mint or settlement — with residual trust made explicit before the settlement model.

This section defines the system model, the actors, the trust assumptions, and the adversary model of the simulated system, so that the security boundaries of the layered architecture are clear before the discussion of the settlement model in Section IV. Since the scope of this work is a design on a simulated system, the analysis below states, in a straightforward manner, both the threats that the current mechanisms mitigate and the trust that still remains (residual trust).

A. System Model and Actors

The system comprises the following principal actors:

- Smart Meter (edge device): an endpoint device that holds a per-device Ed25519 key pair and signs energy readings before transmitting them.
- Aggregator Bridge: the off-chain validation and aggregation layer. It verifies the Ed25519 signature of every reading against the device's public key, decrypts the payload according to the DLMS/COSEM standard, and evaluates the surplus-energy and grid-stability conditions.
- Trading Service: matches buy/sell orders using a Continuous Double Auction (CDA).
- Chain Bridge: acts as the Reference Monitor and is the only service that holds the signing key and calls Solana RPC directly. Every transaction written to the chain is forced through a single signing path.
- Anchor Programs: the on-chain rule-enforcement layer that accepts only already-validated order pairs and serves as the settlement and audit layer.
- PoA / Governance Authority: the principal policy authority of the consortium that controls REC certification, authority permissions, and the aggregator allow-list.

B. Trust Assumptions

The system's trust assumptions are enforced across three boundaries: the device-to-bridge boundary, the service-to-service boundary, and the service-to-chain boundary.

At the device-to-bridge boundary, each device's identity is bound to a registered Ed25519 public key. The Aggregator Bridge verifies the signature of every reading (64-byte signature, 32-byte public key),

both per-reading and in batch, using a fail-closed policy: when a key is not found or the key store is unresponsive, the system rejects (errors) rather than implicitly accepting the reading. The binary DLMS/COSEM payload is encrypted with AES-256-GCM using the per-device key. In production mode, if a device has no encryption key, that frame is skipped (fail-closed), and the plaintext path is enabled only in development mode through an explicit configuration.

At the service-to-service boundary for chain writes (in particular the path to the Chain Bridge), communication uses ConnectRPC over mutually authenticated TLS (mTLS), binding service identity with a SPIFFE [23] X.509 identity verified from the client-side certificate during the handshake. Access permissions are derived from the SPIFFE URI of the verified certificate, mapped to a service role such as *AggregatorBridge*, *TradingMatcher*, or *SettlementService*. Identity is considered from the transport layer (L4), not from application headers (L7); an unverified caller is assigned the role *Unknown*. Note that the meter-data ingest path at the Aggregator Bridge uses request-level authentication via an API key (verified through IAM with a static key fallback) combined with the per-reading Ed25519 signature, not mTLS/SPIFFE. The authentication bypass (insecure mode) that grants full permissions is intended for development mode only.

At the service-to-chain boundary, the Chain Bridge is the only service that holds the key and submits transactions. The signing key is stored in HashiCorp Vault Transit (Ed25519) and never enters process memory; signing is restricted to the explicitly authorized key name only. For the asynchronous write path over NATS JetStream [24], the publisher must sign the message envelope with an ECDSA P-256 signature over the canonical bytes of the envelope, which is verified against the public key in the publisher's client certificate. The Chain Bridge verifies the chain (certificate ⟶ CA ⟶ SPIFFE SAN ⟶ service identity ⟶ signature) before the RBAC step, and value-moving subjects, such as minting energy tokens, are always required to carry a signed envelope.

C. Adversary Model and Mitigations

The adversary model considers an attacker who can craft, intercept, or replay messages on the network and may attempt to impersonate a device or service identity, but cannot access the private keys stored in Vault or the per-device keys. The threats considered and their mitigating mechanisms are summarized in Table III.

Table III
Threats considered and the mechanism that mitigates each.

Threat	Description	Mitigation
T1 Forged/replayed reading	Forge or replay a meter reading	Per-reading Ed25519 signature verification (fail-closed) + the per-*(meter_id, window) gen_mint* PDA, which absorbs a replayed reading into the window it belongs to
T2 Service impersonation	Impersonate a service in the mesh	mTLS + SPIFFE identity ⟶ service role; unverified callers get *Unknown*
T3 Forged publisher / mint	Submit a forged chain-write command over NATS	Signed envelope bound to the certificate; mint subjects require a signature
T4 Replayed on-chain order pair	Settle the same order pair twice	Order nullifier account (PDA) — settle only once
T5 Duplicate/over-authorized mint	Mint more tokens than actual production	Idempotency key *(meter_id, window)* + PDA + REC co-signing

D. Residual Trust and Out-of-Scope

The system retains residual trust that must be stated explicitly. First, the Aggregator Bridge and the governance/oracle authority attest to the surplus-energy and grid-stability conditions in the off-chain layer, and these conditions are not re-verified on-chain; users must therefore trust the correctness of this off-chain validation layer. Compromise or collusion of the Aggregator Bridge or the oracle authority is not yet prevented by cryptographic mechanisms in the current prototype; approaches to further reduce trust, such as multi-oracle attestation or cryptographic proof, are future work (see Section XII). Second, the security of the consensus layer (PoH combined with Tower BFT) rests on the assumption of a permissioned network that controls validator participation via PoA, where validators are authorized entities behaving according to policy — an architectural assumption that has not yet been quantitatively evaluated. Third, the authentication bypasses for development mode (insecure mode and plaintext fallback) must be disabled in production; otherwise they would break all of the above trust assumptions.

E. Bounding the Trust Graph: the ERP Direction Invariant

The residual trust stated above has a specific shape: the unconditionally trusted set is small — the Aggregator Bridge, which attests that a reading is genuine, and the Chain Bridge, which holds the only signing key. A design that keeps this set small is worth more than one that merely adds mechanisms, because every additional service holding mint or settlement authority enlarges the set of principals whose compromise is unrecoverable. This subsection states the rule the architecture uses to integrate an enterprise system without enlarging it.

Integrating the utility's own billing system is operationally necessary — the utility's ERP, not this platform, remains the authority for the customer invoice — but a naive integration would take the unconditionally trusted set from two principals to three. The architecture instead constrains the ERP connector to be a **strictly downstream consumer**: no response, acknowledgement, or read-back originating from the ERP may influence a mint, a settlement, or any on-chain write. The ERP is a sink and a witness, never an oracle.

The invariant is enforced in depth rather than by convention. The connector holds no signing key and has no policy binding to the Vault Transit backend, so it cannot produce a signature the Chain Bridge would accept; its only secret-store access is a read of the client credential for the enterprise gateway. Its advisory publisher accepts a closed enumeration of subjects rather than an arbitrary subject string and refuses any subject outside its own namespace, so the chain-write subjects are not merely unused but unaddressable. Most usefully for review, the constraint is checked against the build artifact rather than asserted in prose: a test reads the resolved dependency lockfile and fails if the shared blockchain crate, the Solana client libraries, or the Anchor client libraries appear anywhere in the transitive graph, and a companion test scans every source file for a chain-write subject literal. A future revision that reintroduced a path to the chain would fail the build rather than silently widen the trust boundary. A complementary control — omitting the connector's service identity from the Chain Bridge authorization list, so that a publish would be refused at the bridge even if the code were changed — is specified but lives in bridge configuration and is not exercised here.

The security consequence is worth stating precisely, because it is easy to overclaim. Compromising the ERP connector yields **false**

enterprise records, which reconciliation against the chain detects; it does not yield fraudulent minting. The attack surface grows, but the mint trust graph does not. The gain runs in the other direction: because the utility's system holds an independently sourced meter reading, divergence between the three records — meter, chain, and ERP — becomes observable. This is the first **detective** control the architecture has against a compromised Aggregator Bridge, which is otherwise trusted unconditionally at the point where a reading is admitted. It is detective and not preventive: it reports a discrepancy after the fact rather than preventing a forged attestation, and it does not close the attestation gap identified above, for which hardware-backed or multi-party attestation remains future work.

Two limits apply to this claim in the present prototype. The connector's first phase is implemented against a mock enterprise adapter; the adapter for a real enterprise system builds its payload but deliberately refuses to transmit, and the downstream posting and certificate-registry paths remain switched off by configuration flags. Furthermore, the enterprise-side interface model is written against a canonical utilities schema and would require per-utility validation before deployment, since the module footprint of a specific utility is not established from public sources. The three-way divergence check is therefore a property of the design and of the enforced direction invariant, not a measured result, and it is not evaluated in Section XI.

IV. SETTLEMENT MODEL

The central thesis: cleanly split off-chain verification from on-chain settlement, so the chain records only verified matches and enforces correctness invariants at every step — the value is the composition, not any single part.

A. Architecture Overview

This work partitions responsibility into two interconnected trust domains joined through a single point. The off-chain trust domain performs validation and matching: it comprises the metering layer (a Smart Meter Simulator following the AMI standard and DLMS/COSEM) that feeds data into the Aggregator Bridge to verify Ed25519 signatures and screen the data, before forwarding it to the Trading Service, which matches orders using the Continuous Double Auction (CDA) algorithm, with the IAM Service and Notification Service supporting identity and notification. The on-chain trust domain settles transactions and records evidence: it comprises a set of Anchor programs (registry, trading, oracle, energy-token, governance, and treasury) on a permissioned (consortium) Solana-compatible network that records state into PDA accounts and an event log.

The crossing from the off-chain domain to the on-chain domain occurs solely through the Chain Bridge, the only service that contacts Solana RPC directly. It receives write commands over NATS JetStream and serves state reads over ConnectRPC on a channel protected by mTLS. The on-chain programs are Anchor programs [25] running on the Solana Virtual Machine (SVM) [26] and use Program Derived Addresses (PDA) to create accounts whose ownership is deterministically verifiable. Separating the two domains in this way means that power-engineering validation and access control happen before a transaction is recorded, while the blockchain serves as a thin settlement and audit layer. The full details of the services, their connectivity, and the architecture diagram appear in Section VII. This article is an architectural evaluation on a simulated system, not a measurement from a production Solana network.

B. Settlement Model

The settlement model separates responsibility into two layers with clearly distinct trust boundaries. The off-chain layer performs validation and matching: it comprises the Aggregator Bridge, which verifies the Ed25519 signatures of meter readings, checks the data format against the DLMS/COSEM standard, and evaluates available-surplus conditions together with grid stability; and the Trading Service, which matches buy/sell orders using the Continuous Double Auction (CDA) algorithm. The output of this layer is a validated order pair together with an order payload signed by both parties. The on-chain layer is an Anchor program [25] that acts as a thin settlement and audit layer, accepting only validated order pairs and recording an auditable result.

Before recording a settlement, the on-chain program enforces only conditions that are verifiable from the payload and account state, namely: account-ownership checks, the Ed25519 signatures of both buyer and seller on the order payload, double-submission prevention via the order nullifier account, and the validity of the escrow state. Conditions that require external data, such as available surplus and oracle attestation, are enforced in the off-chain layer before submission and are not re-checked on-chain. This division of responsibility means the blockchain need not directly trust external data, yet can confirm that every settlement originates from an order pair genuinely signed by both parties. All account structures use Program Derived Addresses (PDA) to isolate the state of each transaction and to verify ownership deterministically. The details of the programs and PDA accounts are described in Section VII.F [27].

C. Atomic Delivery-versus-Payment

A matched order pair is settled through the ***settle_offchain_match*** instruction of the trading program, which operates as delivery-versus-payment (DvP) within a single transaction; that is, the transfer of funds and the transfer of energy happen together or not at all. If any transfer fails, the entire transaction is reverted, so there is no lingering state in which one party has received while the other has not. Before the transfer, the program checks the Ed25519 signatures of both the buyer (instruction sysvar index 0) and the seller (index 1) over a message that binds the order's key fields, namely order_id, user, energy_amount, price_per_kwh, side, zone_id, and expires_at, making it impossible to alter the amount or price after signing. It then checks the price-crossing condition, namely that the match price must lie within the range $p_s \leq p^* \leq p_b$, checks the side of both parties, and checks the expiry time.

Settlement is partial-fill: the order nullifier account (seed *[nullifier, user, order_id]*) stores the accumulated filled amount (filled_amount) instead of a boolean value, so an order can be filled multiple times up to the signed amount, but the total will not exceed that amount ($\mathrm{match} \leq \mathrm{energy\_amount} - \mathrm{filled}$) on either side. For cross-zone transactions that use inter-zone transmission lines, the program additionally enforces a cap on the accumulated committed flow on-chain ($\mathrm{committed\_flow} + \mathrm{match} \leq \mathrm{capacity}$), whereas transactions within the same zone are exempt because they do not use inter-zone transmission lines. Once all conditions pass, the total value and the shares are computed as follows, with currency

flowing from the buyer escrow to the fee/wheeling/loss collectors and the seller escrow, and energy (energy token) flowing from the seller escrow to the buyer escrow. All transfers out of the escrow accounts are signed by the market_authority PDA as the owner of the escrow accounts (signing the token-transfer CPI), while authorization of the settlement comes from the Ed25519 signatures of the buyer and seller as described above. Besides the single-match instruction, the program also has a batch settlement instruction (*batch_settle_offchain_match*) that can combine up to 4 pairs per transaction to reduce cost, using the very same set of validation conditions (Ed25519 signatures, order nullifier, price crossing, and the cross-zone flow cap), and binding accounts from the signed payload by checking the Program Derived Address (PDA) of every account passed in.

The escrow structure just described — one escrow PDA per party, address-bound to the signed payload — is the model this section analyzes and the one the *settle_offchain_match* and *batch_settle_offchain_match* instructions implement. The program also carries a second, custodial settlement entry point (*execute_atomic_settlement*) whose escrows are pooled token accounts owned by the platform key rather than by the parties, and it is that path the prototype deployment currently calls (see Section VII.F.1). Every invariant stated below concerns the non-custodial path; on the custodial path the invariants that derive from **address binding** — that an attacker cannot substitute a victim's escrow, and that authority to disburse is a property of the code path rather than of a key an operator holds — are replaced by trust in the platform key, while the invariants enforced from the **payload** (both signatures, the nullifier, price crossing, the flow cap, the checked value split) continue to hold unchanged.

$$V = \text{match} \cdot p^* \tag{1.1}$$

$$\text{fee} = \lfloor V \cdot \varphi / 10000 \rfloor \tag{1.2}$$

$$\text{net}_s = V - \text{fee} - w - c_{\text{loss}} \tag{1.3}$$

This set of equations is the on-chain rounded-down integer (fixed-point) form of Equation 7.1 through Equation 7.3 in Section VI.A, where V is the total value leaving the buyer escrow, φ is the market fee (market_fee_bps), w is the wheeling charge, c_{loss} is the loss cost (see Equation 5), and net_s is the seller's net amount. All computations use checked arithmetic instead of clamping values; that is, the value multiplication rejects the overflow case, and the deduction of the seller's net will **reject** the transaction (require! with error *ChargesExceedValue*) when the sum of the fee, wheeling and loss exceeds the value V, instead of rounding the amount down to zero, together with a cap on the total network fees of no more than 20% of V. When the market is configured to settle in THBC, the system enforces recording the gross value through the *record_settlement* CPI to the treasury program, so that the settlement counter reconciles against the actual cash flow leaving escrow.

D. Discovery Path and Settlement Path

The trading program exposes two families of instructions that are easily mistaken for one another, and the distinction is central to reading the rest of this section. The **discovery path** — *match_orders*, *clear_auction*, *submit_limit_order* — operates on the on-chain order book: it establishes which orders pair with which, at what price, and records the outcome. It moves no tokens. The **settlement path** — *settle_offchain_match* and *batch_settle_offchain_match* — is the only path in the system that moves value, and it accepts pairs that were matched **off-chain** and signed by both parties. Table IV contrasts them.

Table IV
The two instruction families of the trading program. Only the settlement path moves tokens; the discovery path produces an on-chain record of a match, not a transfer. The two also differ in what they trust: the on-chain book versus a pair of Ed25519 signatures over off-chain orders.

	discovery path	settlement path
instructions	`match_orders`, `clear_auction`, `submit_limit_order`	`settle_offchain_match`, `batch_settle_offchain_match`
writes	`Order`, `TradeRecord`, zone book depth	escrow transfers, `TradeNullifier`, `ZoneCapacity`
moves tokens	no — the recorded value is informational	yes — the only value-moving path
trust anchor	the on-chain order book	Ed25519 signatures on off-chain orders

Because the two paths coexist, the same quantity is represented at two different scales, and this dual-scale contract must be stated explicitly to avoid misreading either the code or the events it emits. On the discovery path, *TradeRecord.total_value* is the raw product of amount and price with **no** rescaling by the nine-decimal energy divisor, because it is a discovery artifact that no transfer consumes. On the settlement path the same product **is** rescaled, because it becomes an actual currency movement. Conflating the two would inflate a settled amount by a factor of 10^9, and the on-chain auction verifier depends on the raw scale being preserved on the discovery side, so neither representation can simply be normalized to the other. The dual scale is therefore a contract between the two paths rather than an inconsistency, and it is documented as such at both ends.

The pricing rules of the two paths also differ, and both are correct within their own scope. The on-chain *match_orders* instruction clears at the **resting sell price**: given a crossing pair, the clearing price is the seller's ask, so any price improvement accrues to the buyer rather than being split at the midpoint. Its only admission test is the crossing condition $p_b \geq p_s$. The off-chain matching engine, by contrast, clears at the landed cost p^* of Equation 6, which folds in wheeling, losses, the incentive multiplier, and the intra-zone discount (Section VI.A) — a richer rule that the on-chain book deliberately does not attempt to reproduce. The batch alternative *clear_auction* computes supply and demand curves and their intersection to give one uniform clearing price for an entire batch, but it likewise defers all token movement to the settlement path.[1]

Orders themselves carry an explicit lifecycle. An order is created *Active*, becomes *PartiallyFilled* once any quantity settles against it, and reaches *Completed* when the cumulative filled amount meets the order quantity, at which point the zone book's active-order count is decremented; *Cancelled* and *Expired* are the two terminal states reached without full execution. The filled amount is monotonically non-decreasing and the remaining quantity is always the order amount less that filled total, which is what makes partial fills safe to apply repeatedly (Section IV).

E. The Settlement Pipeline

[1] The uniform-auction instruction pair is implemented and tested but has no on-chain, test, or client caller in the evaluated configuration; the live path is continuous double-auction matching off-chain followed by *settle_offchain_match*. We report it as available design surface, not as a measured mechanism.

The settlement instruction executes a fixed, ordered pipeline in which every stage is a rejection point: no state is written and no token moves until all checks have passed, and any failure reverts the entire transaction. The order of the stages is itself a design decision — the cheapest and most security-critical checks run first, so a forged or replayed settlement is rejected before any compute is spent on token arithmetic.

The first stage verifies both parties' Ed25519 signatures through the instruction sysvar, with the buyer's verification instruction at index 0 and the seller's at index 1, each over the 77-byte canonical message of Section IV.K. The second bounds the economics: the match price must not exceed the buyer's signed limit nor fall below the seller's, both parties' declared sides must be correct, and neither order may have expired. The third stage applies the transmission constraint, but only for cross-zone matches — the committed flow on the inter-zone *ZoneCapacity* account is incremented and checked against the zone's capacity, while an intra-zone match does not touch that account at all. The fourth stage is the two-layer replay guard: the per-order nullifier caps the cumulative filled quantity on each side, and the per-match *TradeNullifier* account is created atomically with the settlement, so a re-sent match — one re-signed with a fresh blockhash, which would defeat a transaction-hash-based deduplication upstream — fails on an already-existing account even when the underlying orders still have unfilled headroom.

Only then does the fifth stage compute the charge waterfall. Writing q for the matched quantity in nine-decimal energy units, p^* for the match price, and $D = 10^9$ for the energy scale divisor:

$$V = \lfloor q \cdot p^* / D \rfloor \tag{2.1}$$

$$f = \lfloor V \cdot \varphi / 10000 \rfloor \tag{2.2}$$

$$W = \lfloor q \cdot r_w / D \rfloor \tag{2.3}$$

$$L = \lfloor V \cdot \ell / 10000 \rfloor \tag{2.4}$$

$$\mathrm{net}_s = V - f - W - L \tag{2.5}$$

Two asymmetries here are easy to miss and both are deliberate. The wheeling charge W is computed from a **flat per-kWh rate** r_w scaled by the energy divisor, not as basis points of the trade value, whereas the market fee φ and the loss factor ℓ **are** basis points of value. And both r_w and ℓ are read directly from the on-chain *TariffConfig* account rather than taken from the caller's arguments, so a submitting operator cannot inject a tariff of its choosing; the account is passed as a mandatory trailing account and its bytes are validated and decoded in place. Two guards then bound the result: the network charges $W + L$ may not exceed 20% of V, and the total deductions may not exceed V at all. Both are hard rejections rather than saturating subtractions — an earlier formulation clamped the arithmetic and could silently pay the seller zero, which is precisely the failure mode that a rejection makes impossible.

The sixth stage performs the transfers, and it is worth noting that three distinct assets move, at three different scales and in two directions. Currency (six decimals) flows out of the buyer's escrow to the fee, wheeling, and loss collectors and then to the seller for the net amount, with each zero-valued leg skipped. Energy (nine decimals, kWh scaled by 10^9) flows in the opposite direction, from the seller's energy escrow to the buyer's. Where the renewable attribute is being conveyed, an optional REC leg moves from the seller to the buyer at 1,000 base units per kWh, reflecting a six-decimal REC mint against nine-decimal energy. The seventh and final stage records the settlement in the treasury: for markets configured to settle in THBC, the invocation carries the **gross** value and is mandatory — omitting the treasury accounts is rejected rather than silently skipped, so the treasury's settled total cannot drift below the cash that actually left escrow.

F. Write-set Structure and Parallel Execution

Because Solana schedules transactions in parallel whenever their write sets are disjoint, what a settlement **write-locks** determines how many settlements can proceed concurrently. The design pushes every hot write onto a per-entity account: orders live at a PDA derived from the owner and order identifier, each match creates its own nullifier, and per-zone volume accumulates into shard accounts rather than into the zone book.

The sharpest instance of this discipline is the treatment of the zone market account. Capacity and zone identity are **read** from it on every settlement, but the mutable committed-flow counter lives on a separate *ZoneCapacity* account that is passed as a mutable account only on the cross-zone path. An intra-zone settlement — the common case, since local matching is what the market's discount is designed to encourage — therefore never write-locks the shared zone book, and any number of intra-zone settlements in the same zone remain mutually parallelizable. Had the counter stayed on the zone market account, every settlement in a zone would have serialized on it regardless of whether it used an inter-zone line at all. The per-shard accumulators are drained into the zone book by a separate administrative reconciliation instruction, which is off the hot path by construction. What remains genuinely serial is the treasury's central settled total and the collector accounts, which every settlement must write — and that, rather than any property of the shards, is the bottleneck the throughput measurement in Section XI.F reports.

G. Authorization to Settle

A final gate governs **who** may submit a settlement at all. The transaction's fee payer must correspond to an active, governance-admitted aggregator entry, validated by the same read-only pattern described in Section IV.L: owner check, canonical PDA derivation, and in-place decoding of the entry's bytes. Settlement is consequently not self-serviceable — possessing two validly signed orders is not sufficient to settle them; the submitting operator must itself be admitted by the consortium's governance layer. Zones additionally carry a market segment, and a wholesale zone requires the submitting aggregator to hold wholesale admission specifically, whereas retail zones accept any admitted aggregator. Entries predating the segment field default to retail rather than failing, so the field could be introduced without invalidating operators already admitted. This closes the loop between the on-chain settlement mechanics described above and the consortium governance model of Section VII.E: the cryptographic checks establish that a trade was genuinely agreed by both parties, and the admission check establishes that a licensed party is the one recording it.

H. Consensus

The system's smart contracts are Anchor programs running on the Solana Virtual Machine (SVM). The ordering and consensus of transactions are therefore the responsibility of the Solana-compatible network on which the programs are deployed, and are not defined

or tuned at the program level. The platform's consensus mechanism is split into two cooperating parts, namely Proof of History (PoH), a cryptographic verifiable clock that orders events in time before voting, and Tower BFT, a Byzantine Fault Tolerant voting mechanism developed from the Practical BFT (PBFT) [17] concept to confirm blocks and declare block finality [26].

In this work's evaluation, the Anchor programs are run on an SVM-level test environment (LiteSVM) and a single-node validator (localnet) to verify correctness and measure the compute-unit cost of the settlement path (see Section XI.E). Consequently, consensus properties such as leader selection, validator voting, and multi-node finality time are platform properties that are not measured in this work. The control of participation rights and consortium governance is a program-level governance layer separate from network-level consensus; details are in Section VII.E.

I. Block and Settlement Transaction Structure

Because the programs run on a Solana-compatible network, the block structure and block header follow the platform's definition and are not tuned at the program level. Each block is bound to a slot with a target of approximately 400 milliseconds (see Section VII.E). The order of transactions within is determined by the Proof of History (PoH) sequence before voting with Tower BFT, while each transaction references the latest blockhash to set its lifetime and prevent replay at the transaction level. These structures are platform properties [26], so this work emphasizes the structure of the settlement transactions that the program designs itself, rather than the block format. A block chain linked by previous blockhash is shown in Fig. 1.

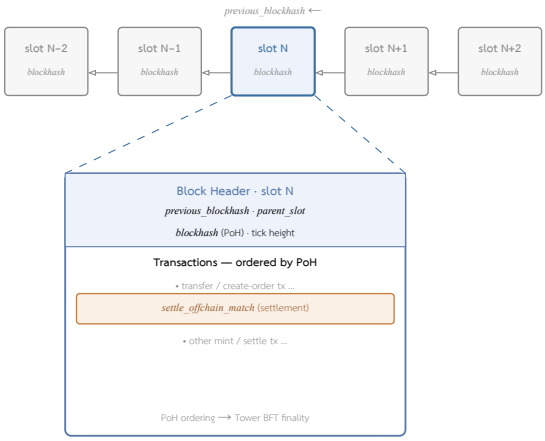


Fig. 1. Block structure on a Solana-compatible network (defined by the platform, not tuned at the program level): a continuous chain of blocks (slot N−2 to N+2) in which each block references its predecessor through previous blockhash. Below, slot N is expanded; the block header contains the blockhash derived from PoH and parent_slot, while the transactions within, including the ***settle_offchain_match*** settlement instruction (see Section IV.I), are ordered by Proof of History before finality is declared by Tower BFT.

Because the block format is what a settlement is ultimately recorded into, and because it differs fundamentally from the mined-block model most readers carry, it is worth stating precisely what a block is on this platform. A block is not a fixed data structure with a proof-of-work header; it is the ordered sequence of **entries** that the slot's leader streams during one time-boxed slot. There is no nonce, no difficulty target, and no mined Merkle root, and the value called the "blockhash" is simply the last Proof-of-History hash produced during the slot — a timestamp and ordering value, not a hash of a header.

A slot is a **time** box rather than a **size** box: a fixed window of approximately 400 milliseconds, numbered monotonically so that the slot number is also the block height. It ends when the time elapses, whether the leader filled it, half-filled it, or produced nothing — an offline leader simply yields a skipped slot, and the chain builds on the last real block. This is the opposite of a size-capped, proof-of-work-timed block that arrives whenever a puzzle is solved. Which node leads which slot is not decided by an election at slot time: the leader schedule for an entire epoch is computed locally by every node, deterministically and weighted by stake or, in this system's permissioned setting, by admitted authority, before the epoch begins. No messages are exchanged to discover the leader of slot N — every node has already looked it up. This is the second latency saving after PoH itself: where a classical Byzantine-fault-tolerant protocol spends a message round choosing a proposer, this platform spends none.

The unit a block is built from is the entry, which carries the number of PoH hashes elapsed since the previous entry, the PoH hash value at that point, and a vector of zero or more transactions pinned at that hash. An entry with no transactions is a **tick** — pure advancement of the verifiable clock at a fixed number of hashes per tick, keeping time moving even when there is no traffic — while an entry with transactions fixes their order by mixing them into the hash chain at that point. When the configured number of ticks per slot is reached, the slot ends. A block is therefore the entry list for its slot, and replaying it reconstructs two proofs from one stream: the PoH chain establishes time and order, and re-executing the transactions establishes the resulting state.

The Proof-of-History clock underneath has a deliberate asymmetry that matters for a permissioned deployment. A tick is a fixed number of sequential SHA-256 iterations, each hashing the previous output, and a slot is a fixed number of ticks — so a slot is a fixed quantity of strictly sequential single-core hashing, on the order of 400 milliseconds. Producing it is inherently serial and $O(N)$ in the hash count: the leader must compute every hash in order, because that sequentiality **is** the clock and cannot be parallelized away. Verifying it, by contrast, is parallel: because the leader publishes the hash value at each tick checkpoint, a verifier can split the chain at those checkpoints and hand each segment to a different core, each recomputing its segment from the published starting hash and confirming it reaches the published ending hash. With K cores a block verifies roughly K times faster than it was produced. A transaction mixed in at a given tick is thereby provably ordered after the previous checkpoint and before the next, giving a time resolution of one tick.

What ***getBlock*** returns as block metadata reflects this model rather than a mining header: the slot and block height give position, the blockhash is the slot's final PoH hash, the previous blockhash and parent slot link to the block built upon (which may skip empty slots), and the block time is an estimated wall-clock instant derived from PoH together with validators' clocks. Acceptance then separates three concerns that a proof-of-work chain fuses into one: ordering is already given by PoH, validity is checked by replaying the transactions and recomputing the bank hash (a mismatch rejects the block), and finality is voted by Tower BFT with stake-weighted votes until the block is rooted. Because production optimistically streams entries before finality, two valid blocks can briefly coexist on competing forks; Tower BFT roots one and prunes the other. This three-way

separation — order, then validity, then finality — is precisely the property the settlement design leans on, since it lets the program treat the blockhash as an ordering-and-recency token independent of the vote that will later finalize it.

That recency role is the one place the block model surfaces directly in the application. Every transaction carries a recent blockhash — a recent slot's final PoH hash — and the runtime rejects any transaction whose blockhash is older than roughly 150 slots, about a minute. This is why the throughput harnesses of Section XI.F fetch a fresh blockhash immediately before sending and re-fetch it when re-firing a dropped transaction: a settlement built against a stale blockhash is rejected before it executes, independent of whether its contents are valid. The programs' own notion of time — the oracle's fifteen-minute clearing windows and an order's expiry — is read through the runtime clock, whose value traces back to this same PoH stream.

Two caveats bound how much of this the present work exercises, and they are the same as elsewhere. The evaluated configuration is a single-node permissioned localnet: PoH runs, entries pack into blocks, and block time advances, but the node self-produces with no voting quorum, so leader rotation, fork choice, and Tower BFT finality are never triggered in development or test (the same limitation noted for consensus in Section VII.E). The mainnet-state simulation used elsewhere runs against forked state with no local validators at all. Real Tower BFT among several operator-run validators is the target deployment and is outside the scope measured here.

A single settlement transaction comprises one settle instruction together with two Ed25519 signature-verification instructions per matched pair (for the buyer and the seller) that the program verifies through the instruction sysvar, where the signature payload (signature, public key and the canonical message, totaling about 189 bytes per instruction) resides in the instruction data, not in the accounts, so the address lookup table (ALT), which compresses only the account list, cannot reduce the size of this part. For this reason, even though the program permits batch settlement of up to 4 pairs per transaction (*batch_settle_offchain_match*), the practical cap is constrained by the 1,232-byte transaction packet size: combining two pairs (4 signature-verification instructions at about 760 bytes, together with the serialized BatchMatchPair at about 370 bytes, plus the account index list and header) already exceeds the packet size. Increasing the actual number of pairs per transaction therefore requires changing how the signatures are packed (for example, pre-verified signature accounts, or an off-chain aggregated multisig), not merely increasing the number of pairs. The accounts the transaction touches comprise the buyer's and seller's escrow accounts, the order nullifier accounts of both sides, the zone_market account, and the fee/wheeling/loss collector accounts in the treasury. This structural constraint explains why the batching cap and the throughput of settlement are tied to the transaction format, which is consistent with the measurement results in Section XI.F. The exact byte layouts behind these figures are given in Section IV.K.

J. The Transaction as a Signed Message

The byte budget of the previous section is best understood from the shape of the transaction itself, because the scheduling behavior, the packet wall, and the one-match batch cap all fall out of how a transaction packs its accounts and its instruction data. A transaction is a list of 64-byte signatures over a single serialized **message**, and the message is where all the structure lives: a three-byte header, a flat ordered array of account public keys, the recent blockhash, and a list of compiled instructions. Each compiled instruction does not repeat public keys — it names its program and its accounts by **index** into the shared account array, and carries only its own opaque data. That shared, indexed account array is precisely what lets the runtime compute a transaction's write-lock set cheaply, which is what Sealevel schedules on (Section IV.F).

The header does more than count: privilege in this model is **positional**. The account array is ordered in four contiguous classes — writable signers, read-only signers, writable non-signers, read-only non-signers — and the header's three counts are the boundaries that slice the array into them. Whether an account is a signer and whether it is writable is therefore derived from **where it sits** in the array, not from a per-account flag, and the first signer is by convention the fee payer. The recent blockhash field is the transaction's time-to-live and its replay guard at once: it must name a slot within roughly the last 150, or the transaction is rejected as stale (Section IV.I), and because it is bound into the signed message a captured transaction cannot be hoarded and replayed later. This is why the harnesses of Section XI.F fetch a fresh blockhash immediately before every send and again before every re-send.

Two mechanisms in the settlement transaction deserve to be named at this level, because they are what make the design's security rest on structure rather than on trust. The first is that the Ed25519 verification of each signer is **not** a cross-program invocation but a separate native precompile instruction placed ahead of the program instruction; the settlement program reads those instructions back through the instructions sysvar and hand-parses their bytes — walking the sysvar's offset table to the precompile entry, skipping its account list, and lifting out the exact message the precompile verified. It then re-derives the identical 77-byte payload from its own arguments and checks the two match byte-for-byte. Signature validity is thereby established with no invocation at all, which is why the signature material sits in instruction **data** (and so cannot be compressed by an address lookup table, the fact that ultimately caps the batch at one match). The second mechanism is that every account the settlement touches must equal the canonical Program Derived Address for the **signed** payload — the buyer's nullifier must be exactly *find_program_address([nullifier, buyer, order_id])*, and so on for each escrow and collector — checked with an equality assertion. Address derivation is therefore the authorization: an attacker cannot substitute a victim's escrow to divert funds, nor an unrelated nullifier to bypass the replay guard, because the account addresses are cryptographically bound to the very order bytes both parties signed.

Finally, the stack ceiling of Section IV.K.8 leaves a visible mark on the message. The accounts that ride in *remaining_accounts* to keep the validation context under 4 KB — the governance config whose maintenance byte is read without deserializing its 405-byte struct, the tariff config, the optional zone-capacity account, the treasury accounts — still occupy slots in the message's account array exactly like typed accounts; what they skip is only Anchor's stack-heavy materialization during *try_accounts*, not their place on the wire. The message pays for them in bytes either way; the stack workaround buys frame space, not packet space. Address lookup tables reclaim the byte cost of those account **keys**, but as established, they cannot touch the per-match signature data that is the binding constraint.

K. Binary Data Formats and Byte Budgets

The preceding argument — that settlement is bound by transaction **bytes** rather than by compute — only carries weight if the byte figures are derived from the actual encodings rather than estimated. This subsection therefore states the binary layout of the data structures that dominate the pipeline's byte budget: the metering frame that crosses the trust boundary into the Aggregator Bridge, the signed order payload that authorizes an on-chain settlement, the settlement transaction that carries both, and the account frames the settlement leaves behind. All fields are given in declaration order.

Three distinct binary framings coexist on-chain, and conflating them is a source of error, so we distinguish them at the outset. (i) **Borsh**, the serializer Anchor uses for instruction arguments, events, and ordinary *#[account]* state, is little-endian and **packed**: fields follow one another with no alignment padding, variable-length fields carry a 4-byte length prefix, and every account is prefixed with an 8-byte discriminator derived from the hash of its type name. (ii) **Zero-copy** accounts, declared *#[account(zero_copy)]* with *#[repr(C)]*, are not deserialized at all — they are cast in place from the account's byte buffer on every access, which requires C layout rules, so explicit padding fields are load-bearing rather than decorative. (iii) The **signed off-chain payload** is a hand-rolled byte buffer built by concatenation, which happens to be byte-identical to Borsh for that struct because all its fields are fixed-size and appended in declaration order. The rule the system follows is that data parsed once per transaction (instruction arguments, signed messages, events) uses Borsh, whereas accounts read in place on every transaction (orders, meter state, shard accumulators) use the zero-copy form; conversions between the two — for instance a fixed *[u8; 32]* plus a length byte in state that surfaces as a *String* in an event — are always explicit, never a cast.

1) Metering Frame (Secure DLMS-lite v4)

Meter telemetry is admitted in two interchangeable encodings. The first is an OBIS-coded JSON document whose registers follow DLMS/COSEM (IEC 62056) object codes, for example *1.1.1.8.0.255* for active energy import and *1.1.2.8.0.255* for active energy export, both in Wh. The second — and the one that determines the wire cost — is a compact binary frame, "Secure DLMS-lite v4", whose layout is shown in Table V. It is deliberately parsed in two stages: the cyclic redundancy check and the plaintext header are validated **before** decryption, because the logical device name in the header is the identity used to resolve that device's key. A meter that is not known to the bridge is therefore rejected without any cryptographic work being spent on its ciphertext.

Table V
Byte layout of the Secure DLMS-lite v4 metering frame. The header is plaintext so that the device identity (LDN) can be resolved before decryption; the payload is an AES-256-GCM-sealed TLV block; CRC-32 covers the entire frame. Offsets are byte positions from the start of the frame; multi-byte integers are big-endian.

off	field	B	encoding
0	`version`	1	`0x04` = v4, else reject
1	`total_length`	1	u8; frame size = value + 2
2	`manufacturer_id`	3	ASCII
5	`logical_device_name`	8	ASCII, NUL-padded (meter id)
13	`timestamp`	8	u64 BE, Unix seconds
21	TLV block	*n*	AES-256-GCM ciphertext
—	GCM auth tag	16	trailing, inside the block
end−4	`CRC-32`	4	u32 BE, over the whole frame

Three consequences follow directly from this layout. First, the header floor is 25 bytes (21 header bytes plus the 4-byte checksum), which is the smallest frame the parser will consider. Second, because *total_length* is a single byte, a frame can never exceed 257 bytes, and the sealed plaintext can never exceed 216 bytes — an upper bound on how many registers one reading may carry, enforced by the encoding rather than by policy. Third, the payload is a sequence of tag–length–value triples: energy registers carry an 8-byte big-endian value (10 bytes per triple with its 2-byte header) and the instantaneous registers carry a 4-byte value (6 bytes per triple), so a reading carrying the full five-register set (import and export energy, voltage, current, and battery state of charge) occupies 38 bytes of plaintext, 54 bytes once sealed, and 79 bytes on the wire. A verified reading is thus roughly two orders of magnitude smaller than the transaction that eventually settles the energy it represents — which is why the ingest path saturates on request handling (Section XI.C) rather than on bandwidth.

The 96-bit GCM nonce is not transmitted; it is reconstructed on both sides as manufacturer identifier (3 bytes) ‖ timestamp (8 bytes) ‖ version (1 byte), which saves 12 bytes per frame but makes nonce uniqueness a property of the **timestamps**: because the key is per-device and the timestamp has one-second resolution, two distinct frames from the same meter within the same second would reuse a nonce. The metering layer's monotonically increasing reading times (and the oracle's minimum reading interval of 60 seconds) keep this condition satisfied in the tested configuration, but it is a design dependency worth making explicit rather than an incidental detail. Separately, the signed value is a canonical text form, *device_id:kwh:timestamp_ms*, over which the meter produces a 64-byte Ed25519 signature transmitted as 88 base58 characters; the canonical form must be byte-identical on both sides, so number formatting (integral values, negative zero, and the absence of scientific notation) is part of the protocol rather than a presentation choice.

2) The Ingest Trust Chain

The byte layout of Table V is the innermost of three layers a reading crosses on its way into the Aggregator Bridge, and the order in which those layers are applied — and unwound — is itself load-bearing. The meter first encodes the reading in the OBIS-keyed form and signs the canonical text *device_id:kwh:timestamp_ms* with its per-device Ed25519 key, so the signature covers the reading's economically meaningful value and travels **inside** the frame; authenticity is thereby established independently of how the frame is later carried. The signed frame is then sealed under AES-256-GCM with a **separate** per-device key. Two encodings of this seal coexist: the binary v4 frame of Table V derives its nonce from the manufacturer, timestamp, and version and so transmits no nonce, whereas the JSON *dlms-enc* envelope used on the REST path transmits a random 96-bit nonce explicitly and binds a monotonically increasing invocation counter into the authenticated additional data as *device_id:counter*. Replay resistance rests on that counter in two ways, and only the second actually refuses a resent frame: folding the counter into the GCM tag prevents its silent alteration, while — the operative guard — after a frame authenticates the bridge performs a stateful compare-and-set against a per-device counter it keeps in the cache tier, rejecting any value that is not strictly greater than the last accepted and advancing the stored value only on an authenticated frame. A replayed **identical** frame therefore decrypts but is refused at this step, a property distinct from the inner signature, which establishes only authenticity. This is the REST-path analogue of the

binary frame's reliance on strictly increasing timestamps, themselves rate-limited on-chain by the oracle's sixty-second minimum reading interval, for nonce uniqueness. The sealed frame is finally carried over a TLS transport and authenticated at the HTTP layer by an API key the bridge validates against the identity service, with a short-lived cache bounding that lookup.

The bridge unwinds these layers in reverse, and each is an independent gate that can drop the reading. The API key is checked first; the envelope is then decrypted, and under the hardened profile a frame that is not encrypted is refused outright with *426 Upgrade Required*, so there is no silent plaintext downgrade. Only after decryption does the bridge reconstruct the canonical string from the decoded fields and verify the Ed25519 signature against the device's registered public key, failing closed: an unverifiable reading is rejected rather than admitted unattributed. A verified reading is decoded from OBIS into the internal reading model, its owner resolved from the meter registry — a three-tier cache over Redis backed by the durable Postgres roster — and only then disseminated along the fan-out of Fig. 5: the zone-partitioned Redis stream that feeds the operational path, an independent time-series store for history, the message bus, and the fifteen-minute settlement bin from which a net-surplus window later mints (Section XI.G). The signing key and the encryption key are held per device and looked up separately; both lookups are cached to bound load on the identity and cache services, but the signature is verified on every reading — only the static key material is cached, never the verification verdict.

3) The Signed Mint Envelope

The authorization for a surplus generation-mint is not embedded in the on-chain transaction as settlement's is; it is lifted out into the envelope the Aggregator Bridge sends to the Chain Bridge over NATS on the *chain.tx.mint* subject. The design intent is to separate provenance from transaction construction: the Chain Bridge assembles and signs the *mint_generation* transaction with its *platform_admin* key only once the envelope has passed an application-level identity check, so envelope signing is an authorization layer that runs **ahead of** the blockchain rather than a payload the on-chain program inspects.

Signing is performed over a canonical encoding designed to be unambiguous and replay-resistant. The scheme begins with the domain tag *gridtokenx-nats-envelope-v1* terminated by a zero byte, followed by a message-kind tag (*mint*) that domain-separates each kind — a signed cancel cannot replay as a submit. Each field is then encoded as its name, a zero byte, a *u64* little-endian length, and the raw value bytes. The per-field length prefix prevents shifting byte boundaries across fields to replay a captured signature, the field order is fixed (not JSON order), and the *auth* field carrying the signature is always excluded from the signed bytes because a signature cannot cover itself. The full field layout of the *mint* envelope is given in Table VI. The encoding functions destructure every field exhaustively on purpose: adding a schema field without declaring it either signed or explicitly excluded is a compile error, which prevents a field that travels on the wire from silently escaping the signature.

Table VI

Canonical signed-byte layout of the *mint* envelope on the NATS *chain.tx.mint* subject (message kind *mint*). Each row is one field encoded as *name* + *0x00* + *u64-LE length* + *value*, emitted in signing order; the buffer is prefixed with the header *gridtokenx-nats-envelope-v1\\0mint\\0*, and the *auth* field is excluded from the signed bytes.

Field	Value encoding	Meaning
`correlation_id`	UTF-8	Per-attempt UUID; routes the reply subject
`idempotency_key`	UTF-8	`mint:{serial}:{window}`, effect-level dedup key
`reply_subject`	UTF-8	NATS subject the sender awaits the result on
`recipient_wallet`	UTF-8	Recipient wallet (base58)
`service_identity`	UTF-8	SPIFFE URI of the requesting service
`created_at_ms`	`u64` LE (8B)	Publish time (epoch ms)
`energy_kwh`	`f64` LE (8B)	Energy amount (kWh)
`meter_id`	16 raw bytes	Meter identifier (UUID)
`window_start_ms`	`i64` LE (8B)	15-minute window start (epoch ms)

The canonical bytes above are only the **input to the signature**. What actually travels on the NATS wire is the JSON envelope (*MintEnergyMessage*) carrying those fields plus an appended *auth* object (*EnvelopeAuth*): a scheme tag *ecdsa-p256-sha256-v1*, the publisher's leaf mTLS certificate (PEM), and an ECDSA P-256/SHA-256 signature encoded ASN.1-DER then base64. The signer is the aggregator, holding the same mTLS client key that anchors the gRPC trust boundary; when no certificate/key pair is present (insecure dev mode) the envelope ships unsigned and is accepted only while the Chain Bridge runs signature verification in log-only mode. The Chain Bridge verifies in three ordered steps before building any transaction: (1) the envelope certificate chains to a trusted CA; (2) the certificate's SPIFFE SAN equals the envelope's self-asserted *service_identity* field, binding the claimed identity to a CA-issued certificate; and (3) the P-256 signature verifies against canonical bytes **recomputed** from the received fields, so tampering with any field in transit — the recipient wallet or the energy amount, say — makes the recomputed bytes differ and the signature fail. Enforcement is gated by *CHAIN_BRIDGE_REQUIRE_SIGNED_NATS*, defaulting to log-only for gradual rollout. The engineering point is that signer and verifier share one canonical-bytes implementation from a common crate (*gridtokenx-blockchain-types*) rather than two hand-copied byte layouts, guarded by golden-vector tests that independently reconstruct the expected bytes to catch cross-crate schema drift.

Delivery of the envelope is engineered to be loss-free even when the blockchain or validator is transiently down. The envelope is published to JetStream and awaits its PubAck (durable stream storage) before awaiting a reply, closing the window where a consumer might be momentarily absent. A surplus settled out of a window but not yet landable on-chain is additionally **enqueued** into a durable outbox on Redis (*gridtokenx:billing:mint_outbox*, field *{serial}:{window}*), and a drain loop retries each entry until the mint confirms on-chain, only then removing it. The outbox survives a restart (reloaded from Redis), and the recipient wallet is re-resolved on every attempt, so a meter that registers its wallet **after** its window still mints on a later retry. Retries are safe because the Chain Bridge deduplicates on *mint:{serial}:{window_start_ms}* and the on-chain *(meter, window)* PDA is the final backstop — a retry of a mint that already landed but whose reply was lost does not double-mint. An entry that stays unlandable past *MINT_OUTBOX_MAX_AGE_SECS* (7 days by default) is **parked** out of the retry path rather than silently deleted, so an operator can re-enqueue it. The batch request on the *chain.tx.mintbatch* subject uses the same scheme but appends an item-count field and frames each item under its own length prefix, binding both the **number** and the **order** of recipients into the signature: adding, dropping, or reordering a recipient fails verification.

One field of the envelope crosses a representation boundary that is worth making explicit, because it is where an honest reading and

a hostile one are separated before the token is ever minted. The energy amount travels as an IEEE-754 *f64* in kWh, but the on-chain mint operates in integer base units at nine decimals (1 kWh = 10^9 base units, the energy token's precision). The conversion is a single shared function used by **both** the producer and the bridge verifier — the same single-source discipline as the canonical-bytes scheme — so the two never disagree on the bound or the scaling. It rejects any value that must never reach *mint_to*: non-finite (*NaN*/±∞), non-positive, above a hard cap *MAX_MINT_KWH* of 10^6 kWh (1 GWh, far above any honest 15-minute window yet far below overflow), and any positive value that truncates to zero base units. The cap is load-bearing rather than cosmetic: a Rust float-to-integer cast **saturates** to *u64::MAX* for a huge input rather than wrapping, so without the bound a single envelope could request an astronomically large mint; the cap keeps the scaled value near $10^{\{15\}}$, comfortably inside *u64*. The on-chain program then re-checks the floor as defense in depth — a zero amount is rejected (*ZeroAmount*, error *6011*) before any state write, since *mint_to(_, 0)* is a silent no-op that would still stamp the window's record *minted* = *true* and permanently poison it — alongside the 15-minute window-boundary check in milliseconds (*MisalignedWindow*, error *6010*) and the requirement that the REC-validator co-signer differ from the platform authority (*RecValidatorIsAuthority*, error *6012*). The exactly-once record account is checked **first**, so a replay short-circuits to a no-op before any of these guards even run.

4) Signed Order Payload and Signature-Verification Instruction

The authorization that a settlement rests upon is a fixed-size 77-byte message, not a signature over a hash of the transaction. Its layout is given in Table VII. Every field that could change the economics of the trade — the amount, the price, the side, the zone, and the expiry — is inside the signed region, so no intermediary can alter the terms after signing.

Table VII

Borsh layout of the signed order payload (*OffchainOrderPayload*). The same 77-byte sequence is what the buyer and the seller sign off-chain and what the on-chain program reconstructs and compares byte-for-byte against the message embedded in the Ed25519 verification instruction.

off	field	B	encoding
0	`order_id`	16	UUID, raw bytes
16	`user`	32	Pubkey (Ed25519 public key)
48	`energy_amount`	8	u64 LE
56	`price_per_kwh`	8	u64 LE
64	`side`	1	u8: 0 = buy, 1 = sell
65	`zone_id`	4	u32 LE
69	`expires_at`	8	i64 LE, Unix seconds
	total	**77**	fixed size, no length prefix

This message is carried to the validator inside a native Ed25519 precompile instruction whose data is a 2-byte preamble (signature count and padding), a 14-byte offsets header of seven *u16* values, the 64-byte signature, the 32-byte public key, and the message itself: $2 + 14 + 64 + 32 + 77 = 189$ bytes per signer. The offsets header is load-bearing for security, not merely for parsing: the program reads the public key and the message from the offsets **declared in that header**, and rejects any header whose fields point at a different instruction. Reading from fixed offsets instead would permit an attacker to construct a blob in which the native verifier validates the attacker's own key while the program reads a victim's key planted elsewhere in the same data — a settlement the victim never authorized. Verifying at the declared offsets forces the program and the native verifier to agree on the same triple.

5) Settlement Transaction Byte Budget

Table VIII accumulates these figures into the packet budget for the batch settlement instruction. Per matched pair the transaction must carry two verification instructions (378 bytes) plus the pair's own serialized *BatchMatchPair* — the two 77-byte payloads again, together with the match amount, match price, and a 16-byte trade identifier that keys the per-match replay guard, for 186 bytes. The signed payload is therefore transmitted **twice** per signer: once inside the precompile instruction, where the native verifier needs it, and once inside the program instruction, where the program needs it to reconstruct the message. That duplication, not the account list, is what consumes the packet.

Table VIII

Instruction-data budget of *batch_settle_offchain_match* against the 1,232-byte packet limit. Values are derived from the serialized layouts (Borsh and the Ed25519 precompile format), excluding the transaction signature, message header, and blockhash (100 bytes) and the ALT-compressed account indices (1 byte per account). Two pairs overrun the packet before those fixed costs are even added.

component	1 pair	2 pairs
Ed25519 verify ix (189 B × 2 per pair)	378	756
`BatchMatchPair` (186 B per pair)	186	372
program ix fixed part[2]	63	63
instruction data total	**627**	**1,191**
remaining of 1,232 B packet	605	41
fixed tx overhead (sig + header + blockhash)	100	100
fits in one transaction	**yes**	**no**

The result is a hard structural cap of one matched pair per settlement transaction, even though the instruction accepts a vector of matches and the 1,400,000-CU per-transaction ceiling would admit roughly fourteen pairs at the measured 96,707 CU per pair (Section XI.E). The gap between what compute permits and what the packet permits is therefore about a factor of fourteen, and it cannot be closed with an address lookup table: the ALT replaces 32-byte account keys with 1-byte indices, and the accounts a pair adds — seven per pair, plus three or four trailing accounts for the whole batch — cost only seven bytes more per pair under compression. The binding constraint is instruction **data**, which no lookup table compresses. Closing the gap requires removing the duplicated signature material, for instance by pre-verifying signatures into accounts that the settlement then references, or by aggregating both parties' authorizations into a single off-chain multisignature. This is the concrete form of the observation in Section XI.F that settlement throughput is a property of the transaction format.

The same budget shapes the telemetry path in the opposite direction. A single *submit_meter_reading* instruction carries at most 76 bytes of data (an 8-byte discriminator, a length-prefixed meter identifier of at most 32 bytes, two 8-byte energy counters, an 8-byte timestamp, and a 4-byte zone identifier), and its four accounts are identical across every reading of the same meter. Readings therefore batch: the month-long replay used in Section XI.F packs ten readings per transaction by default and asserts the serialized size against the 1,232-byte limit before sending. Where settlement is packet-bound at one unit of work per transaction, metering is packet-bound at roughly an order of magnitude more — a direct consequence of one path carrying signatures inline and the other not.

[2] 8-byte discriminator, 4-byte vector length prefix, 32-byte merkle root, 8-byte VAT amount, 2-byte VAT rate, 8-byte batch identifier, and a 1-byte settlement shard identifier.

The surplus generation-mint path sits at yet a third point on this spectrum. When a settlement window closes with net generation, the Aggregator Bridge mints the surplus through the energy-token program's ***mint_generation*** instruction, whose data is a fixed 40 bytes — an 8-byte discriminator, the 16-byte meter identifier, an 8-byte window-start timestamp in milliseconds, and an 8-byte amount — and which carries no inline signature material at all. Provenance is instead established by a mandatory registered-REC-validator co-signer at the transaction's signature level, so unlike settlement the mint neither embeds an Ed25519 payload nor pays for one in bytes. The transaction the Chain Bridge submits comprises three top-level instructions — a compute-budget instruction, an idempotent associated-token-account creation, and ***mint_generation*** — and occupies roughly 700–750 bytes on the wire (two 64-byte signatures, an 80-byte header and blockhash, about thirteen 32-byte account keys, and about 50 bytes of instruction data), comfortably inside the 1,232-byte packet. A single-recipient generation mint is therefore bound by neither packet nor compute (Section XI.G); its scaling behaviour is the inverse of settlement's, since many recipients batch into one transaction (***build_generation_mint_instructions***) and approach the packet wall only as the recipient count grows, rather than being capped at one unit of work. The authorization a mint rests upon travels not in the transaction but in the NATS request that precedes it: the aggregator signs a domain-tagged, length-prefixed canonical serialization of the request — the correlation and idempotency keys, the recipient wallet, the meter identifier, the window, and the energy amount — under a P-256 key whose certificate the Chain Bridge binds to the requester's SPIFFE service identity, rejecting a spoofed identity before any transaction is built. Exactly-once minting is then enforced on-chain by a per-***(meter, window)*** record account whose ***minted*** flag short-circuits any replay to a no-op, so a settlement retried after a crash cannot double-mint.

6) The Zero-copy Account Frame

The hot-path accounts — the ones a settlement reads or writes on every transaction — use the zero-copy framing, and the ***Order*** account in Table IX is the representative case. Its six-byte ***_padding*** field is the point of interest: the two single-byte enumerations at offsets 96 and 97 leave the cursor at 98, and without the pad the following ***i64*** would begin at an offset that is not a multiple of eight. The pad advances the cursor to offset 104, so ***created_at*** and ***expires_at*** both land on 8-byte boundaries, which is what makes the account castable in place rather than deserializable. The consequence for maintenance is that the padding must be recomputed by hand whenever a field is added — an obligation that Borsh accounts, being packed, do not carry.

Table IX

Zero-copy (*#[repr(C)]*) layout of the ***Order*** account. Offsets are fixed by C alignment rules, so the account is cast in place on every access rather than deserialized. The 6-byte pad is load-bearing: it realigns the two trailing ***i64*** fields to 8-byte boundaries.

off	field	B
0	`seller` · Pubkey	32
32	`buyer` · Pubkey	32
64	`order_id` · u64	8
72	`amount` · u64	8
80	`filled_amount` · u64	8
88	`price_per_kwh` · u64	8
96	`order_type` · u8	1
97	`status` · u8	1
98	`_padding` · [u8; 6]	6
104	`created_at` · i64	8
112	`expires_at` · i64	8
	body (+ 8-byte discriminator)	**120**

7) On-chain State Footprint

The last binary dimension is what each settlement leaves behind on-chain, summarized in Table X. The design intent visible in these sizes is that per-entity accounts stay small enough for rent to be negligible and for the write set of unrelated transactions to remain disjoint, which is what allows Sealevel to schedule them in parallel (Section VII.F).

Table X

On-chain footprint of the accounts written on the settlement and metering paths, including the 8-byte Anchor discriminator. Per-entity accounts are deliberately small; only the singleton governance account is oversized, by design, to reserve room for upgrades.

account	B	encoding · composition
`TradeNullifier`	9	Borsh; 8 + bump — existence *is* the guard
`OrderNullifier`	65	Borsh; 8 + 16 + 32 + 8 + 1 (cumulative fill)
`MeterState`	102	Borsh; 8 + 32 + 1 + 1 + 4 + 7 × 8
`SettlementRecord`	120	zero-copy; 8 + 112 (root, VAT, id, pad)
`Order`	128	zero-copy; 8 + 120 (see Table IX)
`GovernanceConfig`	413	Borsh; 8 + 405, reserved space

Two entries deserve comment. Both zero-copy accounts (***Order***, ***SettlementRecord***) carry explicit padding to a multiple of eight for the reason given above, whereas the Borsh accounts do not. The per-match ***TradeNullifier*** is nine bytes — a discriminator and a bump seed — because it stores no state at all: its mere existence, created atomically with the settlement it guards, is what makes a replayed match fail. It is the cheapest possible encoding of a replay guard, and it is distinct from the ***OrderNullifier***, which must store a cumulative filled amount because orders settle partially (Section IV). At the other extreme, ***GovernanceConfig*** is pinned at a constant 405 bytes of payload with explicit reserved space, precisely because other programs deserialize it themselves — they skip the 8-byte discriminator and decode the fields by position — so its layout is a compatibility surface. Fixing the size and reserving the tail means new governance fields can be added without migrating an account that every order-creation instruction in the system reads.

8) Which Ceiling Binds: Six Runtime Limits

The byte budgets of the preceding subsections are not isolated facts; they are three of the six hard runtime ceilings a Solana-compatible program must fit inside simultaneously, and the design of this system is legible only when all six are seen at once. Table XI lists them together with the escape each one forces. The compute-unit budget of Section XI.E is the ceiling most often assumed to bind, but in this workload it almost never does: the transaction packet size and the validation stack bite first.

Table XI

The six runtime ceilings a settlement transaction must satisfy at once, and the mechanism this system uses to stay under each. Compute units — the ceiling usually assumed to bind — is here the slackest of the three that matter; packet size and validation-stack size are the ones that actually constrain the design.

ceiling	limit	escape used here
CU budget	200k / 1.4M per ix	zero-copy cast; one-CPI batch
tx size	1,232 B (packet)	ALT — but cannot compress Ed25519 data
BPF stack	4 KB per frame	`remaining_accounts` slice, not typed field
heap	32 KB bump, no free	avoid heap growth; `Box` only to offload stack
CPI depth	4	acyclic graph, longest chain 3 hops
account data	10 MB; 10 KB realloc/ix	sharded per-entity PDAs

A compute unit is a meter of execution work — bytecode instructions, syscalls such as clock reads, SHA-256, and Ed25519 verification, cross-program invocations, and account (de)serialization — and it is distinct from the fee, which is charged separately per signature and per priority bid. The property that makes it easy to over-attribute cost to compute is that a CPI adds the **callee's** consumption to the **same** meter: invocation depth and context fatness compound on

one budget. Even so, every instruction in this system's real execution path is measured to sit well under the 200,000-CU default — the hot writes at a few thousand CU each (Fig. 12), the signature-verifying settlement near half the budget (Section XI.E) — and the LiteSVM profile tests assert each instruction stays under that default, so a handler that suddenly needs a raised budget is treated as a regression rather than accommodated. The default budget is, in effect, a health bar.

The first ceiling that actually binds is the packet size, and its analysis is exactly the batch-settlement argument of Section IV.K: because each match carries a 77-byte signed payload and a 64-byte signature inside instruction **data**, which no address lookup table can compress, the packet fills at one match per transaction long before the 1.4-million-CU ceiling — which would admit roughly fourteen — is approached. Compute is cheap here; the packet is the wall.

The second binding ceiling is subtler because it strikes before the handler even runs. Anchor materializes every account of a *#[derive(Accounts)]* context onto the current 4-KB stack frame during *try_accounts*, so a context with too many typed fields overflows validation with a build-time **stack offset exceeded** error, independent of anything the handler does. The settlement context sits at this ceiling: one more typed account field would overflow it. This is the direct cause of a design choice visible throughout the settlement code — the governance config, the tariff config, the optional zone-capacity account, and the treasury accounts all travel through *remaining_accounts*, a bare slice that costs no stack, and are validated in the handler body by owner check and PDA re-derivation on their raw bytes. That is why the read gates of Section IV.L are raw-byte reads rather than typed Anchor accounts: it is not merely an optimization but the only way the context fits under 4 KB. The cost paid is Anchor's automatic typed validation, and it is paid back explicitly with the manual owner-and-PDA re-derivation described earlier.

The remaining three ceilings shape the system more quietly. The 32-KB bump-allocated heap, which is never freed within an instruction, is the reason handlers avoid growing large vectors and use *Box* only to move an account off the stack rather than to allocate freely. The CPI depth limit of four is the reason the composition graph of Section IV.L is kept acyclic and shallow, its longest chain three hops. And the 10-MB account-size limit, with its 10-KB per-instruction growth cap, is one of the reasons state is sharded into per-entity accounts rather than accumulated into a few large ones — the same sharding that Section IV.F relies on for parallelism. The shape of the whole system is thus not a response to any single ceiling but the intersection of all six: zero-copy layout answers compute and stack together, per-entity sharding answers account size and parallelism together, the one-CPI batch answers packet size and depth and compute at once, and *remaining_accounts* answers the stack at the price of hand-written validation.

L. Composition of Programs: Cross-Program Invocation

The six on-chain programs are not independent contracts that happen to share a network; they compose. A settlement in the trading program reaches into the treasury to reconcile the cash figure, a registry mint reaches into the energy-token program to issue supply, and a governance certificate reaches into the registry to mark the underlying energy as spent. This composition is expressed through Cross-Program Invocation (CPI), the mechanism by which one program calls another within a single transaction, and its structure is what makes the invariants of Section IV hold **across** program boundaries rather than only within one.

What matters architecturally is that the system uses two different mechanisms for two different kinds of dependency, and does not confuse them. A **state-changing** dependency — one program must cause a write in another — is a CPI, and the caller must sign it. A **policy-reading** dependency — one program must consult another's state to decide whether to proceed — is deliberately **not** a CPI: the reading program validates the account's owner and its Program Derived Address, then deserializes the bytes itself. The second form avoids the compute cost and the account-passing overhead of an invocation, and reduces policy enforcement to checking the accounts already submitted with the transaction. Table XII enumerates both kinds across all six programs, and Fig. 2 shows the resulting call graph.

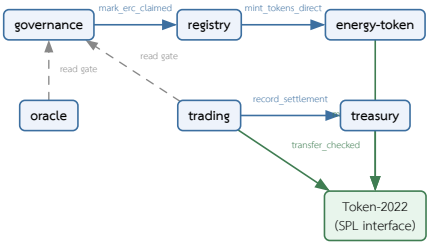


Fig. 2. Program composition across the six on-chain programs. Solid edges are Cross-Program Invocations (a state change in the callee, authorized by the caller's signing authority); dashed edges are read-only policy gates that validate owner and PDA and deserialize in place **without** an invocation. Every CPI signer is a Program Derived Address except the governance-to-registry edge, which carries the human authority's signature.

Table XII
Composition edges among the on-chain programs. The upper block lists Cross-Program Invocations with the authority that signs each one; the lower block lists read-only policy gates, which perform no invocation. SPL token transfers, mints, and burns are ordinary CPIs into the token programs and are shown once per calling program.

caller ⟶ callee	instruction	signer
Cross-Program Invocation (state-changing)		
trading ⟶ treasury	`record_settlement`	`market_authority`
trading ⟶ treasury	`record_settlement_batch_sharded`	`market_authority`
registry ⟶ energy-token	`mint_tokens_direct` (settle-and-mint)	`registry`
registry ⟶ energy-token	`mint_tokens_direct` (airdrop claim)	`registry`
governance ⟶ registry	`mark_erc_claimed`	authority
energy-token ⟶ token	`mint_to`	`token_info_2022`
trading ⟶ token	`transfer_checked` (DvP legs)	`market_authority`
governance ⟶ token	`mint_to` (REC issuance)	`governance_config`
treasury ⟶ token	`transfer_checked` — stake deposit	user
treasury ⟶ token	`transfer_checked` — unstake, rewards	`treasury`
registry ⟶ token	`transfer_checked` — stake deposit	user
registry ⟶ token	`transfer_checked` — unstake, slash	`registry`
Read-only policy gate (no invocation)		
trading ⟶ governance	`GovernanceConfig` — maintenance mode	—
trading ⟶ governance	`AggregatorEntry` — operator admitted	—
oracle ⟶ governance	`AggregatorEntry` — operator admitted	—
registry ⟶ oracle	meter totals bound registry totals	—

Two properties of this structure carry the correctness argument. First, the direction of authority follows the direction of value. Every CPI that moves value **out of** a program-controlled account is signed by a Program Derived Address rather than by a user key: the settlement's transfers out of escrow and its treasury reconciliation are both signed by the *market_authority* PDA, the registry's mints by the *registry* PDA, the energy-token program's underlying token mint by its own *token_info* PDA, and staking payouts by the *treasury* and *registry* PDAs respectively. Transfers **into** a program-controlled account — a stake deposit, an escrow funding — are signed by the depositing user

instead, which is the correct assignment: a program should need no authority to receive, and a user should retain sole authority to part with their own balance. Because a PDA has no private key and can only be signed for by the program that owns its seeds, authority to disburse is a property of the **code path**, not of a key an operator holds. Authorization of the settlement itself still comes from the buyer's and seller's Ed25519 signatures (Section IV.K); the PDA signature only executes what those signatures already authorized. The one administrative edge is governance-to-registry, where the certificate issuer's own signature is carried through, which is appropriate because issuing a Renewable Energy Certificate is a deliberate act rather than an automated one.

Second, the composition is shallow and acyclic. The three internal edges form a directed acyclic graph — governance points at registry, registry at energy-token, trading at treasury — with no back edges, so no program can be re-entered through a chain it began, which removes an entire class of reentrancy hazard by construction rather than by guard. The deepest chain is the token-issuance path — a client instruction into the registry, which invokes the energy-token program, which in turn invokes the Token-2022 program — giving an invocation depth of three against the platform's limit of four. The settlement path is shallower still: ***settle_offchain_match*** invokes the token program for the delivery-versus-payment legs and the treasury for reconciliation, both at depth two. Shallow composition matters here for the same reason thin instructions do: each nested invocation carries its own account-passing and compute overhead inside the caller's budget, so a deep chain would consume the compute headroom that Section XI.E reports as available. Keeping every value-moving path at depth two or three is what allows the measured settlement cost to remain near 48% of the per-instruction budget even though a settlement spans three programs. Every internal invocation follows the Anchor 1.0 idiom in which the ***CpiContext*** is constructed from the callee program's public key rather than its account handle — a breaking change from the 0.30 series that the codebase has adopted uniformly.

The callee's admission rules deserve their own note, because a CPI edge is only as safe as the gate on the receiving side. Two of the three internal edges carry a deliberate exemption for their sole legitimate caller. The energy-token program normally requires that a mint be co-signed by a registered Renewable Energy Certificate validator; the registry's mint path is exempt from that membership check, because those are protocol mints — an airdrop claim or a settled-generation mint — guarded by their own on-chain conditions rather than by a validator co-signature, and the registry passes its own PDA where a validator key would otherwise go, a placeholder that can never appear in the validator set. This is the one path allowed to create supply without an external co-signer, so its gate is instead that the caller **is** the registry PDA. Symmetrically, on the receiving side of the governance edge, ***mark_erc_claimed*** accepts only the registry authority as signer, and since ***issue_erc*** already forces its own signer to equal that authority, the CPI is the sole path that can satisfy the gate.[3]

Finally, this is where the conservation invariant of the energy tokens becomes enforceable across programs rather than within one. The registry may only mint energy that the oracle has accepted, because the registry's totals are bounded by the meter totals it reads; governance may only certify surplus that the registry has not already tokenized, because ***issue_erc*** invokes ***mark_erc_claimed*** in the same transaction as the certificate it creates; and the trading program may only settle tokens that exist, because the transfer legs are real token movements rather than ledger entries. The chain metered ≥ minted ≥ settled ≥ burned, with the withheld remainder certified as RECs, therefore holds not by convention between services but because each link is a CPI that fails atomically with its caller.

M. Token Model, Mint and Burn Authority, and the THBC Peg

The composition described above moves three distinct token types, and the correctness of the whole economy rests on each having a mint authority that no operator can forge and a burn that is a genuine supply reduction. Table XIII lists the three. Two points are easy to get wrong and worth stating plainly: the energy token appears under two names — GRID in its energy role and GRX in its collateral and staking role — but it is a **single** mint, not two, and the units of the three tokens differ by orders of magnitude, so every cross-token computation carries an explicit scale factor.

Table XIII
The three token types, their owning program, and their units. GRID and GRX are one mint under two role-names, not two mints. All three are Token-2022 mints; each mint authority is a Program Derived Address of its owning program, so no operator key can mint directly.

token	mint owner	dec	unit · role
GRID / GRX	energy-token	9	1 kWh = 1 GRID (kWh × 10^9); energy, and collateral as GRX
REC	governance	6	1 token = 1 MWh (kWh × 1000); renewable certificate
THBC	treasury	6	1 THB = 10^6 minor; settlement currency, THB-pegged

Each mint is gated by a condition that ties new supply to something real. GRID is minted only through the energy-token program's three paths, every one of which requires the mint to be co-signed by a registered Renewable Energy Certificate validator — with the single registry exception noted in Section IV.L — and a zero-amount mint is rejected rather than allowed to emit a phantom event against no supply. REC is minted by governance at 1,000 base units per kWh, bounded by net unclaimed generation and coupled to the ***mark_erc_claimed*** write, so certified renewable supply can never exceed the metered generation that backs it. THBC, the settlement currency, is minted only by the treasury's swap instruction under the peg guards below. Burns are symmetric and each is a real reduction: GRID is burned on consumption or compliance retirement, REC is burned to retire a certificate, and THBC is burned on redemption back to GRX.

The THBC peg is the part that most resembles a conventional stablecoin, and its safety reduces to four inequalities across two instructions. Minting THBC by swapping in GRX computes

$$\text{gross} = \lfloor \text{grx}_{\text{in}} \cdot \rho / 10^9 \rfloor \tag{3.1}$$

$$\text{fee} = \lfloor \text{gross} \cdot \varphi_s / 10^4 \rfloor \tag{3.2}$$

$$\text{net} = \text{gross} - \text{fee} \tag{3.3}$$

where ρ is the configured GRX-per-THBC rate and φ_s the swap fee in basis points, and the mint proceeds only if two guards hold: the

[3] Two further hardenings of these gates — pinning the callee to ***energy_token::ID*** on the registry side, and narrowing ***mark_erc_claimed*** to the registry authority alone (previously it also accepted the oracle authority) — exist in the working tree but are not part of the commit evaluated in this paper; they are reported here as the intended end state, not as measured behavior.

reserve attestation must be fresh — the current time minus the attestation timestamp within the configured time-to-live — and the resulting supply must not breach the reserve ceiling,

$$\text{thbc\_supply} + \text{net} \leq \text{attested\_reserve}. \tag{4.1}$$

A single minor unit over the attested reserve rejects the whole swap. Redemption runs the inverse, $\text{grx}_{\text{out}} = \lfloor \text{thbc}_{\text{in}} \cdot 10^9 / \rho \rfloor$, under two collateral guards: the burned amount may not exceed the tracked THBC supply, and the GRX paid out may not exceed what the swap vault actually holds. Together these mean that even a mis-set rate cannot let a redeemer drain more GRX than the vault contains, and cannot let outstanding THBC exceed the attested reserve — the peg holds by arithmetic at each instruction, not by an external market maker. Every product in these computations is evaluated in 128-bit arithmetic and checked on truncation back to 64 bits, so an overflow is a rejection rather than a wrap. The reserve attestation itself is refreshed by a custodian through a dedicated instruction, which is the one trusted input the peg depends on and the natural audit point.

One structural separation makes the peg robust against the system's own staking. GRX is held in four **distinct** vault accounts — swap-redemption collateral, staker custody, staker rewards, and a regulator or slash pool — each its own Program Derived Address. Staked GRX therefore lives outside the collateral that backs THBC and is never counted in the attested reserve, so a staking withdrawal cannot weaken the peg and a redemption cannot touch stakers' bonds. This also keeps the two staking systems apart: the treasury's yield staking and the registry's validator-bond staking (Section VII.F.6) draw on separate vaults, and neither one's staked balance backs the settlement currency. The result is that the token economy composes into a single closed loop — generation mints GRID under a validator co-sign, governance certifies the unclaimed remainder as REC, a producer may swap GRID-as-GRX into THBC under the peg guards, trades settle in THBC through the charge waterfall of Section IV.E with the gross value recorded in the treasury, consumers burn GRID for compliance and retire RECs, and holders redeem THBC back to GRX under the vault guards — in which every mint is gated to something real, every burn genuinely reduces supply, and the settlement currency cannot be printed beyond its reserve.

N. Renewable Energy Certificates: The Dual Representation

The Renewable Energy Certificate deserves its own treatment because it is the one instrument in the system that deliberately exists in **two** forms at once, and both are created in a single issuance. The reason is that a certificate must serve two incompatible purposes: it is a legal and audit object, which wants a unique, individually addressable record carrying a full lifecycle, and it is simultaneously a market instrument, which wants a fungible balance that can be split, traded, and retired by quantity. Rather than choose, the governance program mints both — an *ErcCertificate* account and a quantity of fungible REC tokens — atomically in the *issue_erc* instruction, so the record and the tradeable balance can never disagree about how much renewable energy was certified. Table XIV contrasts the two forms.

Table XIV
The two representations of a Renewable Energy Certificate, both created in one *issue_erc* call. The account is the legal/audit record; the fungible token is the market instrument. Their quantities are locked together by construction — one kWh of certified energy is one kWh on the record and 1,000 base units of the token.

	`ErcCertificate` PDA	fungible REC token
owner	governance program (Borsh `#[account]`)	Token-2022 mint `[rec_mint]`
nature	one record per certificate, full lifecycle	fungible balance, 6-decimal
unit	`energy_amount` in kWh	1 token = 1 MWh (1 kWh = 1,000 units)
managed by	issue / validate / revoke / transfer	mint on issue, burn on retire, trade
purpose	audit trail: status, expiry, revocation	tradeable / retireable instrument

The certificate record is intentionally verbose where the token is intentionally minimal. It carries the certificate identifier, the issuing authority and the current owner, the certified energy amount in kWh, the renewable source, free-form validation data, issuance and optional expiry timestamps, a status, a separate trading-eligibility flag with its own timestamp, and explicit revocation and transfer tracking. Consistent with the audit-document discipline noted for the metering frame in Section IV.K, every variable-length field is a fixed byte array plus a length byte rather than a *String*, so the record's size — and therefore its on-chain layout — is constant regardless of content.

The record moves through a small state machine, and the subtlety worth stating is that **validity** and **tradeability** are two independent flags, not one. Issuance sets the status to *Valid* but leaves the trading-eligibility flag false, so a freshly issued certificate is a legitimate audit object that is not yet tradeable; a separate authority action, *validate_erc_for_trading*, flips that flag and is the point at which the certificate becomes a market instrument. Transfer of the record is permitted only while it is *Valid* and validated for trading, whereas revocation is permitted while it is *Valid* or *Pending*; revocation is terminal, clears the trading flag, requires a stored reason, and cannot be applied twice. Expiry, when a certificate carries an expiry timestamp, moves it out of the tradeable set once the clock passes it. Splitting validity from tradeability this way lets the system issue a certificate the instant generation is certified while gating its entry to the market behind a distinct, separately auditable approval.

The quantity relationship between the two forms is fixed everywhere by the same factor. The fungible mint is six-decimal with one token defined as one megawatt-hour, so one kWh is exactly 1,000 base units, and that factor appears identically at the three points where the two representations meet: when *issue_erc* mints the token against the record's kWh amount, when a sell order's optional REC balance is checked against the order's kWh quantity, and when settlement transfers the REC alongside the energy. Because the same factor is used at every crossing, a certificate's recorded kWh and its fungible balance stay in lockstep by construction rather than by reconciliation.

What ultimately bounds the fungible supply is the same net-generation constraint that governs GRID. Issuance requires the certified amount to be no greater than the meter's **unclaimed** net generation, computed as generation minus consumption, minus energy already claimed as certificates, minus energy already settled — and while the governance program checks this locally to fail fast, the authoritative check is the *mark_erc_claimed* invocation into the registry (Section IV.L), which re-verifies on the same net basis and reverts the whole transaction if exceeded. Combined GRID settlement and REC issuance therefore can never exceed a meter's net generation, so total fungible REC supply is bounded to certified-and-claimed net generation, and because retirement only ever burns, that supply is monotonically non-increasing once certificates are retired. This is the certificate-side half of the conservation chain of Section IV.L: the

green attribute is minted only against real, unclaimed generation and is destroyed when surrendered.

At the market, the two representations are checked independently and both must agree. When a sell order references a certificate, the order-time gate requires the record to be *Valid*, unexpired, validated for trading, sized to at least the order's energy, and — a check the record alone does not imply — actually owned by the order's author, so that a validated certificate belonging to someone else cannot satisfy the gate. Separately and optionally, if the seller appends a fungible REC token account, it must be an account of the real governance mint, owned by the seller, and holding at least the order's kWh times 1,000 base units. At settlement the fungible REC then moves from the seller's escrow to the buyer's in the same instruction that moves the energy, so the green attribute follows the energy it certifies rather than drifting free of it.

O. Authority Catalog and Cross-Program Identities

The gates named throughout this section are enforced by a small set of stored authority keys, and it is worth collecting them in one place because the security of the whole system reduces to **who** each key is and **what** it may do. Table XV is that catalog, grouped by owning program. Two organizing principles run through it. First, the human-held keys are few and administrative — a single Proof-of-Authority key in governance, an admin key per program — while the keys that actually move value on the hot path are Program Derived Addresses, not keys any operator holds. Second, several of those PDAs are the **same identity wearing two hats** across a program boundary, which is precisely what lets a cross-program invocation carry authority without sharing a private key.

Table XV
Stored authority keys by program and what each gates. Human-held keys (the governance PoA authority, per-program admins, meter/user owners) are administrative; the value-moving authorities on the hot path are Program Derived Addresses. Aggregator admission and validator bond are enforced through per-entity accounts rather than a single stored key.

authority	kind	gates
governance — the PoA control plane		
`authority`	PoA key	issue/validate/revoke/transfer ERC, config, aggregator admit/revoke, oracle-authority set, authority-change propose
`pending_authority`	PoA key	2-step transfer target; must sign `approve`; 48 h expiry
`oracle_authority`	ref key	oracle-validation reference
`AggregatorEntry` PDAs	per-node	admitted nodes; segment 0 = retail, 1 = wholesale; read by oracle + trading
energy-token		
`authority`	admin key	mint-authority ref; add/remove REC validator
`registry_authority`	PDA	the registry PDA allowed to CPI-mint, exempt from REC co-sign
`rec_validators[5]`	co-signers	REC co-signer set; every human-driven mint needs one
registry		
`authority`	admin key	admin ops; 2-step `update_authority`
`slash_destination`	sink key	sink for slashed bonds; slash refuses until set
`UserAccount.authority`	owner key	user / meter owner; validator bond + status
trading		
`Market.authority`	PDA	`market_authority`; = treasury recorder; signs settle + record CPI
`wheeling/loss_authority`	ref keys	bind tariff rates to grid-operator authority
settlement payer	admitted	must be a governance-admitted aggregator; wholesale needs wholesale segment
`OrderNullifier.authority`	owner key	**V. the order signer (replay binding)**
treasury		
`authority`	admin key	set params, pause, refresh reserve attestation
thbc mint authority	PDA	`[treasury]` PDA mints / burns THBC
`settlement_recorder`	PDA	only caller allowed to record settlement = trading `market_authority`
oracle		
`authority`	admin key	oracle admin
`chain_bridge`	ref key	the sole key authorized to submit meter readings

The cross-program identity aliases are the heart of why this catalog is small rather than sprawling. The trading program's *market_authority* PDA **is** the treasury's stored *settlement_recorder*, so the treasury can insist that only that one PDA records a settlement, and the trading program can satisfy that check simply by signing the reconciliation invocation as itself — no shared secret, no third key. In the same way, the registry PDA is exactly the *registry_authority* that the energy-token program exempts from the REC co-sign requirement, so "a mint the registry initiated" and "a mint allowed to skip the validator co-signature" are one and the same condition rather than two that must be kept in agreement. And the governance config PDA is the mint authority of the REC token, so issuing a certificate and minting its fungible form are authorized by a single identity. Each alias collapses what would otherwise be a trust relationship between two keys into a single PDA that only one program can sign for, which is the same property that made the disbursement argument of Section IV.L hold.

Finally, the keys that **can** change hands change hands carefully. Every administrative authority that supports transfer does so in two steps rather than one — governance through a propose-then-approve handshake that the incoming authority must itself sign and that expires after 48 hours, and the registry and energy-token admins through their own equivalent two-step updates. The effect is that authority can never be moved to a mistyped or unreachable key, because the recipient must actively accept it within a bounded window; a fumbled handover simply expires and leaves the incumbent in place. For a permissioned network whose entire trust model rests on the identity of a small authority set, making the handover of that identity fail-safe is not a detail but a load-bearing part of the design.

VI. PRICING AND MARKET MECHANISM

A Continuous Double Auction clears trades transparently, with fee, seller-net, and treasury token-pricing computed at settlement.

A. Pricing and Settlement Model

The pricing model of this system is divided into three parts: clearing-price determination in the CDA mechanism, fee and seller-net computation at settlement, and the token pricing mechanism at the treasury layer. The symbols used in the equations are summarized in Table XVI.

Table XVI
Nomenclature for the pricing and settlement equations.

Symbol	Meaning
p_s	unit sell ask price
p_b	unit buy bid price
p^*	clearing price / landed cost
λ	loss factor ($\lambda \geq 1$)
c_{loss}	unit loss cost
w	wheeling charge per unit
m	incentive multiplier
δ	intra-zone discount
q	matched energy quantity (kWh)
V	total transaction value
f	market fee
φ	market fee rate (bps)
W	total wheeling charge ($q \cdot w$)
L	total loss cost ($q \cdot c_{\text{loss}}$)
r	exchange rate, GRX atoms per THBC
ψ	swap fee (bps)
A	reward accumulator per unit stake
s, s_{total}	staked amount and total staked amount
R	reward funded into the system

Clearing-price determination (CDA clearing) uses the seller-side price (maker price) adjusted by network costs. The per-unit loss cost is defined from the loss factor $\lambda \geq 1$ as in the equation

$$c_{\text{loss}} = p_s(\lambda - 1) \tag{5}$$

The clearing price, or landed cost, is computed from the sell ask price p_s plus the wheeling charge w and the loss cost, then adjusted by the incentive multiplier m and the intra-zone discount δ

$$p^* = (p_s + w + c_{\text{loss}}) \cdot m \cdot \delta \tag{6}$$

where $\delta = 0.95$ when buyer and seller are in the same zone, and $\delta = 1$ when crossing zones. An order can be matched when $p^* \leq p_b$ (where p_b is the buy bid price), and the system orders sellers with the lowest landed cost to be matched first under price-time priority. The matched quantity equals the smaller of the two sides' remaining amounts, $q = \min(q_b, q_s)$.

The fee and seller-net computation at settlement defines the total value, the market fee, and the seller's net amount as follows

$$V = q \cdot p^* \tag{7.1}$$

$$f = \frac{V \cdot \varphi}{10000} \tag{7.2}$$

$$\text{net} = V - f - W - L \tag{7.3}$$

where φ is the market fee in basis points (the on-chain default is 25 bps, or 0.25%), $W = q \cdot w$ is the total wheeling charge, and $L = q \cdot c_{\text{loss}}$ is the total loss cost. The amounts f, W, L, and net are transferred separately to the fee-collector, wheeling, loss, and seller accounts respectively. The zone parameters are interpreted in bps, namely $m = m_{\text{bps}}/10000$ and $w = w_{\text{bps}}/10000$, where the default wheeling charge equals 0 intra-zone and 0.02 cross-zone, while the loss factor equals 1.01 intra-zone and 1.03 cross-zone.

To illustrate the use of the equations above, consider an example of intra-zone matching with sell ask price $p_s = 4.00$ baht per kWh, quantity $q = 10$ kWh, and the intra-zone defaults ($\lambda = 1.01$, $w = 0$, $m = 1$, $\delta = 0.95$, $\varphi = 25$ bps). From Equation 5 we obtain $c_{\text{loss}} = 4.00(1.01 - 1) = 0.04$, and from Equation 6 the clearing price $p^* = (4.00 + 0 + 0.04) \cdot 1 \cdot 0.95 = 3.838$ baht per kWh, which can be matched when the buy bid price $p_b \geq 3.838$. The total value is then $V = 10 \cdot 3.838 = 38.38$ baht, the fee $f = 38.38 \cdot 25/10000 \approx 0.096$ baht, the total wheeling charge $W = 0$, and the total loss cost $L = 10 \cdot 0.04 = 0.40$ baht, giving the seller a net amount of $\text{net} = 38.38 - 0.096 - 0 - 0.40 \approx 37.88$ baht. The actual computation in the code uses floor-rounded fixed-point integers, so the resulting values may differ from this example at the rounding-fraction level.

The token pricing mechanism at the treasury layer covers the exchange between GRX and the THBC stablecoin pegged to the baht, using the rate r (number of GRX atoms per THBC) and the swap fee ψ (bps)

$$\text{thbc} = \frac{g \cdot r}{10^9} \cdot (1 - \psi/10000) \tag{8.1}$$

$$g = \frac{\text{thbc} \cdot 10^9}{r} \tag{8.2}$$

where redemption per Equation 8.2 incurs no fee, and the peg is maintained using a 1:1 supply-to-reserve condition, namely $\text{supply}_{\text{thbc}} \leq \text{reserve}_{\text{attested}}$. Staking rewards use a MasterChef-style accumulator, where a staker's accrued reward is computed from the staked amount s

$$\text{reward} = s \cdot A/10^{12} - \text{debt} \tag{9}$$

When a reward R is funded, the accumulator A is updated to $A \leftarrow A + (R \cdot 10^{12})/s_{\text{total}}$ pro-rata by stake share, and a slash deducts the requested amount but not more than the staked principal (capped at principal), then redistributes it back to the remaining stakers through the same accumulator.

On the production CDA path, the incentive multiplier is set to $m = 1$; that is, the incentive multiplier takes effect only on the feed-in or grid-export settlement path, not on CDA matching. In addition, the on-chain default of the market fee (25 bps) differs from the default in the off-chain configuration file (50 bps), and the zone parameters in the governance program are stored at ×1000 scale, while the consumer of the actual values in the trading layer interprets them in bps ($/10000$), which is the value the system actually uses. The computation in the code uses floor-division fixed-point integers, and the seller net uses checked arithmetic that rejects a transaction when the total fees exceed the matched value (with a network fee ceiling of 20%) instead of clamping the amount down to zero. Therefore the equations above constitute a real-value model that may differ from the actual computed result at the rounding-fraction level.

B. Sensitivity of Seller Net and P2P Trading Uplift

To show the economic implications of the pricing model above, this section analyzes the sensitivity of the seller's net amount to the zone parameters and the fee. This is a purely model-derived computation from the settlement equations in Equation 6 and Equation 7.3, not a measurement of revenue from running the real system. We fix the base sell ask price $p_s = 4.00$ baht per kWh, the matched quantity $q = 10$ kWh, and the incentive multiplier $m = 1$ (per the real CDA path), then vary the intra-zone/cross-zone state, the fee rate φ, and the loss factor λ as in Table XVII.

Table XVII
Model-derived seller-net sensitivity computed from Equation 6 and Equation 7.3 at $p_s = 4.00$ ฿/kWh, $q = 10$ kWh, $m = 1$. "net" is the seller's settled amount; wheeling (w) and loss (λ) are pass-through to the buyer via the landed price p^*.

Scenario	δ	w	λ	φ (bps)	p^*	net (฿)	net/kWh
S1 intra-zone	0.95	0	1.01	25	3.838	37.88	3.79
S2 cross-zone	1	0.02	1.03	25	4.14	39.9	3.99
S3 intra-zone, high fee	0.95	0	1.01	100	3.838	37.6	3.76
S4 cross-zone, high loss	1	0.02	1.05	25	4.22	39.89	3.99

From Table XVII, two important patterns are visible. First, the seller's net amount barely changes when the wheeling charge w or the loss factor λ is increased (compare S2 with S4, which differ at the level of fractions of a satang), because both costs are added into the landed price p^* paid by the buyer and then split off to the wheeling-collector and loss accounts. They are therefore pass-through costs that do not directly reduce the seller's amount, so the seller always receives an amount close to $q \cdot p_s$. Second, the variables that significantly affect the seller's amount are the intra-zone discount δ and the fee rate φ: intra-zone matching ($\delta = 0.95$) reduces the seller's amount by about 5% to incentivize local consumption, while raising the fee from 25 to 100 bps (S1 ⟶ S3) reduces the net amount only slightly.

Compared with a flat feed-in tariff for surplus power under a hypothetical citizen-solar program assumed at around 2.20 baht per kWh, the seller's per-unit net amount in the P2P market (approximately 3.76–3.99 baht per kWh) represents an uplift of about 72–81%. At the same time, the buyer still pays a landed price p^* (approximately 3.84–4.22 baht per kWh) lower than the usual retail electricity price, so gains from trade arise on both sides. Here the 2.20-baht purchase rate is merely a hypothetical reference value for comparison, not a value measured from a real market.

A limitation of this analysis is that the numbers in Table XVII are model-derived results under fixed parameters and a single matched quantity (10 kWh). Moreover, the actual computation in the code uses floor-rounded fixed-point integers, so the resulting values may differ at the rounding-fraction level, and this is not yet a measurement of revenue distribution under a full-scale simulated workload, which is left as future work (see Section XII).

C. Continuous Double Auction Matching

The market mechanism uses Continuous Double Auction (CDA) matching that orders by price-time priority and accounts for the topological constraints of the power network. Sell orders are partitioned into a zone-segmented order book, where each zone stores sell orders in an ordered structure (BTreeMap) keyed by *(price, created_at, id)*, so that the key ordering prioritizes the lowest price first, then the earlier order-creation time, and uses the order id as the final tie-breaker, thereby yielding price-time priority implicitly without re-sorting. Orders whose remaining quantity falls below the minimum threshold (MIN_TRADE_AMOUNT) or that have expired are not inserted into the book.

For each buy order, the matching engine gathers candidate sellers only from zones from which the network can deliver energy to the buyer's zone, via two-stage topology pre-filtering: the first stage checks at the minimum quantity to immediately exclude zones that cannot reach each other, and the second stage re-checks at the actual matched quantity (*can_accommodate_flow(sell_zone, buy_zone, amount)*) to enforce the transmission-line capacity ceiling. Within each reachable zone, a range query is used to retrieve only sell orders whose ask price does not exceed the bid price, then the landed cost is computed per Equation 6 (including wheeling, loss cost, multiplier, and the intra-zone discount $\delta = 0.95$). Only sell orders whose landed cost does not exceed the bid price ($p^* \leq p_b$) pass through as candidates. The system prevents self-trade by skipping pairs where the buyer and seller are the same user.

Once candidates from all reachable zones are obtained, the system consolidates the list and sorts by landed cost from low to high, so the buyer gets the cheapest landed total price first, regardless of which zone that sell order is in. It then matches progressively, with the per-pair quantity equal to the smaller remaining amount of the two sides ($q = \min(q_b, q_s)$) as a partial-fill, and immediately removes sell orders whose remaining quantity falls below the threshold from the book. For Fill-or-Kill (FOK) buy orders, the system first checks whether the total candidate quantity is sufficient for the entire order; if not, it does not match at all. In addition, there is match consolidation when the same buyer-seller pair and the same price occur consecutively, to reduce the number of settlement records that must be sent on-chain. Each match result records the match price (landed cost), wheeling charge, loss cost, and source/destination zones, before sending the matched pairs to atomic settlement per Section IV. The on-chain processing cost of the settlement path fed by the matching engine is reported in Section XI.E, while the matching throughput of the matching engine in the in-memory layer is reported in Section XI.D.

VII. SYSTEM DESIGN AND IMPLEMENTATION

A microservice realization: the Aggregator Bridge verifies Ed25519-signed readings, the CDA matches, and the Chain Bridge settles over a NATS JetStream queue — decoupling ingest, validation, and on-chain recording.

A. Architecture Overview

The system architecture is designed to support the simulation of Peer-to-Peer energy trading. This work presents a microgrid model that divides the data path into the Smart Meter, Aggregator Bridge, Trading Service, Anchor Smart Contract Program, Settlement Engine, and Frontend Dashboard layers. Energy generation and consumption data are produced by the Smart Meter Simulator and then sent into the Aggregator Bridge to be screened before being converted into transaction instructions for the Anchor program on a permissioned Solana-compatible network. The decomposition of the system into off-chain and on-chain domains, together with the primary data paths and the trust-boundary crossing point, is summarized in Fig. 3.

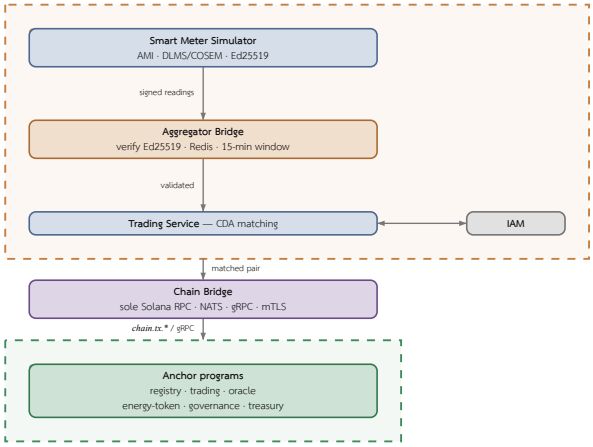


Fig. 3. System architecture and the off-chain/on-chain trust boundary. The off-chain domain (orange, dashed) verifies Ed25519-signed meter telemetry, aggregates it, and matches orders via CDA; the on-chain domain (green, dashed) settles and audits the verified pairs. Chain Bridge is the sole crossing between the two — the only service touching Solana RPC, writing via NATS ***chain.tx.**** and reading via gRPC over mTLS.

This decomposition ensures that electrical-engineering validation and access control occur before transactions are recorded, while the Anchor program acts as a verifiable rule-enforcement layer that records state for retrospective auditing. The Settlement Engine, in turn, manages signed transactions, tracks outcomes from the transaction signature, and reads the event log to bring state back for display on the Frontend.

B. Backend Services

The Backend layer acts as the intermediary between the Smart Meter Simulator (see Section VII.C), the Aggregator Bridge, and the Smart Contract. It is responsible for collecting data, validating correctness, queuing transactions, and dispatching instructions to the blockchain only after the data has passed the grid-stability conditions. The design adopts a Microservice approach to split the workload into smaller sub-workloads, allowing the system to scale only the parts with high request volume and to limit the impact when any single service fails. Details of the inter-service communication channels (ConnectRPC over mTLS and NATS JetStream) and the associated trust assumptions are described in Section III. The main components of the Backend layer comprise:

- IAM Service: manages identity, on-chain data access rights, and user roles such as Prosumer, Consumer, and Operator.
- Aggregator Bridge: receives electricity generation and consumption data from the Smart Meter, validating the data format, reference time, device identifier, and energy values before forwarding them to the analysis stage.
- Trading Service: validates and matches energy trade orders together with grid-stability conditions such as remaining energy quantity, network constraints, and Smart Meter connection status.
- Chain Bridge: the sole intermediary that dispatches instructions to the Solana Program and tracks results from the transaction signature or event log; no other service calls Solana RPC directly.
- Notification Service: sends notifications, such as Email and Alerts, when a trade order or settlement status changes.

This separation of services helps the system better support large volumes of real-time Smart Meter data, since the data-ingestion service can scale its instance count without affecting the transaction-validation service or the blockchain-connection service. In addition, the Backend layer serves as the security control point before transactions are recorded to the Smart Contract, so that the system does not rely on the blockchain alone but integrates electrical-engineering validation with user access control.

C. Grid and Meter Simulation

The Grid Modeling is designed to simulate complex electrical systems, Distributed Energy Resources (DERs), and the operation of a Virtual Power Plant (VPP), thereby enabling highly accurate simulation of Advanced Metering Infrastructure (AMI) and grid management. The core of the simulation system integrates Physics-validated State Estimation in real time and Optimal Power Flow (OPF) using Pandapower, which enables the system to generate deterministic meter data for large electrical grids.

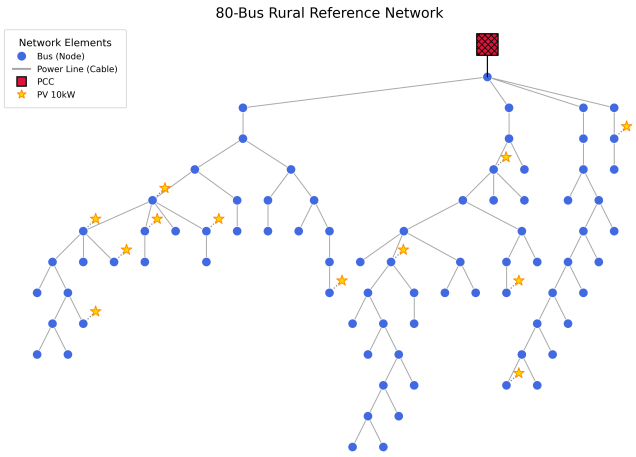


Fig. 4. Grid bus network topology used by the simulator.

The Grid Simulator is developed with Python 3.11 [28] to simulate the electricity-consumption behavior of a Distribution Grid. It uses the Network Topology format from GridLAB-D GLM files [29] and converts it into a grid model consisting of bus, line, load, and photovoltaic units, as shown in Fig. 4. The main tools used include FastAPI [30], NetworkX [31] for representing the topology structure, pvlib [32] for simulating photovoltaic generation, and pandapower [33] for computing power flow.

The simulation process operates in discrete time-steps, where each round consists of two stages: generating readings at the meter level (see Section VII.C.1), followed by solving the network state at the grid level (see Section VII.C.2). In addition to synthetic readings, the system can also replay real telemetry data from CSV files and associate the data with a bus through the meter registry, thereby supporting hybrid simulation between measured data and synthetic data. The correctness of the simulator is verified through a test suite covering topology loading and meter-to-bus mapping.

1) Smart Meter Simulator

For each bus in the topology, the system generates a meter population according to the proportions of user types, where each meter (SmartMeter) is composed of modular sub-device models, namely the Load, the Solar (photovoltaic) panel when installed, and an optional Battery Energy Storage System (BESS). Electricity-consumption behavior is simulated with a voltage-dependent ZIP load model, in which the real power drawn from the network depends on the per-unit voltage according to the equation

$$P(V) = P_{\text{base}}(Z \cdot V^2 + I \cdot V + P), \quad Z + I + P = 1 \tag{10}$$

where the Z (impedance) component varies with V^2, the I (current) component varies with V, and the P (power) component is constant. The meters in this evaluation set the impedance fraction at 0.20 (*ZIP_IMPEDANCE_FRACTION=0.20*) and clamp the voltage into the range $[0, 1.5]$ per unit before computation. The generation from the photovoltaic panels is computed with pvlib [32] using the Ineichen clear-sky model together with the PVWatts model for direct-current power, then derated by a weather derate factor — for example, cloudy multiplied by 0.42 and partly cloudy multiplied by 0.72. If pvlib is unavailable, the system falls back to a functional generation profile as a backup.

To make the simulation deterministic, each meter draws noise from its own dedicated stream by seeding its random number generator from the XOR between the system-wide seed and the SHA-256 digest of the meter identifier, so that adding or removing a single meter does not shift the readings of other meters. The result per round is an energy reading record (EnergyReading) that holds the energy produced, the energy consumed, the surplus energy, and the deficit energy as non-negative kWh values, together with the interval length of that round. Before export, each meter has a unique Ed25519 identity (per-meter key) generated deterministically from the SHA-256 of *secret:meter_id*, and signs a canonical string of the form *device_id:kwh:timestamp_ms*, exported as a base58 signature, allowing the Aggregator Bridge to verify the provenance of each reading point-by-point without keeping the meters' secret keys centrally.

2) Grid Simulation

In each time tick, once the readings of all meters have been collected, the system computes the net power injection of each bus from the difference between generation and consumption, then solves the power flow to update the network state, namely the per-unit voltage of each bus, the line flow, the power loss, and the line utilization. The main solver uses pandapower [33] with a backward/forward sweep (BFSW), which is an accurate AC power flow solver for radial distribution networks; and when pandapower is unavailable or the computation does not converge (non-convergence), the system falls back to an approximate DistFlow solver as a backup so that the simulation can continue.

In addition to solving the basic power flow, the grid simulation layer also simulates voltage-control mechanisms and abnormal events of the distribution network, namely the volt-watt and volt-VAR response of inverters, on-load tap changer (OLTC) adjustment, fault injection at a line, bus, or transformer, and tie-switch operation to transfer load, so that the dataset sent to the Aggregator Bridge reflects the physical behavior of the network under a variety of operating and abnormal conditions, rather than merely independently randomized energy values. That said, the above grid-physics capabilities (volt-VAR, OLTC, fault injection, and tie-switch) are part of the simulation suite but are not yet used as variables in the evaluation reported in this article, which measures only the ingest rate of signed readings, the on-chain settlement cost, and the matching rate (see Section XI). Evaluating the impact of these grid-stability conditions on matching and settlement is therefore left as future work.

To conform to industry standards and to certify data provenance, the Aggregator Bridge receives high-frequency real-time meter data and disseminates it through zone-partitioned Redis Streams for dynamic monitoring of network state, while aggregating readings into 15-minute time windows for use in the assessment of generation capacity and dispatch in the Backend layer. The meter data format follows the DLMS/COSEM standard (IEC 62056), decoding the binary payload portion with AES-256-GCM and re-publishing it in JSON form. In addition, the system guarantees cryptographic non-repudiation by verifying asymmetric Ed25519 cryptographic signatures, both for individual readings and in batches, before admitting data into the system. That said, in the current prototype there is not yet a path to construct an on-chain Settlement Attestation directly from the metering layer, which is an avenue left open for future work. The data-dissemination path of the Aggregator Bridge is summarized in Fig. 5.

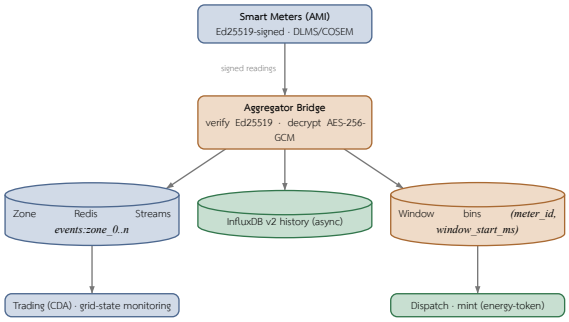


Fig. 5. Aggregator Bridge telemetry dissemination: verified readings fan out to zone-partitioned Redis Streams (real-time), an InfluxDB history sink (async, fire-and-forget), and 15-min aggregation windows keyed by *(meter_id, window_start_ms)*. Each sink sits above the consumer it feeds; the InfluxDB history sink is terminal.

D. Frequency-Driven Flex Dispatch

The aggregation windows described above serve a second consumer besides settlement: a demand-response path that turns the metering fleet into a dispatchable resource. The design point worth stating is that **the fleet is its own frequency sensor**. There is no external SCADA feed and no separate grid-state oracle; the frequency signal that triggers dispatch is extracted from the same signed meter frames that the ingest path already verifies, which means demand response inherits the provenance guarantees of Section III rather than introducing a second, unauthenticated source of truth about grid state.

Each reading carries a frequency register decoded from its DLMS/COSEM payload. The zone ingester feeds these samples into a rolling window whose length defaults to 60 seconds, discarding physically implausible values — those below 40 Hz or above 70 Hz — so that a corrupt or misconfigured meter cannot drag the fleet mean. A publisher task converts the window mean into a grid-status event on the message backbone at a default interval of 30 seconds, decoupling the sampling rate of the fleet from the evaluation rate of the dispatch logic.

A listener feeds each published frequency to the dispatch engine, which applies a symmetric deadband about the 50 Hz nominal of the Thai system: a mean below 49.8 Hz is under-frequency and calls for load reduction or injection (FLEX_UP), a mean above 50.2 Hz calls for the opposite (FLEX_DOWN), and the dispatched quantity defaults to 100 kW. All three values are configuration rather than constants. Two guards keep the loop stable. First, the engine refuses to dispatch when no aggregation window has completed, so a claim of available flexible capacity is never made before telemetry has

actually established it. Second, repeat suppression is tracked per pair of action and target rather than per most-recent action, with a cooldown defaulting to 900 seconds — exactly one 15-minute aggregation window. This distinction is deliberate and load-bearing: were only the latest action tracked, an oscillating frequency could reset the timer by alternating directions and emit one event per published grid-status message. Because each direction holds an independent timer, a genuine reversal still reacts immediately while a sustained or oscillating excursion produces at most one dispatch per window. A cooldown begins only on a successful dispatch.

Dispatch is delivered through adapters that may be enabled together, in which case a single excursion fans out to every configured target at once — for example an in-mesh controller and a utility-operated node — with partial-failure isolation, so that one failing target neither suppresses the others nor, unless all targets fail, marks the dispatch as failed. Three adapters exist: a ConnectRPC channel to edge controllers; an IEEE 2030.5 DERControl adapter, which in the present prototype is a simulation stub that logs without actuating; and an OpenADR 3 [34] adapter that acts as the business-logic side against a Virtual Top Node (VTN), emitting each excursion as an event carrying a signed-kilowatt setpoint whose sign encodes the direction. A complementary listener implements the VEN side, consuming setpoint events from an upstream utility VTN, which allows the platform to appear as a dispatchable resource to an external operator rather than only as a dispatcher of its own fleet.

This path is implemented and wired, but it is not driven as an independent variable in any experiment reported here, and the IEEE 2030.5 adapter does not actuate. What the design demonstrates is therefore the integration argument — that verified metering telemetry can carry the grid-state signal that drives standards-compliant demand response, without a second trusted sensing path — rather than a measured demand-response outcome. Evaluating dispatch latency, the stability of the deadband under realistic frequency traces, and the accuracy of delivered against requested flexibility is left as future work (see Section XII).

E. Consortium Network

The blockchain network in this system is designed as a Consortium Network under the assumption of Proof of Authority (PoA) participation governance, as described in Section IV; this section therefore focuses primarily on the details of the network design and its governance. The design ensures that block validators are entities that have a duty to verify, or have obtained consent from, the DSO — for example, a Load Aggregator, a regulatory body, or an authorized organization. The main reason for choosing PoA is network governance that is easier to control than public networks such as Solana Mainnet or Ethereum, and its suitability for regulatory-compliance requirements and cost predictability in experiments or deployments in energy systems that must estimate transaction costs in advance [15], [35]. The important point to emphasize is that PoA here is a governance and admission-control layer, not a replacement for the Layer 1 consensus mechanism of the Solana-compatible network, which still relies on Proof of History (PoH) together with Tower BFT for ordering and announcing block finality, as described in Section IV.

Permission and governance policies are enforced through the on-chain governance program, which covers three main mechanisms. The first mechanism is governing the primary authorized entity (the PoA authority) through a two-step handover, where propose_authority_change lets the current authority set a pending authority together with an expiry time, and approve_authority_change must be called by the pending authority itself to accept the transfer — so that authority cannot be handed to an unconsenting or erroneous account, and only one pending change request is allowed at a time. The second mechanism is controlling the allow-list of Aggregators authorized to sign readings, via admit_aggregator and revoke_aggregator. The third mechanism is DAO voting via create_proposal, cast_vote, and execute_proposal for adjusting the economic parameters of each zone (zone_config), namely the incentive multiplier, wheeling charge, loss factor, and maintenance mode.

The voting uses weighting by the cumulative generation of meters (stake-by-generation), where a voter's weight is computed as $w = \max\left(100, \frac{\text{total generation}}{1000}\right)$ — that is, every 1,000 kWh of cumulative generation is equivalent to one weight unit, with a minimum of 100 so that small participants still have a voice. Double-voting is prevented using a PDA vote record account per (proposal, voter) pair, one account per vote. When the voting period ends, execute_proposal tallies the result automatically: a proposal Passes only if the total votes reach the minimum quorum threshold (min_quorum_votes set in poa_config) and the votes in favor exceed the votes against; otherwise it is Rejected, thereby preventing parameter changes by too few voters.

The difference from the Solana public mainnet is that this network is not open to external validators joining permissionlessly, and it can define permission policies, program upgrades, and data retention according to the project's requirements. However, the execution layer still references the Solana Virtual Machine architecture and the Sealevel parallel runtime so that transactions not using the same account can be processed in parallel. The slot time near 400 ms and the compute budget mentioned in this article are design targets referenced from the Solana documentation, not measured results of the permissioned network. The separation of roles between the PoA governance layer (admission control) and the Layer 1 consensus layer — which still relies on PoH for time ordering and Tower BFT for voting and announcing finality — is shown in Fig. 6 [26].

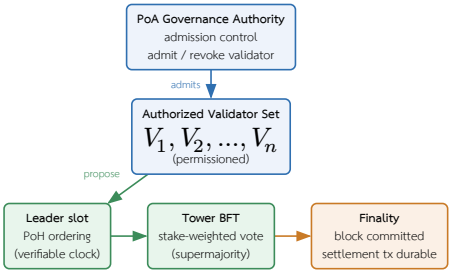


Fig. 6. Separation of concerns in the consortium network (illustrative): the PoA layer governs admission control over an authorized validator set, while Layer 1 consensus is unchanged — PoH provides verifiable time ordering and Tower BFT provides stake-weighted voting and finality. Roles, not measured throughput.

At this network level, at least five items must be defined before real deployment, namely the number of validators and the entities owning the nodes, the method of binding identity to a validator key, the policy for adding or removing validators, the process for upgrading programs or the genesis/configuration, and the acceptable fault model. This article therefore treats the PoA Solana-compatible network as an architectural assumption of the simulation system, not

as a conclusion that Solana's public-network consensus has been modified.

F. Smart Contract Programs

The Smart Contract is divided into several programs by responsibility, namely registry, trading, oracle, energy-token, governance, and treasury, as well as programs for performance benchmarking (blockbench and tpc-benchmark), developed with the Anchor Framework to systematically define account validation, instruction handlers, and the event log. The registry program has register_user and register_meter for registering users and Smart Meters. The trading program has create_sell_order and create_buy_order (as well as submit_limit_order and submit_market_order) for submitting sell and buy energy orders, match_orders and clear_auction for matching the trade orders proposed by the Backend, and settle_offchain_match and execute_atomic_settlement for settling transactions as an atomic settlement, where settle_offchain_match verifies the Ed25519 signatures of both buyer and seller and uses an order nullifier to prevent replay. The energy-token program has mint_generation and mint_to_wallet for creating energy tokens that must be co-signed by the REC certifying authority (Renewable Energy Certificate), where mint_generation uses the key (meter_id, settlement window) to guarantee idempotency per time window, and burn_tokens for burning energy tokens through SPL instructions. In addition, the oracle program has submit_meter_reading for receiving readings from the AMI, trigger_market_clearing for setting the boundary of the 15-minute time window (900 seconds), and aggregate_readings for aggregating the counters of readings that pass and fail validation. The timeline of the aggregation window and market clearing is shown in Fig. 8. The governance program acts as a PoA control layer for issuing and revoking Renewable Energy Certificates (REC), governing authority permissions, managing the aggregator allow-list, and DAO voting (the account structure and control-layer roles are described in Section VII.F.2; the network-governance details are in the Consortium Network section). The treasury program manages staking of GRX tokens, reward payouts, and maintaining the peg of the THBC stablecoin through instructions such as stake_grx, unstake_grx, claim_rewards, swap_grx_for_thbc, redeem_thbc_for_grx, and record_settlement, which is called via Cross-Program Invocation (CPI) from the trading program. The blockbench and tpc-benchmark programs are used to run standard workloads, namely the BlockBench microbenchmark, YCSB, SmallBank, and TPC-C, for performance evaluation on the Solana-compatible network [25], [36]. The on-chain program IDs of the six core programs, declared with *declare_id!*, are summarized in Table XVIII.

Table XVIII
On-chain program identities (Anchor *declare_id!*) of the six core programs on the Solana-compatible consortium network. These deterministic addresses pin the deployed artifact for reproducibility.

Program	Program ID (**declare_id!**)
registry	`FcSd5x4X1nzJMKLZC4tMZXnQ1ipLrGsEfeoH8N4mvJX7`
trading	`CnWDEUhTvSixeLSyViWgAnnu9YouBAYVGcrrFm1s9WcX`
oracle	`64Vgos61STZ8pW9NnHi2iGtXMTQr7NqBoMorK6Zg8RJU`
energy-token	`6FZKcVKCLFSNLMxypFJGU4K14xUBnxNW9VAuKGhmqjGX`
governance	`FokVuBSPXP11aeL7VZWd8n8aVAhWqVpyPZETToSxdvTS`
treasury	`FfxSQYKUmx9NGdCC9TDPmZSYjWYE1h4ruu3JatzHN5Tn`

The account structure uses Program Derived Addresses (PDA) to separate the system state into sub-accounts keyed by specific seeds, such as registry and user account, meter account, market and market_shard, order and trade record, escrow, order nullifier for replay prevention, oracle_data, and mint authority (gen_mint, thbc_mint). Using PDAs allows the program to verify the ownership and deterministic address of an account without relying on the private key of each state account. Resource-intensive work — such as complex price computation, matching large order books, or grid-stability analysis — is therefore offloaded to the Backend and the Aggregator Bridge before the verified results are submitted to the Smart Contract, so as to stay within the compute constraints of a transaction [27], [37]. The order-matching mechanism in the Trading Service uses a Continuous Double Auction (CDA) algorithm with price-time priority that partitions the order book by zone and accounts for energy-flow constraints (wheeling and loss) before sending the matched order pairs for atomic settlement on the Smart Contract. The pricing and fee formulas are described in Section VI.A. The relationships among the six programs are of two kinds: Cross-Program Invocations (CPI) that write state, and control-plane reads in which one program reads another program's account to determine operation rights without invoking a CPI, as summarized in Fig. 7.

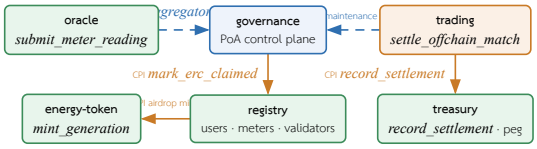


Fig. 7. Inter-program relationships among the six Anchor programs. Solid arrows are Cross-Program Invocations that write state (governance⟶registry on REC issue, registry⟶energy-token on airdrop, trading⟶treasury on settlement). Dashed arrows are control-plane reads with no CPI: trading reads ***governance*** for the maintenance gate and REC validity, and oracle validates an admitted-aggregator entry before accepting a reading. Governance (blue) is the policy source; trading (orange) initiates on-chain settlement.

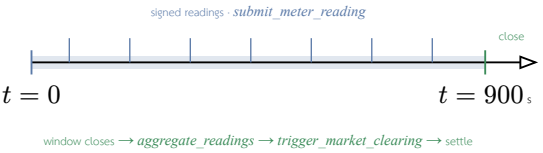


Fig. 8. Market-clearing timeline of the oracle program: signed meter readings arrive continuously over the 15-minute (900 s) aggregation window; at window close the oracle aggregates readings, triggers clearing, and the matched pair is settled on-chain. This is the on-chain telemetry path as designed and as benchmarked in isolation (Section XI.H.4); the deployed pipeline does **not** write readings on-chain — it terminates ingest in the Redis streams and window bins of Fig. 5 and reaches the chain only at mint and settlement.

1) Trading: On-chain Order Lifecycle and Escrow

The trading program manages the lifecycle of trade orders and the on-chain escrow accounts, complementing the settlement path in Section IV. Users create orders via create_sell_order and create_buy_order (as well as submit_limit_order and submit_market_order), which create a per-order PDA Order account bound to the owner, with the order expiry (expires_at) set to 24 hours (86,400 seconds) from the creation time. A sell order must reference a REC certificate that has not yet expired before it can be created, consistent with the REC governance in Section VII.F.2. Before entering the market, the user deposits tokens into the escrow account via deposit_escrow and withdraws the unused portion via withdraw_escrow, where the escrow account is an SPL token account under a PDA signed by the market_authority.

The program exposes two settlement entry points, and it matters which one a deployment uses. ***settle_offchain_match*** is the non-custodial form analyzed throughout Section IV: each party's funds sit in that party's own escrow PDA, and every account the instruction

touches is required to equal the canonical PDA derived from the payload the parties signed. *execute_atomic_settlement* is a custodial variant in which the escrow accounts are pooled associated-token accounts owned by the platform key (which is also the market authority), so a single platform signature satisfies both escrow-authority slots and the parties hold no escrow of their own. The deployment measured in this article routes its settlements through the custodial variant, because the prototype onboards users with platform-managed wallets rather than self-custodied keys. The consequence is stated plainly: for that path the guarantee that address derivation binds funds to the signing party (Section IV) does not hold, and users must additionally trust the platform key not to disburse from the pool outside a matched settlement. Moving the deployment onto the non-custodial *settle_offchain_match* path — for which the per-party escrow PDAs and the funding instruction already exist — is the pay-down, and it is not a change to the settlement model but a change to which entry point the off-chain settlement service calls.

Order matching has two complementary paths. The first path is the off-chain matching engine (trading-engine) that runs CDA with price-time priority over the whole zone's order book at high throughput (see Section VI.C and the measured rate in Section XI.D). The second path is the on-chain match_orders instruction that verifies and records each matched pair, touching only the two Order accounts and the zone_market, then writing a trade record without moving tokens, hence with a low compute cost (see Fig. 12). The cancel_order instruction removes an order still pending in the book.

Market-level accounts are split into per-zone zone_market accounts that store order book depth, transmission capacity, and committed_flow, separated from the central Market account, so that order submission and the enforcement of cross-zone flow caps can run in parallel under the Sealevel model (see Section VII.F.7). Once an order pair has been matched, it is sent for atomic settlement via settle_offchain_match according to the settlement model in Section IV.

2) Governance Program: On-chain PoA Control Plane

The governance program acts as a Proof of Authority control plane on-chain, designed as a single program that consolidates 20 instructions divided by function into five subsystems (19 main instructions, together with one statistics-query instruction get_governance_stats). The key architectural point is that governance does not operate in isolation but is the source of truth from which other on-chain programs read state to determine their own behavior; that is, governance records the "policy" while trading and oracle are the ones that "enforce" that policy at the boundary of their own program. The five subsystems comprise:

- Authority system: initialize_governance creates the initial singleton account, and authority transfer is done in two steps, propose_authority_change ⟶ approve_authority_change, where the recipient must sign themselves, together with cancel_authority_change and a 48-hour expiry. There is no single-step authority-transfer path (further described in Section VII.E).
- Config gates: update_governance_config (enabling/disabling the REC check and transfers), update_erc_limits, set_maintenance_mode (the system-wide stop switch), update_authority_info, and set_oracle_authority. Every instruction must be signed by the authority.
- REC certificates: issue_erc validates against the policy in GovernanceConfig then invokes a Cross-Program Invocation (CPI) to the registry to mark the energy as claimed (mark_erc_claimed), together with validate_erc_for_trading, transfer_erc, and revoke_erc. (The *erc* identifiers name issuance under ERC authority; the certificate itself is a REC.)
- DAO voting system: create_proposal ⟶ cast_vote (weighted by generation) ⟶ execute_proposal, restricted to modifying only the defined parameters of the ZoneConfig, without touching the PoA authority (the details of weighting and quorum are in Section VII.E).
- Aggregator allow-list system: admit_aggregator and revoke_aggregator (authority only), where each entry is a dedicated PDA account that performs the admission of an off-chain validator node one at a time.

The program's state accounts are PDA accounts whose addresses are deterministically derived from specific seeds, as summarized in Table XIX.

Table XIX
State accounts (PDA) of the governance program: seeds and stored data. The GovernanceConfig account is a singleton with a fixed size of 405 bytes to support upgrades, while the remaining accounts are created one entry per entity (per-cert / per-node / per-zone / per-proposal).

Account (PDA)	Seed	Stored data
`GovernanceConfig`	`[poa_config]`	singleton: authority, pending_authority, REC policy, maintenance flag, min_quorum_votes, and counters
`ErcCertificate`	`[erc_certificate, cert_id]`	REC: owner, energy (kWh), status, expiry date
`AggregatorEntry`	`[aggregator, pubkey]`	admitted off-chain validator node (allow-list)
`ZoneConfig`	`[zone_config, zone_id]`	per-zone parameters: wheeling charge, incentive, loss factor
`Proposal`	`[proposal, zone, id]`	DAO proposal: target parameter, votes for/against, status, expiry time
`VoteRecord`	`[vote, proposal, voter]`	one vote per (proposal, voter) pair

The point that makes governance truly a control plane is the way other programs consume its state. The trading program reads the GovernanceConfig account on every order-creation instruction, then calls is_operational() to block operation when the system is in maintenance mode, along with checking the validity of the REC before accepting a sell order; while the oracle program checks the AggregatorEntry account to allow only admitted nodes (or the designated chain bridge) to submit readings via submit_meter_reading. Both cases are read-only state accesses that deserialize the account themselves and check the owner and the PDA address without invoking a CPI, which avoids the cost of a CPI and reduces policy enforcement (gating) to merely checking the accounts submitted within that transaction. In this sense the AggregatorEntry account is the on-chain record of the admission of an off-chain validator node, enforced for real at the point where the oracle admits a reading into the system.

This structure reflects two separate PoA layers: the operational layer that determines who is authorized to run a validator on the permissioned network, and the application layer where GovernanceConfig.authority controls the program-administration rights, with the DAO being power-bounded to adjusting only zone

parameters and unable to seize the authority's power or modify the validator allow-list. Furthermore, the GovernanceConfig account is fixed at a constant size of 405 bytes with reserved space (*_reserved*) for upgrades, so that new fields can be added without migrating existing accounts — a design discipline that is necessary when other programs each rely on this account's layout to deserialize it themselves.

The lifecycle of the REC certificate is designed around the anti-double-claim property. Before issuing a certificate, the issue_erc instruction computes the unclaimed energy as $\text{unclaimed} = \text{total\_generation} - \text{claimed\_erc\_generation} - \text{settled\_net\_generation}$ in a saturating manner, then enforces that the requested issuance amount must not exceed this value ($\text{energy\_amount} \leq \text{unclaimed}$); otherwise it is rejected. It also checks the policy from GovernanceConfig via can_issue_erc() (the system must be operational and have the REC check enabled, and the amount must be within the range of min_energy to max_erc). When the conditions are met, the program invokes a CPI to the registry to increment the claimed-energy counter (mark_erc_claimed), so that the same unit of energy cannot be certified twice. A newly issued certificate has the status Valid but sets validated_for_trading to false until it passes the validate_erc_for_trading instruction (it must be operational, in the Valid status, and not yet expired), which is a mandatory condition before a certificate can back a trade order. The revoke_erc and transfer_erc instructions control revocation (preventing double revocation) and transfer of rights (allowed only when certificate transfer is enabled or it is a transfer from the issuer itself), with every handler updating the aggregate counters in GovernanceConfig (the number of certificates issued/validated/revoked and the cumulative energy certified).

The DAO proposal lifecycle has several layers of guards beyond the generation-weighted vote (described in Section VII.E). The create_proposal instruction rejects a non-positive voting period and enforces that the proposer be the owner of the referenced meter, by reading the MeterAccount account zero-copy then checking that meter.owner matches the proposer. The cast_vote instruction is accepted only when the proposal is in the Active status and the time has not yet passed expires_at, otherwise it is rejected with ProposalExpired, and the voter must likewise be the owner of a meter. Double-voting is prevented with a VoteRecord PDA account per (proposal, voter) pair, where the second creation of the account fails because the account already exists. The execute_proposal instruction enforces that the time has passed expires_at first, then tallies the result automatically: if the total votes are below quorum (min_quorum_votes) the proposal is rejected, while if quorum is reached and the votes in favor exceed those against the proposal passes (a tie counts as rejected); and when it passes it may adjust only four ZoneConfig parameters, namely the incentive multiplier, wheeling charge, loss factor (which must be positive), and maintenance mode, reaffirming the DAO's bounded authority.

The maintenance mechanism exhibits the property of a system-wide kill switch through a single-point account write. When the authority calls set_maintenance_mode(true), a single boolean value on the singleton GovernanceConfig account is flipped, causing every order-creation instruction in the trading program in every zone to be rejected immediately with MaintenanceMode, without redeploying the trading program. The read on the trading side skips the first 8-byte discriminator of the account then decodes Borsh directly according to the struct layout, so the gate is tolerant to changes in the governance account's discriminator as long as the field order does not change, consistent with the fixed account size and reserved space mentioned above.

3) Registry: Validator Registration and Bond

The registry program is the central registry of users, meters, and validators. The register_user and register_meter instructions create UserAccount and MeterAccount accounts zero-copy, bound to the owner via a PDA. Beyond its registry role, this program also holds the validators' security bond: register_validator requires staking a minimum of 10,000 GRX (MIN_VALIDATOR_STAKE) before being granted validator status, while stake_grx transfers GRX into the central treasury with a 24-hour withdrawal cooldown. When a validator misbehaves, the slash_validator instruction, signed by the PoA authority, slashes the bond proportionally to the severity, namely $\text{slash} = \lfloor \text{bond} \times \text{slash\_bps} / 10000 \rfloor$, paying compensation to the injured party not exceeding the proven loss ($\text{compensation} = \min(\text{slash}, \text{proven\_loss})$), with the remainder going to the slash fund to remove the incentive for accusations made in hope of reward. A full slash changes the status to Slashed (permanent, re-registration prohibited), while a partial slash that drops below the minimum changes it to Suspended (recoverable by topping up the bond). This stake-and-slash mechanism is an economic incentive that complements the PoA admission control. In addition, the initial grant of 10 GRX to new users is separated out of register_user into a distinct claim_airdrop instruction (which invokes a CPI to energy-token to mint, signed by the registry's PDA) so that registration does not fail the whole transaction if the mint has a problem.

4) Oracle: Parallel Reading Ingest and Market Clearing

The oracle program ingests meter readings on-chain via submit_meter_reading, designed to write to a per-meter MeterState account (seed *[meter, meter_id]*) while the central OracleData account is read-only. Readings from different meters therefore have non-overlapping write sets and can be processed in parallel under the Sealevel model. However, the code states the real-world constraint that parallelism occurs only when the signers paying the fee (the fee payers) differ; if multiple readings use the same gateway signer, those entries are still processed sequentially because the payer account is write-locked. Before accepting a reading, the program checks for anomalies by checking the energy-value range (min/max) and the production-to-consumption ratio, computed as integers to avoid floating-point numbers ($\text{produced} \times 100 \leq \text{max\_ratio} \times \text{consumed}$). The right to submit readings is limited to the designated chain bridge, or to nodes in the governance allow-list, via the AggregatorEntry account check in authorize_node_caller (see Section VII.F.2). Market clearing is done via trigger_market_clearing, which enforces that the epoch window boundary align to 900 seconds (*epoch_timestamp % 900 == 0*) and records last_cleared_epoch to prevent re-clearing or going back in time.

One scoping point must be stated to avoid overstating what runs. In the deployed prototype the ingest path does not write readings on-chain: the Aggregator Bridge disseminates verified readings into the zone Redis Streams and the 15-minute window bins (Fig. 5), and the forward to the on-chain oracle is a best-effort side channel

whose unavailability must not block ingest. The chain is reached at the **settlement** boundary — the surplus mint at window close, and the trade settlement — not per reading. The on-chain ingest path described in this subsection is therefore implemented and measured in isolation as a scaling experiment (Section XI.H.4), where it demonstrates that per-meter *MeterState* PDAs keep reading writes parallelizable to 200,000 meters, but it is not the path the end-to-end runs of Section XI exercise. Promoting it to the live pipeline would require a per-reading on-chain cost the design deliberately avoids paying.

5) Energy Token: Idempotent Minting and REC Governance

The energy-token program manages the minting of energy tokens under the governance of the REC certifying committee. All three minting paths (mint_to_wallet, mint_generation, mint_tokens_direct) require the signer to be in the set of REC certifiers (up to 5) once certifiers have been set (*rec_validators_count > 0*), making it a co-signature of the renewable-energy certifying authority before minting tokens. The mint_generation instruction, which creates tokens from actual generation, guarantees idempotency per time window using a GenerationMintRecord account per (meter_id, window_start_ms) pair, created with init_if_needed and enforced to align the window boundary to 900,000 milliseconds, setting the minted flag to true only after a successful mint. A repeated call at the same window therefore returns a no-op, and if the mint fails the flag remains false so it can be retried. The mint_tokens_direct instruction is the CPI target that the registry invokes to grant the initial tokens, while burn_tokens burns tokens through the SPL Token-2022 instruction. The TokenInfo account, which stores the certifier set and counters, is zero-copy so that the high-frequency path can read it without a write lock.

6) Treasury: Settlement Recording and THBC Peg

The treasury program is the CPI endpoint for recording settlements from the trading program and is the financial core of the system. Batch settlement recording (record_settlement_batch) writes a SettlementRecord account per (zone_id, batch_id) pair that stores the merkle root (merkle_root, 32 bytes), the total value, and the value-added tax (vat_amount, vat_rate_bps) as a commitment for retrospective auditing, where the merkle root binds the set of transactions in the batch on-chain while the leaves of the tree are stored off-chain. This base instruction reconciles the central total (total_settled_thbc) directly. To support a large number of concurrent settlements without the treasury account becoming a bottleneck, there is a parallel instruction (record_settlement_batch_sharded) that records the SettlementRecord as before but adds the amount to a per-shard accumulator account instead of the central total, partitioned into 16 shards (*settle_shard_for(key) = key[0] % 16*), then reconciled into the center afterward with aggregate_settlement_shards. On the THBC stablecoin peg, the swap_grx_for_thbc instruction enforces that the new supply must not exceed the attested reserve (*new_supply <= attested_reserve*) and that the attestation has not yet expired (*now - attestation_ts <= attestation_ttl*), while redeem_thbc_for_grx limits redemption so that it does not exceed the collateral in the vault (*grx_out <= swap_vault.amount*).[4] In addition, staking GRX to earn rewards uses a MasterChef-style accumulator (acc_reward_per_share scaled by 10^{12}) that increases according to the rewards added ($\text{delta} = \text{amount} \times \text{ACC} \,/\, \text{total\_staked}$), and when a slash occurs the bond is redistributed to the remaining stakers through an increase of the same accumulator.

7) Parallelism via Sealevel and Sharding

A design pattern that recurs across several programs is the use of a per-entity PDA account, so that unrelated transactions have non-overlapping write sets and can be processed in parallel under Solana's Sealevel model. Examples include the per-meter MeterState account in the oracle program, the per-order Order and order nullifier accounts in the trading program, and the per-user escrow account. For aggregate counters that would become a bottleneck if written to a single account (such as the cumulative user count, or the cumulative settled total), the system uses sharding into 16 shards deterministically derived from the first byte of the key (*key.to_bytes()[0] % 16*), both in the registry program (users and meters) and the treasury (settled amounts), and then reconciles into the central counter afterward with administrator-level instructions (aggregate_shards, aggregate_settlement_shards, and aggregate_readings) that prevent double-counting with a bitmask. As a result, central accounts such as GovernanceConfig, OracleData, and Market are designed to be readable in a stale manner on high-frequency paths, accepting that the aggregate value lags slightly in exchange for parallelizable throughput. This design is consistent with the slot-time target near 400 milliseconds referenced in Section VII.E. The actual on-chain transaction throughput is measured on a single validator and reported in Section XI.F, while the figures on a multi-node permissioned network that include the consensus cost remain future work (see Section XI.E).

G. Data Model and Trust Boundary

The core data of a trade order consists of order_id, user_id (the prosumer or consumer role), meter_id, side (offer or bid), energy_amount (quantity_kwh), price_per_kwh, status, expires_at, epoch_id, and zone_id, where the energy_amount must not exceed the available_surplus_kwh that the Aggregator Bridge certifies for the seller in the same time period — a condition checked in the off-chain layer. Replay prevention uses an on-chain nullifier account (order nullifier) rather than storing a nonce in the order itself. The settlement record consists of trade_id, the buy_order_id/sell_order_id pair, the cleared energy_amount, price, fee_amount/net_amount, wheeling_charge, loss_factor, erc_certificate_id, blockchain_tx, and the settlement timestamps (created_at/confirmed_at). The escrow state is stored separately as its own PDA account (an SPL token account under the seed *escrow*), not within the settlement record.

The auditability that enables the blockchain to serve as an audit layer comes from the event log emitted by the programs at every important step of the trading cycle, namely order creation (SellOrderCreated, BuyOrderCreated), matching and settlement via the OrderMatched event that records the order pair, the buyer and seller, the quantity, price, total value, and fee, emitted from both settle_offchain_match and batch_settle_offchain_match, order cancellation (OrderCancelled), bond deposit (EscrowDeposited), and market clearing (AuctionCleared). Batch settlement recording in the treasury layer emits the SettlementBatchRecorded event that binds

[4] Both instructions were replaced after the commit evaluated here. The reserve ceiling and the attestation-freshness check now live on *issue_thbc*, the sole instruction that increases supply, and the currency-exchange pair *exchange_grx_for_thbc* / *exchange_thbc_for_grx* transfers against a platform-held THBC inventory

the batch's merkle root on-chain. These events are append-only records not modified retrospectively, which external auditors use to assemble an auditable transaction history without having to trust the off-chain layer directly, consistent with the settlement and audit layer role described in Section IV.

The boundaries of responsibility are defined clearly as follows: the Backend and the Aggregator Bridge are the ones that assess the generation and consumption data, the grid-stability conditions, Islanding safety, network constraints, and the condition that the kWh quantity does not exceed the certified value (available_surplus); then the order pairs that pass the conditions are sent to the on-chain settlement stage. The Anchor program checks only account ownership, the signer, the Ed25519 signatures of both buyer and seller on the order payload, timestamp validity (expires_at), replay prevention via the order nullifier account, the slippage/zone-capacity conditions, the escrow state, and the order/trade state not yet reused. That is, the conditions on energy quantity and oracle attestation are enforced in the off-chain layer before submission and are not re-checked on-chain. Therefore, in this prototype the blockchain serves as a settlement and audit layer, not as a full power-flow or grid-stability engine.

H. Trade Lifecycle Sequence

The trading sequence is clearly scoped between off-chain and on-chain, as in Fig. 9. The off-chain side is responsible for collecting Smart Meter data, authentication, verifying the reference time, assessing the grid-stability conditions, and computing the order pairs proposed for clearing. The on-chain side is responsible only for confirming the account state, the Ed25519 signatures of the buyer and seller on the signed order payload, the bounded matched pair, the escrow, and recording the settlement event.

This process makes energy transactions transparent and auditable while reducing the risk of bringing unverified Smart Meter data directly onto the blockchain. The separation of the boundary between off-chain verification and on-chain settlement is therefore a key principle in preserving both system performance and the safety of the microgrid. The limitation of this approach is that users must trust the Aggregator Bridge and the governance of the oracle_authority; to further reduce trust, one should add multi-oracle attestation or cryptographic proofs in the future.

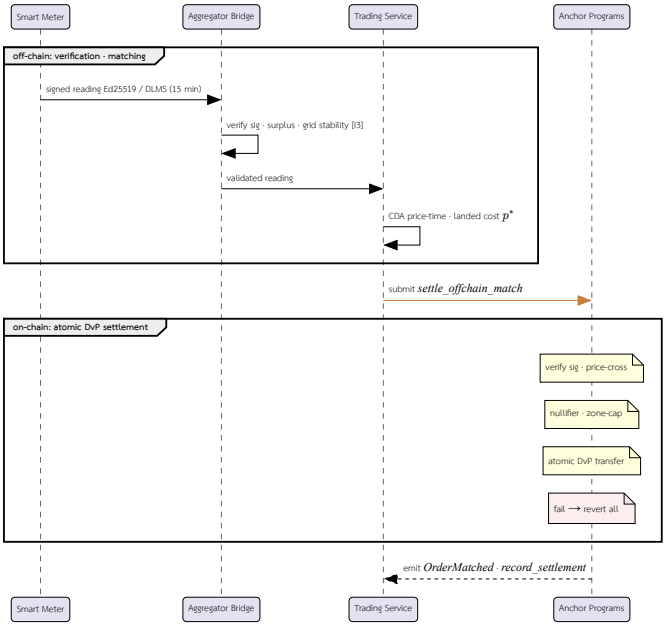


Fig. 9. Trade lifecycle as a sequence diagram: off-chain participants (Smart Meter ⟶ Aggregator Bridge ⟶ Trading Service) verify and match, then submit to the on-chain Anchor Programs (via the Chain Bridge) which settle and audit. Notes tag the on-chain checks enforced during settlement (described in Section IV); any failed CPI reverts the whole atomic DvP settlement.

VIII. IDENTITY, ONBOARDING, AND THE BINDING OF A METER TO A WALLET

How the trust root the settlement model presupposes is actually constructed: five distinct identities, a deliberately non-atomic registration, and an optimistic-submit path that must be confirmed rather than believed.

The threat model in Section III assumes two facts and does not establish either of them: that every reading arrives signed by a key the platform has already registered for that device, and that every verified reading can be attributed to a wallet that may receive the minted surplus. Both are the output of an onboarding procedure, not a property of the runtime. This section describes that procedure, because its failure modes turn out to be the ones that actually cost the experiments in Section XI.H, and because the way it is decomposed — into steps that are allowed to fail independently — is a design decision rather than an implementation accident.

A. Five Identities, Not One

What informal descriptions call "registering a meter" is the construction of five separate identities, each with its own key material, its own store, and its own failure mode. They are summarized in Table XX.

The first is the **account**: an email and an Argon2 password hash held by the IAM Service, created by *POST /api/v1/auth/register* and activated by *POST /api/v1/auth/verify*. Email verification, not registration, is the pivotal step — it is where every subsequent identity is provisioned.

The second is the **wallet**. On verification the platform generates a Solana keypair, encrypts the full 64-byte key under AES-256-GCM

without changing *thbc_supply*, so that a volatile asset no longer enters the backing set of the fiat-referenced unit. The compute-unit figure reported in this paper measures the instruction as evaluated; the replacement is more expensive rather than less. We report the change as subsequent work, not as measured behavior.

with a PBKDF2 key-derivation at 600,000 iterations, and persists the ciphertext together with the wallet row in a single database transaction, so that a key without a wallet — or an address whose key is unrecoverable — is never observable, even across a crash. The derivation takes both of its inputs from service configuration; the user's password is not among them. That is a custody decision and is discussed in Section IX.

The third is the **on-chain user**: a *UserAccount* created by the registry program's *register_user* at the Program Derived Address seeded by *[b"user", authority]*. The fourth is the **on-chain meter**: a *MeterAccount* created by *register_meter* at *[b"meter", owner, meter_id]*, where the identifier is capped at 32 bytes because the zero-copy account layout stores it as a fixed byte array rather than a *String*. The instruction refuses to create a meter whose *owner* is not the authority recorded on the referenced user account, so a meter can only ever be created beneath its true owner's registered account even though *owner* is a non-signing account in the custodial model.

The fifth is the **device**: a per-meter Ed25519 signing key and a separate 32-byte AES key, published to the Aggregator Bridge's device registry under two Redis keys per device. These are the keys the ingest path checks on every reading, and they are entirely distinct from the wallet keypair — a meter authenticates its telemetry with one key and is paid through another.

Table XX
The five identities constructed during onboarding, and what fails when each is missing.

Identity	Key material	Store	Consequence if absent
Account	Argon2 password hash	Postgres (IAM)	No authenticated API surface; nothing else can be provisioned
Wallet	Ed25519 keypair, AES-256-GCM encrypted	Postgres (IAM)	Verified surplus has no mint destination — readings are retained, not settled
On-chain user	none (PDA *[b"user", authority]*)	Solana	*register_meter* and *claim_airdrop* fail with *AccountNotInitialized*
On-chain meter	none (PDA *[b"meter", owner, id]*)	Solana	No account for *submit_meter_reading* to write per-meter state into
Device	Ed25519 signing key + 32-byte AES key	Redis (bridge registry)	Ingest fails closed at signature verification — the reading is dropped

B. Registration Is Deliberately Not Atomic

The obvious design makes onboarding one transaction: verify the email, mint the welcome grant, register on-chain, return. The system does the opposite at three separate points, and each split is a response to a concrete failure.

The welcome airdrop is not minted inside *register_user*. Solana cannot swallow a failed Cross-Program Invocation — a failing mint would abort the enclosing transaction — so folding the grant into registration would make a token-program problem into a registration outage. The grant is therefore a separate *claim_airdrop* instruction (Section VII.F.3), idempotent and retryable, and registration succeeds independently of it.

On-chain registration is not awaited by the HTTP verification response. The submit path retries with backoff for roughly 14 seconds and the confirmation poll described below adds up to another 15, so awaiting it would hold the verification response for up to half a minute on an oversubscribed validator; in practice this tripped the end-to-end suite's 10-second client timeout. Registration is spawned as a detached task, and the wallet is already persisted and usable off-chain before it runs. The consequence is stated rather than hidden: a freshly verified account is off-chain-complete and on-chain-pending, and the pending half is separately retriable through an explicit endpoint.

The key-derivation step is bounded by the same CPU semaphore the password paths use. PBKDF2 at 600,000 iterations is several hundred milliseconds of pure computation; run inline on the async runtime it starves the worker threads for every other endpoint. The effect was measured during development as a collapse of verification latency from roughly 0.8 s serial to roughly 40 s at 64 concurrent onboards. Moving it to the blocking pool is necessary but not sufficient — without the semaphore, concurrency simply spawns unbounded blocking tasks — so onboarding throughput is capped deliberately at the configured limit.

C. Optimistic Submission and the Confirmation Gate

Chain Bridge returns a transaction signature without confirming execution. This is the correct interface for a write path fronted by a durable queue, but it means a signature is evidence of submission and not of landing: a dropped or simulation-rejected transaction is indistinguishable, at the call site, from a successful one. Recording such a signature as "registered" produces a database that confidently disagrees with the chain.

The service therefore does not trust the return value. It derives the user PDA locally, then polls for the account's existence — twenty attempts at 750 ms, a window of about 15 seconds chosen to absorb both the roughly 1–2 s *confirmed* commitment lag and the provider-side retry that can land the transaction several seconds after the call returns — and marks the user registered only once the account is observable. The same derivation supplies idempotency at the other end: if the PDA already exists when onboarding is requested, the service heals its own flags and returns without spending a transaction, which is what makes the provider's retry safe against the registry program's *AccountAlreadyInUse*.

This is a general point about writing to a blockchain through an asynchronous bridge, and it recurs wherever the platform does so. The authoritative record of a write is the existence of the account it was supposed to create, not the acknowledgement of the request that created it.

D. Shard Binding at Registration Time

Registration is also where an entity is assigned its position in the registry's Sealevel-parallelism scheme (Section VII.F.7). Both *register_user* and *register_meter* take a *shard_id* argument, and both refuse it unless it equals the value derived from the owner's key, *shard_for(k) = k[0] mod 16*. A meter is additionally required to co-locate on its owner's shard. Counters are incremented on that shard account; the global *Registry* account is held read-only on this path precisely so that registration does not take a write lock on a single global account and serialize every concurrent onboard, with the global total reconciled afterwards by *aggregate_shards*.

The derivation is not a hint the client may choose to follow. Because the required value is a pure function of a key the caller does not control, a caller cannot scatter counts across arbitrary shards, and the shard assignment is reproducible by any party that knows the key — including the settlement path, which must find the same accounts later.

E. Attribution at Ingest

Onboarding produces the binding; the ingest path consumes it once per reading. After a frame has been decrypted and its signature verified, the Aggregator Bridge resolves the meter serial to an owner and a wallet through a three-tier lookup: a process-local cache, then Redis, then a Postgres owner read model that repopulates the tiers above it. A serial that misses every tier is recorded in a negative cache with its own expiry, so an unregistered device cannot turn each of its readings into a database round-trip.

An unresolved serial does not fail the reading — it de-attributes it. The reading is admitted, disseminated, and counted, but it has no wallet to settle into. This is the correct behavior for a metering system, in which discarding a physically valid measurement because the platform's records are incomplete is the worse error, and it is the mechanism behind the "no wallet registered, kept for retry" outcome reported in Section XI.H, where surplus was withheld rather than lost and re-minted into its original settlement window once the wallet links were repaired. It also means that the **liveness** of attribution rests on an eventually-consistent projection: the ingest path reads a local read model, and a reading that arrives before that projection catches up is attributed on retry rather than at first sight.

F. What Onboarding Does Not Establish

It is worth stating plainly what this procedure does and does not deliver. It establishes that a reading came from a device the platform enrolled, and that the device belongs to an account the platform verified. It does not establish that the device measures what it claims to measure — meter tampering is outside the cryptographic boundary, as noted in Section III — and it does not establish user custody of the wallet it provisions. The keypair is generated by the platform, encrypted under platform secrets with no user-supplied input to the derivation, and stored beside its salt. The platform can therefore reconstruct any user's key. This is a settled architectural posture rather than a latent gap, and it is treated as such in Section IX, where it appears as a disclosed and violated invariant rather than as a caveat.

IX. THE PAYMENT LEG: FIAT ON-RAMP, REDEMPTION, AND A DISCLOSED-INVARIANT REGISTRY

The baht side of settlement, and a method for reporting what a design guarantees: a machine-readable invariant registry in which two of nine entries are published as unmet.

Section IV.M treats THBC as an on-chain object — a six-decimal mint whose supply is bounded by an attested reserve. That account is only half of a settlement currency. A token referenced to the baht must also have a route by which baht enter and leave, and that route is irreducibly off-chain: a bank credit, a compliance screen, an attestation of what the reserve account holds, and, on the way out, a wire. This section describes that layer, and it does so through the artifact the layer is organized around, which is of more general interest than the layer itself.

A. An Invariant Registry as the Unit of Claim

The payment service does not document its guarantees in prose. It enumerates them as nine numbered invariants — F1 through F9 — in a compiled data structure that is served live from an administrative endpoint, so that any operator, auditor, or reviewer reads the same statement the code compiles against. Each entry carries a formal statement, a status, an enforcement class, and, when the status is anything other than fully met, a mandatory description of the gap.

Two aspects of this construction do the work. The first is that status and enforcement are distinct axes. A property may be **enforced** — with a test that exercises the violating path, not merely a passing happy path — **partial**, **design-only**, or **violated**, which is strictly worse than design-only because code is known to contradict it. Orthogonally, enforcement is classified by the mechanism that provides it: the Solana runtime, an on-chain program check, an off-chain check, or nothing. The ordering of that second axis is the useful part, and it is discussed in Section IX.C.

The second is that the registry is adversarial toward its own author. A non-enforced entry cannot compile without a gap string, so an invariant cannot be quietly downgraded; the accompanying test asserts the expected status of every entry, so a regression that weakens a guarantee fails the suite rather than passing silently under a softened claim. The registry as evaluated is reproduced in Table XXI.

Table XXI
The F1–F9 registry as evaluated. Status and enforcement are independent axes; every non-enforced entry carries a disclosed gap.

	Invariant	Status	By	Disclosed gap
F1	Reserve sufficiency	Enforced	on-chain	—
F2	Issuance conservation	Partial	off-chain	detective, not preventive — no write is rejected for causing drift; a reconciler reports it afterwards
F3	Deposit idempotency	Enforced	runtime	—
F4	Burn-before-wire	Partial	off-chain	ordering is enforced by the redemption state machine, but no fiat rail exists to test the barrier against
F5	Attestation freshness	Enforced	on-chain	—
F6	Backing-set purity	Partial	on-chain	the code no longer mints against volatile collateral, but supply minted by the previous instruction is still outstanding and remains so backed
F7	Redemption liveness	Enforced	on-chain	—
F8	Non-custody	**Violated**	—	the platform provisions and stores every user's key under service-only secrets and signs settlement with a single platform key; custody is the chosen posture, not a drift
F9	Attestation independence	Design-only	—	initialization never compares the attestor key to the parameter-admin key, and the evaluated deployment has them equal

B. Two Orderings Carry the Safety

Most of the payment layer is bookkeeping. Two orderings are not, and they are the reason the layer exists as a distinct service rather than as handlers on an existing one.

On the on-ramp, attestation must precede issuance. A deposit advances through observation, compliance screening, attestation, and only then issuance; the state machine offers no transition from the screened state directly to the issued state, which makes the dangerous interval — tokens issued against a reserve figure that has not yet been attested — unrepresentable rather than merely discouraged. The ceiling checked at that point is the attested reserve less the deposits known to be encumbered — fiat that has cleared the bank but backs no token, because it failed a compliance screen or is disputed. This is also the clearest instance of the registry driving work rather than merely describing it. The encumbrance figure originally had no on-chain field, so the service enforced the tighter ceiling from its own records while the program enforced the bare attested reserve; the service was therefore stricter than the chain, and that asymmetry was published as the F1 gap rather than presented as a safety margin, for the reason that made it a gap at all — a caller who bypassed the service got the looser ceiling. Closing it turned out to require less than the gap text assumed: the field is a 64-bit integer, not a public key, so it fits the tail of the treasury account's existing padding at an already-aligned offset, leaving the struct the same size and every deployed account still deserializable, with no re-initialization. Accounts written before the field existed read it as zero, so their ceiling is unchanged until an attestation reports an encumbrance. With the service now publishing the encumbered total on every attestation, both sides evaluate the same inequality and F1 moved from partial to enforced.

On the off-ramp, a confirmed burn must precede a fiat payout. The barrier is confirmation, not acceptance — a submission acknowledgement leaves the record in its requested state and does not start the recovery clock — and within the payout routine the record is persisted **before** the payout is enqueued. That ordering is chosen against the intuitive one on an explicitly asymmetric basis: if the process dies between the two steps, the surviving record claims a payout that may never have been sent, and an operator investigates. The reverse order loses the record of a wire that **was** sent, and a double payout is unrecoverable in a way that a double burn is not. The payout enqueue is idempotent on the pair of user and sequence number so that the resulting retry is safe.

C. The Strength of an Enforcement Mechanism

The registry's enforcement axis encodes a ranking that the rest of this work applies informally and that is worth making explicit: a guarantee provided by the runtime is stronger than one provided by a program check, which is in turn stronger than one provided by an off-chain check.

F3, deposit idempotency, is the clearest case. The issuance instruction creates a nullifier account addressed by the hash of the bank reference, using the framework's *init* constraint, in the **same instruction** as the mint. A replayed bank webhook is therefore rejected by the Solana runtime at the account level, before any program code executes: no application bug can defeat it, and the mint and the nullifier either both occur or neither does. Splitting these across two transactions would not implement the invariant at all — it would implement a check with a window.

F7, redemption liveness, uses the same idea inverted. Redemption escrows the tokens without burning them, so they remain within the tracked supply and remain recoverable; a confirmation burns and a reclaim returns them after a fixed delay of 24 hours, frozen per record so that changing the parameter cannot extend an in-flight holder's wait. Both terminal instructions **close** the record, so a double confirmation, or a confirmation after a reclaim, fails at the account level rather than through a comparison the program might get wrong. A useful side effect is that the set of live escrow accounts **is** the pending queue, which makes its length derivable rather than a stored counter that can drift. Reclaim is deliberately not gated on the pause flag, on the principle that pausing the system must never trap a holder's tokens.

F5, attestation freshness, is an ordinary program check, and its two details are both about ordering. It is evaluated **before** the F1 ceiling, because comparing supply against a stale reserve figure is not conservative but meaningless; and a future-dated attestation is rejected outright rather than treated as maximally fresh, so that clock skew cannot be used to buy freshness.

F6 illustrates the remaining distinction, between a claim about code and a claim about state. The instruction that previously minted the settlement currency against volatile collateral was replaced by an inventory transfer out of a platform-held vault, at identical pricing, so that no program mints or burns the currency any longer and the invariant now holds by construction: the quote type has no field capable of expressing a supply change. The invariant nonetheless remains partial, because supply minted by the earlier instruction is still outstanding and still backed by the volatile asset. F6 is a statement about what backs the supply, and code alone cannot retire supply that already exists.

D. Two Published Failures

The registry's value is easiest to see in the two entries it does not claim.

F8 asserts that no platform key appears in a signer set able to move user funds, and it is recorded as violated at the system level. The service itself is genuinely non-custodial — no method on any of its ports accepts a key, keypair, or signer, and its sweep of overdue redemptions deliberately **reports** them rather than reclaiming them, because reclaiming would require the holder's key. That is a property of one service and not of the platform. As described in Section VIII, the identity service generates each user's keypair and encrypts it under two service-configuration secrets with no user password in the derivation, so anyone holding those secrets and the database can sign as any user. Nor is this capability dormant: per-user signing was **removed** in favour of a single platform key held in the transaction-signing service, and the settlement entry point the evaluated deployment actually uses pools escrow in platform-owned accounts with the platform as sole signer (Section VII.F.1). Custody is therefore the direction the platform chose rather than a gap it drifted into, and the honest disposition of the invariant is to retire the non-custody claim rather than to describe it as forthcoming.

F9 asserts that the reserve attestor and the parameter administrator are distinct keys, and it is recorded as design-only after an earlier entry claimed it was enforced on-chain. The initialization instruction accepts an attestor argument and stores it without comparing it to the authority, and the program defines no error for the equality case;

the evaluated deployment consequently runs with the two roles held by the same key. There is also a live reason it must currently be so, and the shape of that reason is instructive. Enforcing F9 takes three changes, and any one alone breaks the deployment: a distinct signing key for attestation; a relaxation of the transaction-signing service's authorization gate, which today rejects any key identifier other than the single platform key and is the reference monitor's own check rather than a configuration value; and a means of moving the attestor on an already-initialized treasury, since the field is written once and has no setter — and a setter gated on the parameter admin would partly defeat the invariant, since that admin could then grant itself attestation rights. Adding the equality check alone is worse than doing nothing: it would not touch the treasury already deployed, and it would make the initialization script fail in every fresh environment, because that script passes one key for both roles. The off-chain service does reject the equality, but that is the service declining to submit rather than the chain declining the transaction — advisory, and bypassed by any other caller.

Both entries point at the same place. The reserve figure is a single unsigned integer written by a single signer, and reserve sufficiency, attestation freshness, the peg claim, and the solvency of every user balance all reduce to that number being honest. Nothing in this design improves on that; an attestation that fails to cover supply is logged as an error, which makes a lie visible without preventing it. Collusion between the fiat custodian and the attestor is the single point of failure, and no mechanism here addresses it.

E. Scope: What Is Modeled and What Is Built

The payment leg is a model executed against a simulator, and the code is arranged so that this cannot be mistaken. No fiat rail of any kind exists, and the compliance port refuses every screen outside simulated mode rather than defaulting to a pass. The adapter that talks to the real chain returns an ***Unsupported*** result — surfaced as ***501 Not Implemented*** — for issuance, redemption escrow, confirmation, reclaim, and state snapshot, and this is a deliberate refusal rather than unfinished work: routing issuance to the minting instruction that **does** exist would produce a service that appears to implement the on-ramp while violating backing-set purity on every deposit. ***Unsupported*** is a distinct outcome from ***Rejected*** precisely so that an operator can distinguish "not built" from "the chain said no". The snapshot call refuses for a subtler reason — three of the four fields it would return have no on-chain home, and synthesizing them would report a tighter reserve ceiling than the chain actually enforces. The service reports its posture at startup and on every response, so simulated state cannot be read as live.

Verification is correspondingly scoped. The off-chain layer carries 171 tests that run with no infrastructure at all, which is a direct consequence of the sync-core discipline of Section VII: every invariant decision is a pure function over values, so the whole registry is exercisable without a database, a broker, or a validator. The on-chain half is pinned by 27 in-process SVM cases covering F1, F3, F5, F6, and F7 against the compiled program — including F3's replay revert, which is a property of the runtime's account initialization and can only be demonstrated on an SVM, and F5's freshness halt, which requires warping the cluster clock. That suite is mutation-checked three times: removing the freshness guard kills exactly the three cases that test it and nothing else, removing the redemption timelock kills exactly the boundary case, and removing the encumbrance subtraction from the reserve ceiling kills three of F1's four cases — so the suite demonstrably detects a regression rather than merely passing. Following the standard set in Section III, we report this as evidence about the invariants named and not as coverage of the payment leg as a whole; the fiat halves of F2 and F4 have no rail to be tested against, and no test can establish F8 or F9, which the registry reports as unmet rather than untested.

X. EXPERIMENTAL SETUP

The simulation testbed, meter-fleet workload harness, and hardware — a simulation, not a live power system, stated up front.

This section summarizes the implementation details, the version of the artifact, the workload parameters, and the definitions of the metrics, so that the evaluation in Section XI is reproducible. We state the remaining reproducibility limitations candidly at the end of this section.

A. Implementation and Artifact

All backend services are developed in Rust (Axum) on the Tokio async runtime and communicate with one another over ConnectRPC on top of mTLS, while the path that writes data to the blockchain is decoupled through NATS JetStream [24]. The grid and Smart Meter simulation suite is developed in Python 3.11 [28] (tool details are in Section VII.C). The whole system is arranged as a superproject that incorporates each service as a git submodule, which makes the version of the artifact identifiable from the commit of the superproject together with the pointer of each submodule. The artifact version is stated per experiment rather than as a single pin, because the measurements were taken at different points on the same line of development. The telemetry-ingest experiments (Section XI.B, Section XI.C) refer to the state of the superproject at commit ***4b05661***, which pins the pointers of the core services — namely Aggregator Bridge, Chain Bridge, Anchor programs, blockchain-core, and Smart Meter Simulator. The on-chain cost and throughput measurements (Section XI.E, Section XI.F, and Section XI.G) are carried out on the gridtokenx-anchor benchmark suite at commit ***58cfc79***, which is an ancestor of the pointer pinned by the superproject ***4b05661***. The fleet-scale live-mint and on-chain telemetry-write experiments (Section XI.H) are necessarily later than ***4b05661***: they measure code paths introduced or tuned in the course of those runs — the mint-consumer concurrency increase, the aggregator reply-wait budget, and the request-metrics label fix reported there — and refer to the superproject state up to commit ***cdd9d00***. Reproducing a given number therefore requires the commit named for that experiment, not a single artifact pin for the whole article.

B. Topology and Endpoints

On the evaluated path, the Smart Meter Simulator sends readings into the Aggregator Bridge through the IoT gateway (HTTP) and a gRPC channel for ingest. The Aggregator Bridge then verifies the signatures and disseminates the validated readings into zone-partitioned Redis Streams, while the chain-write path is routed through the Chain Bridge using the NATS ***chain.tx.**** family of subjects (e.g., ***chain.tx.submit*** and ***chain.tx.mint***). Separating the ports and channels in this way allows the ingest path to scale independently of the settlement path.

C. Deployed Components Outside the Measured Scope

The evaluated path described above is a subset of the deployed system. For an accurate reading of what the numbers in Section XI do and do not cover, we state explicitly which running components sit outside the measurement boundary. The deployment additionally runs an APISIX API gateway as the user-facing entry point (the measured ingest path bypasses it and addresses the Aggregator Bridge IoT gateway directly); a Kafka event backbone on three brokers carrying identity events, the meter-owner read model, and the notification pipeline; a separate meter-service that owns meter registration and the meter registry; an ERP bridge that exports meter readings and settlement determinants to a utility SAP system, implemented in its first phase against a mock adapter and discussed as a trust-boundary question in Section III.E rather than as a measured component; a per-service database split (each service owning its own schema, fronted by a connection router) rather than one shared database; two Chain Bridge replicas with a shared deduplication port for submission idempotency; a durable mint outbox that retries a settlement mint until the chain confirms it; a metrics/log/trace observability stack; and three web frontends (trading UI, block explorer, simulator UI). None of these are exercised as an independent variable in this article, and none is reported on. Two of them nevertheless bear on results that are reported and are named where they matter: the durable mint outbox is what carries the delayed tail of Table XXIX to completion, and the observability stack is where the metrics-cardinality incident of Section XI.H was diagnosed. Measuring the gateway, the event backbone, and the multi-database path is future work (Section XII).

D. Workload Parameters

The workload in the evaluation is defined by 80 Smart Meters (*NUM_METERS=80*) that send readings every 15 seconds of simulated time (*SIMULATION_INTERVAL=15*), following the default of the simulation suite. A transmission cycle of every 15 seconds of simulated time corresponds to a time compression of approximately 60× relative to the 15-minute (900-second) window of the real meters' transmission cycle. Here we define a "reading" — the unit used to measure throughput — to mean one meter reading signed with Ed25519 that has passed signature verification and been disseminated into a Redis Stream at the Aggregator Bridge, not a matched trading order or an on-chain settlement transaction. The main workload parameters are summarized in Table XXII.

Table XXII
Workload parameters for the telemetry-ingest evaluation.

Parameter	Value	Meaning
`NUM_METERS`	80	Number of Smart Meters in the simulation
`SIMULATION_INTERVAL`	15 s	Reading transmission cycle in simulated time
Time compression	≈ 60×	Relative to the real 900 s window
Nominal ingest rate	5.33 readings/s	80 ÷ 15 in simulated time
Run duration	≈ 27 min	Wall-clock time of one run
Total readings	26,240	Signature verification succeeded, no loss

E. Metrics

The primary metric of the ingest-path evaluation is the ingest rate, under two definitions: the design-time rate in simulated time (nominal), equal to $80 \div 15 = 5.33$ readings per second, and the rate measured from wall-clock time, which is higher because the simulator's transmission cycle is accelerated beyond the 15-second interval of simulated time, together with the cumulative number of successfully verified readings and the data-loss ratio. The detailed measurement results are in Section XI.B.

F. Reproducibility Limitations

The long-running ingest-path experiment in Section XI.B is a single real run, while the step-load experiment in Section XI.C is repeated 5 times per fleet size and reports the mean together with the standard deviation on the recorded Apple M2 (8 cores, 16 GiB memory) machine. The remaining reproducibility limitation is that the workload is generated from a single sender client (parallel senders have not yet been tested), so the reported ingest-rate figures are a lower bound of the ingest path rather than the ceiling of its maximum capacity, and the on-chain settlement-path measurement is still limited to the compute-unit cost per instruction, without yet covering end-to-end latency and throughput. The next experiment should therefore add parallel senders to find the true ceiling, record the validator configuration, and extend the measurement to cover the on-chain settlement path end-to-end (see Section XII).

XI. EVALUATION

On the ingest path, 80 meters sustain the 5.33 rec/s design rate with zero loss, and a step-ramp to 640 meters holds loss ≤ 0.03% — the measured rate being a single-sender lower bound.

This section presents an architectural evaluation of the prototype system, covering the telemetry ingest rate, the off-chain order-matching rate, and the cost and throughput of on-chain transaction settlement, in order to reflect the suitability of the architecture for Peer-to-Peer energy trading at the microgrid level. In the Performance dimension, this section reports measurements of the ingest rate along the telemetry ingest path, both from a single real run (see Section XI.B) and from a step-load experiment that measures the loss ratio and single-sender cost (see Section XI.C). It also reports the on-chain processing cost of the settlement path directly, expressed as compute-units per settled order pair (see Section XI.E). In addition, it reports on-chain transaction throughput using standard database benchmark workloads (Blockbench, TPC-C, and SmallBank ported onto the Solana-compatible network) together with measurements of the real settlement path in Section XI.F. The evaluation is a characterization of a single system, not a head-to-head quantitative comparison against other platforms. Measuring the end-to-end latency and throughput of the settlement path, as well as field-level testing of the grid-stability model, remains future work.

A. Performance

The design results indicate that the Microservice architecture can structurally reduce the coupling of workloads among Smart Meter data ingest, order validation, and transaction submission to the blockchain. Separating aggregator-bridge, trading-service, chain-bridge, and settlement-engine gives each service a clear scaling point according to its workload type, particularly during periods with large volumes of Smart Meter data or with concurrent trading orders.

In terms of real-time operation, the system uses ConnectRPC and NATS JetStream [24] to separate latency-sensitive work from work that must wait for blockchain transaction confirmation. Therefore, the Backend Service is designed to respond to requests from users and Smart Meters without having to wait for transactions on the Solana-compatible network to complete immediately. This approach

is expected to reduce the impact of blockchain network latency, but it still needs to be tested with a workload that explicitly specifies the number of meters, sampling interval, queue depth, and transaction mix in future work.

B. Telemetry Ingest Throughput

To evaluate the capability of the ingest path under high load, we ran a simulation with the workload in Table XXII (80 Smart Meters, each sending a reading every 15 seconds of simulated time), using the definition of a "reading" as the unit of throughput per Section X (an Ed25519-signed reading that has passed signature verification and been disseminated into the Redis Stream, not a matching order or an on-chain settlement transaction).

The nominal ingest rate in simulated time equals 80 ÷ 15 = 5.33 readings per second, which corresponds to a time compression of approximately 60× relative to the reporting cycle of real meters that use a 15-minute (900-second) window. From a single real run of approximately 27 minutes, the Aggregator Bridge received and verified the signatures of 26,240 readings in total from 80 meters continuously, with no data loss. The rate measured by wall-clock time was approximately 16 readings per second, which is higher than the nominal rate because the simulator's reporting cycle was accelerated to be faster than the 15-second interval of simulated time.

This result shows that the ingest path can handle the reporting load of 80 meters stably. However, this measurement covers only the meter data ingest and verification path; it does not yet include measurements of the throughput and latency of the on-chain settlement path, such as the throughput and latency of order matching and recording transactions on the Smart Contract. Furthermore, this experiment did not record the hardware and validator mode, so the next experiment should specify a reproducible workload, hardware profile, and validator configuration, together with measuring the settlement path end-to-end.

The loss ratio and single-sender cost under step-load are reported next in Section XI.C.

Off-chain matching reaches 3.1 × 10^4 pairs/s, while a single-pair on-chain settlement costs 96,707 compute units (48% of the default budget) — leaving headroom for batch settlement.

C. Ingest Loss and Single-Sender Cost Under Step Load

To evaluate the loss behavior of the ingest path beyond the design rate of 5.33 readings per second from Section XI.B, and to characterize the cost of a single full-rate sender, we apply a stepwise load ramp with meter-fleet sizes of 40, 80, 160, 320, and 640 meters, where each step runs for 60 seconds and is repeated 5 times to report the mean and standard deviation. In each run the simulator sends Ed25519-signed readings continuously at maximum rate (no delay between send rounds). The primary metric, measured at the HTTP boundary, is the fraction of requests that the Aggregator Bridge accepts, signature-verifies, and disseminates successfully into the Redis Stream (HTTP 200), where throughput = number of successful readings ÷ elapsed time and loss = number of failed requests ÷ total requests. The results are summarized in Table XXIII and the trend is plotted in Fig. 10. The experiment runs on an Apple M2 machine (8 cores, 16 GiB memory).

Table XXIII
Ingest throughput and loss vs. meter-fleet size (mean ± sd over 5 runs, 60 s each).

meters (count)	throughput (readings/s)	loss (max)
40	134.4 ± 9.2	0
80	72.8 ± 6.9	2.6×10^{-4}
160	88.4 ± 15.0	2.2×10^{-4}
320	115.0 ± 19.8	0
640	148.9 ± 25.6	1.0×10^{-4}

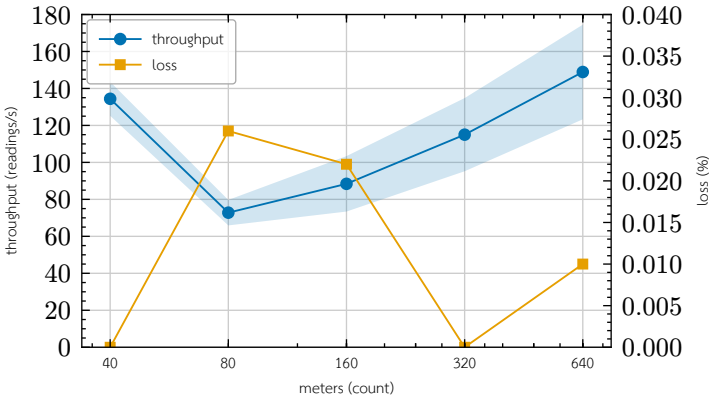


Fig. 10. Request loss (right axis) and single-sender throughput (left axis, mean ± sd) vs. meter-fleet size. The headline result is loss: it stays ≤ 0.03% across the whole tested range. Throughput is non-monotonic and single-client sender-bound — the cost of one full-rate sender, a lower bound, not a service-saturation curve.

The measurements yield two main observations. First, the loss ratio stays near zero across all fleet sizes, with the maximum measured value being approximately 2.6 × 10^{-4} (about 0.03%) at the 80-meter size, and several runs exhibit no loss at all. This indicates that the ingest path maintains a near-zero loss ratio (≤ 0.03%) up to a load of 640 meters. Second, the measured ingest throughput does not increase monotonically with the number of meters but fluctuates in the range of approximately 73–149 readings per second (mean), with a relatively high standard deviation (up to about 26 readings per second at 640 meters). Because this experiment generates load from a single sender client that operates asynchronously and sends per-meter readings sequentially per tick, this fluctuation and non-monotonicity likely reflect sender-side limits (such as the cost of building and signing readings, and single-client processing) together with the cadence of asynchronous dissemination into Redis, rather than saturation of the ingest service. We do not draw a definitive conclusion about the cause of this pattern from the available data.

From these results, the highest measured mean rate (mean over 5 runs at the 640-meter fleet) is approximately 149 readings per second, which is several times higher than the design rate of 5.33 readings per second and occurs at the largest fleet size tested (640 meters). However, the measured rate does not increase monotonically with the number of meters (the mean at 80 meters is lower than at 40 meters), so we do not conclude whether or not the ingest path reaches saturation from this dataset. Nevertheless, because the load is generated from a single sender client that transmits sequentially per meter per tick, this figure is a lower bound, reflecting that the Aggregator Bridge can support one full-rate sender client with near-zero loss, rather than the true capacity ceiling of the ingest path. Measuring the true ceiling requires multiple parallel senders and isolating the sender-side cost (such as onboarding and signing) from the cost of the ingest service, which remains future work. We also note that the rates in this ramp experiment (73–149 readings per second) are higher than the roughly 16 readings per second wall-clock rate of a single continuous run in Section XI.B, because the ramp experiment sends with no delay between rounds (max rate) to

find the scaling envelope, whereas the continuous run ties the send cadence to the 15-second simulated-time interval.

D. Off-chain Matching Throughput

Beyond the ingest path, we measure the order-matching cost of the Continuous Double Auction (CDA) matching engine in the off-chain layer with a micro-benchmark (Criterion) that calls the matching function for one match cycle over an order book of 1,000 buy orders and 1,000 sell orders (2,000 orders total), where every buy order price is set higher than every sell order price so that matching is full, yielding a maximum of approximately 1,000 order pairs per cycle. The experiment runs on the same machine (Apple M2, 8 cores, 16 GiB memory) and collects 100 samples after the warm-up period. The results are summarized in Table XXIV.

Table XXIV
In-memory CDA matching throughput (one match cycle over 1,000 bids × 1,000 asks, 100 samples).

Metric	Value
Time per 1,000 × 1,000 match cycle (median)	32.56 ms
95% confidence interval	32.28–32.75 ms
Order processing rate (≈)	6.1×10^4 orders/s
Order-pair matching rate (≈)	3.1×10^4 pairs/s

The matching engine processes one full cycle of orders on both sides in a median time of 32.56 milliseconds (95% confidence interval approximately 32.28–32.75 milliseconds), which corresponds to a processing rate of approximately 6.1×10^4 orders per second, or about 3.1×10^4 matched order pairs per second on a single thread. This value measures only in-memory matching and does not include the cost of on-chain transaction settlement, persistence, or network communication; it is therefore an upper bound on the matching rate cleanly separated from the settlement cost. This result indicates that in the layered architecture, off-chain matching is not the system bottleneck compared to the on-chain settlement cost in Section XI.E. We note that single-cycle 1,000 × 1,000 matching is a synthetic workload to measure per-cycle cost, not a steady-state rate under a real order stream with diverse price and zone distributions, which is the next step.

E. On-chain Settlement Cost

Beyond the ingest path, we directly measure the on-chain processing cost of settlement by reporting the compute units (CU) actually consumed by the off-chain match settlement instruction, read from the ***computeUnitsConsumed*** value of the confirmed transaction in the escrow settlement test on solana-test-validator 3.1.10 (Agave client). This value is a deterministic on-chain cost metric independent of the validator hardware, unlike latency on a local test network, which does not reflect the design's target network. The result is summarized in Table XXV.

Table XXV
Measured compute-unit cost of the settlement instruction (1 matched order pair).

instruction	compute units
settlement per 1 order pair	96,707

The value of 96,707 CU per settlement of one order pair[5] compared with the per-instruction default compute budget of 200,000 CU and the maximum per-transaction ceiling of 1,400,000 CU indicates that each settlement uses approximately 48% of the default budget. We note that although the maximum per-transaction compute ceiling could in theory support around 14 pairs per transaction, the binding constraint is not compute but the transaction packet size of 1,232 bytes, which squeezes the number of pairs per transaction below the ceiling of 4 pairs that the ***batch_settle_offchain_match*** program allows (see Section IV.I). Therefore, reducing the per-pair cost via batch settlement requires changing the way signatures are packed, not merely relying on the remaining compute budget. The 48% per-instruction cost level remains consistent with the design of having the blockchain serve as a low-compute settlement layer per Section VII.

F. On-chain Transaction Throughput

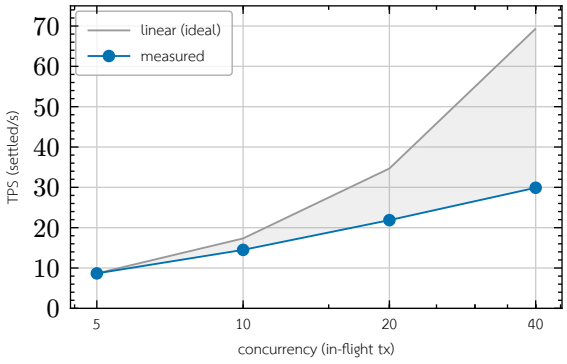
Beyond the per-instruction compute cost in Section XI.E, we measure the throughput of on-chain transaction processing using standard OLTP workloads ported to Solana's account model, namely BlockBench (a SIGMOD 2017-style microbenchmark together with YCSB), SmallBank, and TPC-C, augmented with a measurement of the real settlement path (settle_offchain_match). All run on a single solana-test-validator (single-node; a permissioned network governing participation via PoA) on an Apple M2 machine (8 cores) at commit ***58cfc79*** of gridtokenx-anchor, where latency is wall-clock time of the client⟶confirmed path and compute units are read from the ***computeUnitsConsumed*** of confirmed transactions. The sequential OLTP workload results are summarized in Table XXVI and the TPC-C concurrency sweep results are summarized in Table XXVII.

Table XXVI
Standard OLTP workloads ported to the account model (sequential, n = 150). TPS here is latency-bound by the single-client submit loop, not a throughput ceiling; *ycsb_read* is an RPC account fetch with no consensus round-trip.

workload · op	mean ms	TPS	CU/tx
BlockBench · *do_nothing*	719.95	1.39	648
BlockBench · *cpu_heavy_sort*	590.83	1.69	9,645
BlockBench · *ycsb_read* (RPC)	4.29	233.18	—
SmallBank · *SendPayment*	604.63	1.65	5,963
SmallBank · *Amalgamate*	631.62	1.58	5,936

Table XXVII
TPC-C concurrency sweep (TX = 500, 50% NewOrder / 50% Payment), 100% success at every level. Throughput from concurrent in-flight submission; CU/tx is flat across load.

concurrency	TPS	mean ms	p95 ms	CU/tx (mean)
5	8.67	575.00	726.10	21,263
10	14.50	679.34	879.95	20,633
20	21.87	856.74	1,656.15	21,269
40	29.90	1,057.79	1,707.00	21,508



[5]The compute cost depends on the token-leg transfer pattern of the transaction. The reported value is from one escrow settlement test variant; the path with a classic-SPL to Token-2022 exchange leg measures around 103k CU, and batch settlement using Token-2022 on both legs measures around 80–92k CU. The figure of 96,707 is therefore specific to the measured variant, not a universal constant.

Fig. 11. TPC-C throughput vs. concurrency on the single-node validator. Measured TPS scales sublinearly — 8× concurrency yields only 3.45× throughput — diverging from the linear-ideal reference, with the saturation knee between concurrency 10 and 20.

The most important figure for this system is the throughput of the real settlement path, not that of a generic proxy. When measuring the *batch_settle_offchain_match* path with an open-loop submission sweep (pre-building transactions, then firing them concurrently according to the concurrency level and re-firing dropped transactions until confirmed), we obtain a rate of only 0.5 TPS that stays flat at every concurrency level (this figure comes from a single recorded run, unlike the TPC-C/OLTP set whose raw results were committed as an artifact, so it should be interpreted as a design-indicative value pending re-measurement) (conc 5 ⟶ 0.51, conc 10 ⟶ 0.58 TPS; goodput 100%, no on-chain reverts, CU around 86–89k). It also does not scale with the number of concurrently submitted transactions; even distributing settlement across all 16 shards yields a near-identical figure (0.57–0.59 TPS), because the bottleneck is not the shard but the central set of accounts that every settlement must always write, namely the accumulator account *total_settled_thbc* and the fee/wheeling/loss collector accounts. Settlement is therefore global-write-bound by design; sharding can parallelize only order submission, which uses per-entity PDAs, not the reconciliation of settlement.

In the TPC-C sweep, the compute units per transaction stay flat at around 21,000 CU at every concurrency level, indicating that the throughput bottleneck is the block production time (block time around 400–600 milliseconds) together with the serialization of central account writes, not the on-chain processing. For this reason, compute units per transaction (Section XI.E) is a cost metric independent of workload and machine, whereas TPS is a figure that depends on the single validator used for testing. The sweep also exhibits sublinear scaling (concurrency 5 ⟶ 40, an 8-fold increase, yields only a 3.45-fold increase in TPS) and a saturation knee between concurrency 10 and 20 where the latency variance and p95 increase markedly, as shown against the linear reference line in Fig. 11.

Compared with the 900-second clearing window, even the real settlement rate of 0.5 TPS still supports around 450 settlements per window. The 80-meter workload places at most one order per meter per window, so it generates at most $\lfloor 80/2 \rfloor = 40$ matched pairs per window; the 450-settlement capacity is therefore roughly $11\times$ that demand. More generally, a single-validator settlement budget of about 450 pairs per window covers a fleet of up to roughly 900 meters at one order per meter per window before a second validator or a wider clearing window is required, which bounds the deployment size this design serves on a single node. The proxy ceiling of 29.9 TPS is equivalent to around 2.69×10^4 transactions per window. However, all results come from a single validator and should therefore be interpreted under four limitations. First, measurements on a single validator do not reflect the consensus cost (PoH together with Tower BFT) of a multi-node permissioned network, which is the basis for the observation that block time is the bottleneck. Second, BlockBench, SmallBank, and TPC-C are generic proxies, not energy workloads, so the TPS of the real settlement path is significantly lower than the proxy figures. Third, the sequential path in Table XXVI at 1.4–1.7 TPS is latency-bound, not a throughput ceiling. And fourth, the TPC-C sweep TPS figures are single-point values per concurrency level, reporting confidence intervals only for latency (latency CI95) and not yet repeated over multiple runs to report a confidence interval for TPS. Multi-node open-loop measurement with repetition to find peak TPS at an SLA level is the next step (see Section XII).

G. *Per-instruction Compute-unit Profile*

Beyond the cost of the main settlement path in Section XI.E, we measure the per-instruction compute cost of every on-chain program in the real execution path, to confirm by measurement that the on-chain compute load is kept thin by design. All values are read from the transactions' *computeUnitsConsumed*, measured in-process with LiteSVM over a default-feature build at commit *58cfc79* of gridtokenx-anchor, except for the settlement instruction *settle_offchain_match*, which is measured on a live validator (Section XI.E). Because compute units are deterministic, they are comparable between the two methods. The result is summarized in Fig. 12.

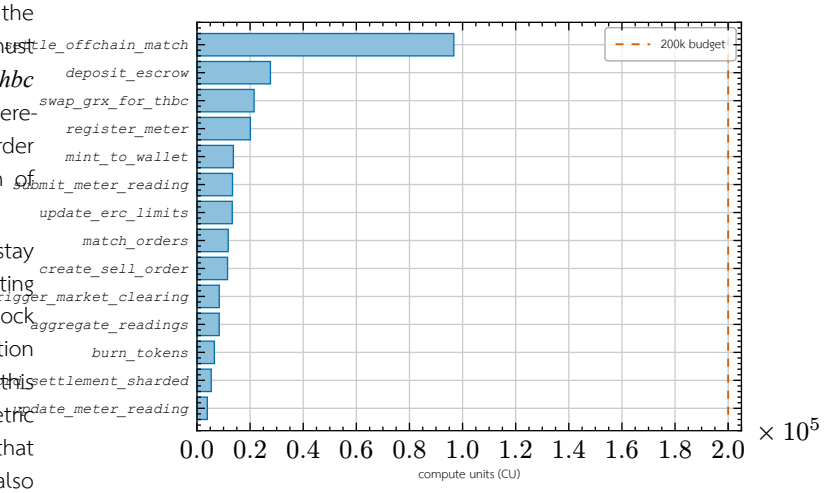


Fig. 12. Measured per-instruction compute-unit cost across the on-chain programs (LiteSVM *computeUnitsConsumed*, default-feature build; *settle_offchain_match* measured on a live validator per Section XI.E), sorted by cost. Only the signature-verifying settlement path approaches half the 200,000 CU per-instruction budget (dashed); every other instruction stays well below — the measured signature of a thin settlement layer.

From Fig. 12, every instruction except the signature-verifying settlement path (*settle_offchain_match*) uses no more than approximately 28k CU, or about 14% of the 200k CU default budget, with the high-frequency paths at the lowest levels, namely *update_meter_reading* (3.9k) and *record_settlement_sharded* (5.4k), consistent with the design of having per-entity PDA accounts on the hot path not write-lock central accounts. This result confirms by actual measurement (not estimation) that heavy processing work — such as data-format validation, evaluation of grid-stability conditions, and queuing of buy/sell orders — is moved to the Backend layer first, while the Smart Contract enforces only the minimal conditions verifiable on-chain (account ownership, Ed25519 signatures, order nullifier, and escrow state), thereby serving as a low-compute settlement and audit layer as designed.

The surplus generation-mint path, exercised end-to-end on a live validator rather than in LiteSVM, confirms the same thin-compute picture from the opposite end of the pipeline. A single-recipient *mint_generation* transaction (Section IV.I) measured on-chain consumes about 4,455 CU for the idempotent associated-token-account creation and about 1,237 CU for the Token-2022 *MintTo* cross-program invocation, keeping the whole minting transaction an order of magnitude below the 200,000 CU per-instruction budget and far

under the signature-verifying settlement path. As with settlement, neither compute nor the packet binds a single mint; the cost of scaling comes only from batching recipients. Its transaction fee of 10,000 lamports corresponds to the two required signatures — the platform authority, which is also the fee payer, and the mandatory REC-validator co-signer — the same two-party control that governs generation minting.

On the I/O and storage side, the architecture sends high-frequency meter data through the Aggregator Bridge for screening first, then records only the event log of transactions necessary for retrospective auditing onto the chain (see Fig. 5). Quantitative measurement of record volume, retention policy, and storage cost remains future work.

H. Fleet-Scale Solver Throughput and Live On-chain Mint Validation

Solver cost is tracked as the fleet grows orders of magnitude, and computed surplus is confirmed reaching a real on-chain mint; a single validator runs 0.5 tx/s (global-write-bound by design), yet 450 settlements fit per 900 s window — about 10× tested demand.

Beyond the ingest-path measurements (Section XI.B, Section XI.C) and the on-chain cost/throughput results (Section XI.E, Section XI.F), we ran two further experiments to probe the simulator's capacity envelope along orthogonal axes: (a) solver and reading-generation compute cost as fleet size grows by orders of magnitude, with no network egress at all, and (b) confirmation that the surplus energy the simulator computes actually reaches a real on-chain mint through every layer of the stack. Both experiments are single exploratory runs, not repeated-trial benchmarks with reported mean and standard deviation as in Section XI.C, and should be read as design-indicative figures pending repeated measurement, not final reference numbers.

1) Solver Throughput at Fleet Scale

The first experiment drives *SimulationEngine.tick()* directly, bypassing the network path entirely (no Aggregator Bridge, no blockchain), to isolate the cost of per-meter device-model generation and the power-flow solve on the 80-bus reference topology at fleet sizes of 10,000, 50,000, and 100,000 meters. Rooftop-PV assignment is randomized per meter at a 10% ratio (rather than pinning one meter per bus as in the original default) via the *SOLAR_PROSUMER_RATIO* parameter. Results are summarized in Table XXVIII.

Table XXVIII
Per-tick solver wall-clock cost vs. fleet size (single exploratory run, 4 ticks/case, no network egress).

Meters	PV share	median tick	p95 tick	max tick
10,000	9.90%	16.5 s	17.7 s	17.7 s
50,000	9.81%	97.6 s	107.8 s	107.8 s
100,000	9.97%	230.4 s	397.5 s	397.5 s

The measured PV share converges toward the configured 10% target as fleet size grows, consistent with the law of large numbers over the per-meter random draw, and confirms that the *SOLAR_PROSUMER_RATIO/ GRID_CONSUMER_RATIO/HYBRID_PROSUMER_RATIO* parameters now actually take effect for topology-backed fleets (prior to the fix, the split was hardcoded to 70/20/10 and these parameters were silently ignored). Per-tick wall-clock cost scales mildly super-linearly with fleet size — the 100,000-meter case takes roughly 14× the 10,000-meter case's tick time for a 10× larger fleet — indicating that per-meter reading generation (single-threaded CPU-bound work dispatched via *asyncio.to_thread*) is the bottleneck, not the power-flow solve, whose topology size is fixed regardless of meter count.

2) Live On-chain Mint Proof

The second experiment probes the opposite axis: a small fleet (100 meters) with the full network path enabled — meter-owner onboarding through the IAM Service (register ⟶ verify ⟶ login, with on-chain PDA registration), Ed25519 device-key registration in the Aggregator Bridge's device registry, signed reading ingest every tick over server-authenticated TLS with an AES-256-GCM-encrypted DLMS/COSEM payload[6], and a real on-chain mint transaction signed and submitted by Chain Bridge once a settlement window closes. The run targets the same single solana-test-validator used in Section XI.F.

Results from a 100-meter, 5-tick run confirm the full path is consistent end to end: owner onboarding for all 100 meters completed in roughly 11 s; every ingested reading passed signature verification and was disseminated into its zone Redis Stream; and — the key proof point — **140 real on-chain mint transactions across 20 unique meters** were observed, each carrying a verifiable Solana transaction signature and slot number (for example, a 0.08568 kWh mint with signature *3NbmxvqEKRLfM2yEYJLNy7axVcibwBXTzVbDAPAhdbNaN643ZPdV* at slot 1424). This closes the gap the first experiment deliberately leaves open — that experiment drives *tick()* directly without calling *start()*, so it never touches the Aggregator Bridge or the chain at all — by demonstrating that the net-surplus figures the simulator computes agree with what is actually minted on-chain.

This experiment is deliberately small in scale: IAM onboarding is one HTTP round-trip per meter owner, and each mint is a genuinely signed Solana transaction per settlement window. Running the 10,000–100,000-meter fleets from the solver-throughput experiment with the full network path enabled is out of scope for this work and is left for future work that scales both IAM onboarding throughput and the test validator's capacity accordingly (see Section XII).

3) Fleet-Scale Live Mint Throughput

A third experiment closes part of the future-work gap the previous one leaves open: driving the **full** network path (IAM onboarding, signed encrypted ingest, settlement, and Chain Bridge minting) at fleet sizes in the thousands rather than 100, using a dedicated harness against the same single solana-test-validator. Each meter's one net-surplus reading is backdated into an already-closed 15-minute settlement window so the sweep evicts it without waiting for real time to pass. Results are summarized in Table XXIX; as with the two experiments above, these are single exploratory runs, not repeated trials.

Table XXIX
Fleet-scale live on-chain mint throughput (single exploratory runs).

Fleet	Minted	Overall TPS	Steady TPS (10 s median)	Peak TPS (10 s)
1,000	1,000 / 1,000	29.3	—	—
10,000	10,000 / 10,000[7]	18.4	37.7	123.2
25,000 (tuned)	25,000 / 25,000	23.2	14.6	193.4

[6] The IoT gateway's client-certificate verification is implemented but left disabled in this deployment, so ingest is server-auth TLS, not mutual TLS; device authentication of record is the API key plus the per-reading Ed25519 signature, per Section III.

[7] 9,921 minted directly within the watch window; the remaining 79 landed later via the durable mint outbox's retry — see the wallet-link incident below.

At an untuned baseline, a 25,000-meter burst overloads Chain Bridge's mint-consumer queue: envelopes queue past the aggregator's 55-second staleness cap faster than the fixed-concurrency consumer (8 in-flight confirmations) can drain them, so rejected envelopes are republished, deepening the queue further — goodput decayed from 16.7 to 3.7 mints/s in a positive-feedback congestion collapse before the run was stopped manually (2,559 of 25,000 minted). Raising the mint-consumer concurrency (8 ⟶ 64), skipping a redundant pre-flight simulation, and lengthening the aggregator's reply-wait budget (30 s ⟶ 120 s) eliminated the collapse: the tuned 25,000-meter row above cleared with zero stale-rejects and zero republish amplification (versus 65,117 stale-rejects and a 22,528-entry outbox re-published every tick at baseline).

The 10,000-meter run also surfaced a genuine operational bug worth reporting on its own terms: a subset of onboarded users' wallet-link writes were lost when the IAM service itself was OOM-killed during the highest-concurrency onboarding bursts. The root cause was that IAM's request-metrics middleware labeled Prometheus counters by the literal request path rather than the route template, so each *PATCH /wallets/{id}* call — issued once per onboarded user to set a primary wallet — permanently added a new label series; resident memory therefore grew with **cumulative onboard count**, not concurrency, until the container's memory limit was exceeded. No surplus was permanently lost — the affected meters were flagged "no wallet registered, kept for retry" and the durable mint outbox re-minted them into their original settlement window once the wallet links were repaired (the 79-meter tail in Table XXIX), which is the de-attribution behavior of the ingest path described in Section VIII rather than a special case added for this run — but the incident is a concrete illustration of why per-request metrics must be labeled by route template, not literal path, under any workload with per-entity path parameters.

4) On-chain Telemetry Write Scaling by Meter Count

A fourth experiment isolates the on-chain layer of the telemetry path — the oracle program's *submit_meter_reading* instruction alone, bypassing IAM and the Aggregator Bridge entirely — to answer a question the mint experiments above leave open: does on-chain state-write capacity degrade as the per-meter PDA account set grows into the hundreds of thousands? The harness builds client-signed transactions against a cached blockhash, submits them *skipPreflight* through a windowed open loop capped at 3,000 in-flight transactions, and confirms via bulk *getSignatureStatuses* polling (256 signatures per call at a 1.5 s cadence, so reported latencies are quantised by ±1.5 s), at fleet sizes of 10,000, 50,000, 100,000, 150,000, and 200,000 meters, two epochs each: the first epoch creates each meter's PDA (init, heavier compute), the second overwrites existing state (steady). All scales run back-to-back on the same single solana-test-validator without a ledger reset in between, so the 200,000-meter case starts against an existing set of over 310,000 meter PDAs and the validator ends the sweep holding 510,000 meter PDAs from this experiment alone; the experiment totals 1,020,000 transactions. Results are summarized in Table XXX; as with the other experiments in this section, each scale is a single exploratory run.

Table XXX
On-chain telemetry write throughput vs. meter count (single exploratory run per scale; latency quantised ±1.5 s by the status-poll cadence).

Meters	ok/total	TPS (init)	TPS (steady)	p50 ms	p95 ms	CU steady (med)
10,000	19,908 / 20,000	439.2	429.1	1,546	2,354	15,025
50,000	99,524 / 100,000	319.8	243.1	1,868	3,064	13,525
100,000	199,048 / 200,000	409.6	455.5	1,553	2,346	15,060
150,000	298,572 / 300,000	449.4	426.2	1,586	2,402	13,560
200,000	398,096 / 400,000	438.5	443.2	1,582	2,391	13,560

The headline finding is flat throughput across scale: steady-state TPS stays in the 426–455 band from 10,000 through 200,000 meters (the 243 at 50,000 is a transient validator stall that does not recur at larger sizes), p50 latency holds at roughly 1.5–1.9 s, and per-transaction compute cost is constant (steady median 13,525–15,060 CU, far below the 200,000 CU per-instruction budget) independent of the accumulated account count. This empirically confirms the per-meter PDA state design — one *MeterState* account per meter, with the hot path never writing the global *OracleData* account — keeps reading writes parallelizable under Sealevel, and that the remaining serialization point is the single gateway fee-payer wallet (write-locked on every transaction), not account-set size. Every failed transaction (0.46–0.48% per scale) is a deterministic logical rejection by the oracle's own anomaly detector — a small fraction of the synthetic readings violate the production-to-consumption ratio cap, checked by integer cross-multiplication (*AnomalousReading*) — with not a single send rejection, queue overflow, or blockhash expiry at any scale. The global aggregate counters on *OracleData* remain zero throughout by design: the hot path treats that account as read-only, and reconciling fleet totals is the job of the periodic administrative *aggregate_readings* instruction. The key limitation is that all results come from a single validator and a single submitting wallet, so the measured ceiling is a property of this configuration; spreading load across multiple gateway fee-payers and measuring on a multi-node cluster remain future work (see Section XII).

5) What "Network Limitation" Decomposes Into

It is tempting to summarize a distributed energy system by a single "network throughput" number, but every ceiling this system actually meets is a scheduling or serialization limit rather than a bandwidth one — no measurement anywhere in this work is bounded by the rate at which bytes move over a link. The apparent network limit decomposes into three distinct walls, at three layers, each with its own fix and its own evidence, summarized in Table XXXI.

Table XXXI
The three throughput walls of the system, none of them bandwidth. Each is a serialization or scheduling limit at a different layer, with a distinct escape and distinct measured evidence. The middle wall is the one the settlement design actually targets.

wall	what it caps	evidence
slot cadence	any op needing one consensus round-trip ⟶ 2 TPS per serial chain	485 ms/slot ⟹ 2.06 TPS write vs 1,454 TPS RPC read
account write-lock	same-slot txs sharing a mutable account serialize	`zone_market` read-only: 1⟶3 settles/slot (2.7×)
ingest event loop	one sender process saturates its own CPU scheduler	200 tx/s per client; 400/s bridge; loss ≤ 0.48%

The first wall is the slot cadence of the consensus layer. A slot on the tested configuration takes roughly 485 milliseconds, and any operation that must wait for one consensus round-trip before the next can proceed is floored by that interval regardless of how little compute it uses. The clearest evidence is a controlled contrast: a trivial no-op instruction that still requires a slot to confirm runs at a flat 2.06 transactions per second, identical to a compute-heavy instruction on the same path, because both are paying for the slot and not for the work — whereas a pure account read served over RPC without any consensus round-trip runs at about 1,454 transactions per second on

the same machine. The three-order-of-magnitude gap between them is the price of consensus, and it means latency-bound operations are block-time-bound, not program-bound. This is a platform property of any Solana-compatible network and is not something the programs in this system can tune.

The second wall is the one the settlement design genuinely targets, and it is account write-lock serialization rather than anything on the wire. Solana's parallel scheduler co-schedules two transactions only if their write sets are disjoint; two transactions that mutate the same account serialize no matter how much network or compute capacity is idle. The settlement path was, in an earlier form, bound by exactly this: every same-zone settlement took a write lock on the shared ***zone_market*** account, capping the zone at roughly one settlement per slot. Splitting the mutable committed-flow counter onto a separate ***ZoneCapacity*** account and leaving ***zone_market*** read-only on the common path (the design already described in the settlement model) lifts that to about three settlements per slot — a measured factor of roughly 2.7, achieved purely by changing which account is locked, with no change to compute. The same effect appears at the order layer: order entry, which contends on shared book state, is measured at about 180 transactions per second at a ten-thousand-order scale, whereas meter ingest at the same scale reaches about 462 because each reading writes only its own per-meter account. And the discovery path, which shares no hot lock, scales cleanly from 7.87 to 74.25 transactions per second as concurrency rises from 5 to 40 — a 9.4-fold gain climbing monotonically with no plateau, which is the positive control proving that when the shared lock is removed the transport does not bind. The lesson is that adding validators or bandwidth does not raise the settlement ceiling; making the hot account read-only does. Sharding helps only across **different** locked accounts, so every same-zone settlement contending on one ***zone_market*** PDA serializes regardless of how many shards exist elsewhere.

The third wall is off-chain, at the ingest client. A single sender process feeding the Aggregator Bridge saturates at roughly 200 transactions per second, and the limit is its own event loop and CPU scheduler rather than signature-verification cost or link bandwidth — pushing more parallel senders past the core count reduces throughput through contention rather than raising it. The bridge itself verifies at roughly twice that rate. Under sustained fleet load the consequence is not error but a small, purely transport-level loss: over a three-thousand-transaction window the delivery loss stays at or below 0.48% (and is frequently zero), with p95 latency under about 3.1 seconds and not a single on-chain error, the losses being expiries and rejections at the sender rather than faults in the chain. This is the same lower-bound-of-a-single-client caveat noted for the saturation measurement in Section XI.C.

Taken together, these three walls say something specific about how to scale the system, and it is not "add bandwidth." Latency-bound operations are floored by the slot and are the platform's to improve; the settlement ceiling is raised by reducing what each settlement write-locks, which is a program-design lever the system already pulls; and ingest is widened by adding sender processes up to the core count, not by faster links. None of the three is a bandwidth problem, which is why none of the measurements in this work reports one. The single-validator, single-wallet caveat of Section XI.H.4 still applies throughout — a multi-node cluster would introduce the consensus-layer transport costs (leader rotation, gossip fan-out, fork choice) that a single-node localnet never exercises, and quantifying those remains future work (Section XII).

XII. DISCUSSION AND LIMITATIONS

Layer separation yields a verifiable, extensible structure; results are simulation-derived and single-run for exploratory experiments, not field-measured on a real grid or devices.

A. Discussion

The results of the simulated system design indicate that the approach of separating the operational layers between the Smart Meter Simulator, Backend Service, and Solana Smart Contract enables the Peer-to-Peer energy trading system to have a verifiable structure with clearer points for system expansion. The Backend Service acts as the primary control layer for validating data, managing permissions, and evaluating grid-stability conditions before submitting transactions to the blockchain, so that the Smart Contract does not have to bear the entire processing burden of the microgrid system.

Using the Solana Smart Contract as a Settlement and Audit layer enhances the transparency of energy transactions, because trade orders and energy settlement states can be audited retrospectively. However, the system must still prioritize the safety of the electrical grid first. Therefore, transaction approval should not consider only the price and energy quantity, but must also take into account system stability conditions and the connection status of the devices.

The three-part evaluation supports the design hypothesis that the blockchain serves as a thin settlement layer. First, the ingest path keeps the loss ratio close to zero (≤ 0.03%) up to a load of 640 devices (see Section XI.C), where the measured ingest rate is a lower bound of a single sender client and does not increase monotonically with the number of meters, so no conclusion is drawn about the service saturation point; this indicates that moving the high-frequency validation and aggregation work to the off-chain layer does not become a system bottleneck under the tested load levels. Second, the measured on-chain settlement cost of 96,707 CU per order pair accounts for approximately 48% of the initial per-instruction compute budget (see Section XI.E), showing that once most business rules are enforced off-chain, the remaining on-chain burden is at a level that opens a compute-wise opening to combine multiple order pairs into a single transaction (batch settlement), although the actual combination is constrained by the transaction packet size before the compute ceiling (see Section IV.I), so the signature-packing method must be adjusted to actually reduce the per-order-pair cost in future work. Third, the fact that the on-chain cost is a deterministic CU value rather than varying with public-network market fees is consistent with the governance rationale for choosing a PoA consortium network (see Section II), where transaction costs are predictable and independent of gas volatility.

In terms of economic mechanism, separating the roles of the three token types — the energy token certified by REC for delivery, the THBC coin pegged to the baht for pricing and payment, and the GRX token for stake and governance — enables transaction settlement to be DvP, which binds energy delivery to payment in a single transaction, and grants network governance rights under a consortium framework that clearly controls REC certification and the aggregator allow-list. We have analyzed the preliminary model-derived economic implications of the net amount received by

sellers and the premium relative to the feed-in tariff in Section VI.B. However, the full measurement of economic properties, such as allocative efficiency / welfare and the strategic behavior of participants, remains outside the scope of the architectural evaluation in this work.

Compared with permissioned platforms widely used in the P2P energy trading literature such as Hyperledger Fabric [15] (see Table II), the design in this work differs architecturally in three points. First, Fabric uses an execute-order-validate processing model in which chaincode runs on endorsing peers before ordering through an orderer (Raft/BFT), whereas this work uses Solana's account model that processes transactions in parallel via Sealevel when the sets of written accounts do not overlap (see Section VII.F.7), which facilitates scaling order submission per entity but makes settlements that reconcile a central balance bound to a central write (global-write-bound), as measured in Section XI.F. Second, the processing cost in this work is expressed in compute units (CU) that are deterministic and have a clear per-transaction ceiling, unlike Fabric, which has no equivalent concept of a per-transaction compute budget, making this work's on-chain cost straightforward to predict and to detect regression. Third, this work enforces the provenance check of meter readings with Ed25519 signatures and replay prevention through an order nullifier directly within the program logic, instead of relying on an external endorsement policy. This comparison is qualitative (architectural) because there is not yet a head-to-head measurement on the same energy workload, which is a quantitative comparison left open for the future.

B. Limitations

A key limitation of this work is that the system is still at the simulation level, with energy data coming from the Smart Meter Simulator and not yet tested together with real Smart Meter devices or field electrical systems. Therefore, the results reflect the suitability of the architecture and the preliminary working process rather than quantitative performance in a real environment.

In addition, the evaluation covers only part of the entire operational path, namely measuring the ingest rate of the telemetry ingest path and the compute cost of the on-chain settlement instruction on a per-part basis, but has not yet measured the end-to-end latency from meter reading to on-chain confirmation, nor the order matching rate under multiple order-volume levels. As for the throughput of the settlement path, it was measured on a single validator (see Section XI.F) but does not yet cover a multi-node permissioned network that includes consensus cost, and batch settlement has been characterized only by its per-instruction compute cost (Section XI.E), not by an end-to-end throughput experiment that would show whether combining pairs actually lowers the cost per settled pair under the packet-size constraint of Section IV.I. In terms of trust, the energy quantity and grid-stability conditions are enforced in the off-chain layer by the Aggregator Bridge and the oracle authority, whose collusion or compromise is not yet prevented by cryptographic mechanisms (see Section III). Likewise, the security assumption of the consensus (PoH together with Tower BFT) under a permissioned network that governs participation via PoA is also an architectural assumption that has not yet been quantitatively evaluated under validators with deviant behavior.

Three further limitations concern the prototype as deployed rather than the design. First, the per-***(meter_id, window)*** mint idempotency that guarantees no double-mint (T5 in Section III) has an inverse failure mode: a reading that arrives for a window whose mint record already exists is absorbed by the ***init_if_needed*** PDA as a no-op, so a meter that buffers telemetry offline and replays it long after its window has settled has that energy **under-minted** rather than double-minted. A settle grace period defers settlement of a just-closed window and removes the boundary case, but a truly late replay still strands its energy; routing a late reading into a correction or into the next window is a design change left open. Second, the settlement path the deployment actually calls is the custodial pooled-escrow variant (Section VII.F.1), so the address-binding guarantee of the non-custodial model does not apply to the runs reported here; Section IX records this as a violated invariant rather than a caveat, on the grounds that custody is the direction the platform chose rather than a gap it drifted into. Third, the operational fault modes observed during the fleet-scale runs — a wedged message-broker producer that does not self-heal without a restart, and a mint that stalls when the on-chain certifier set is misconfigured — are recovered by operator action rather than automatically; the durable outbox preserves the work but does not by itself restore liveness. Continuous integration is likewise not exercised for this artifact, so every result reported here rests on local runs on the single machine described in Section X.

C. Threats to Validity

The measurement results in this work have three caveats in interpretation. First (internal validity), the workload in the ingest experiment was generated by a single sender client operating asynchronously, so the measured ingest rate is a lower bound that also reflects the sender-side limitation, not the true capacity ceiling of the ingest service (see Section XI.C). Second (construct validity), the design rate of 5.33 readings per second is computed from the number of meters and the transmission cycle, not the maximum measured rate, so comparing the two figures must clearly state the context of each value. Third (external validity), the compute cost of the settlement instruction was measured on a single solana-test-validator version, and that value is a logical program cost (deterministic CU) independent of hardware, but the latency and throughput on a real production network may differ. All results should therefore be interpreted as an architectural evaluation on a simulated system.

D. Future Work

Future work should develop the connection with real Smart Meters via the DLMS/COSEM standard and evaluate the entire operational path more completely, including measuring end-to-end latency and extending the settlement throughput measurement reported on a single validator in Section XI.F to a multi-node permissioned network that includes consensus cost, together with open-loop measurement to find the TPS ceiling at the SLA level, and experimenting with batch settlement to confirm the reduction of compute cost per order pair, as well as measuring the ceiling of the ingest path with multiple parallel senders that separate the sender-side cost from the ingest service. In terms of trust, the residual trust of the off-chain layer should be reduced with multi-oracle attestation or cryptographic proof, and the consensus of the permissioned network (governing participation via PoA) should be quantitatively evaluated under nodes with deviant behavior. In addition, models for grid stability and islanding detection should be added so that the safety

evaluation of energy trading more closely approximates real usage; for demand response the mechanism itself is already in place — the Aggregator Bridge carries an OpenADR 3 dispatch adapter against a VTN — but it is not driven as a variable in any experiment here, so what is missing is its evaluation rather than its implementation. The next experiment should clearly define a reproducible workload, such as the number of meters, sampling interval, transaction mix, validator count, queue depth, failure/retry policy, and market-clearing output metrics.

Beyond the above, three quantitative evaluation directions are explicitly left open, namely:

- **Allocative efficiency / welfare:** measure the gains from trade of the CDA mechanism relative to a uniform-price clearing baseline and the feed-in tariff on real matching data from a full simulated workload, in order to confirm the economic premium analyzed at the model level in Section VI.B with actual measurement instead of computation from fixed parameters.
- **Cost per trade:** convert the per-transaction compute cost into a fee in lamport units and baht at a given rate, then report the fee-to-trade-value ratio, which is a key deciding criterion for real-world adoption.
- **Liveness & baseline:** evaluate the availability of the permissioned network when some validator nodes fail (1-of-N down), and conduct a quantitative head-to-head comparison with other permissioned platforms, especially Hyperledger Fabric, on the same energy workload, in order to extend the qualitative comparison in Table II into a directly comparable joint measurement.

XIII. CONCLUSION

Four measured axes support the blockchain as a thin settlement layer — value lies in the integration; next come end-to-end latency, batch settlement, and real devices and networks.

This work presents the development of a simulation system for Peer-to-Peer solar energy trading in a microgrid, integrating a Smart Meter Simulator, a Backend/Aggregator Bridge, and a Solana-compatible Smart Contract. The system is designed to handle real-time energy data, to validate trading orders through the Backend service, and to record the results of transactions that satisfy the conditions on the blockchain in order to increase transparency and auditability.

The architectural evaluation indicates that the Microservice approach combined with transaction queuing through NATS JetStream has the potential to reduce the coupling of workloads among data ingestion, transaction validation, and recording results on the blockchain. In addition, restricting the role of the Smart Contract to a thin settlement and audit layer helps to more clearly bound the on-chain compute. The evaluation covers four aspects, namely: the telemetry ingest path, which sustains the design rate of 5.33 readings per second continuously with no data loss, and which, under a step load increasing to 640 meters (averaged over 5 runs), still keeps the loss ratio below 0.03% with no saturation knee observed within the limit of the single sender client used in the test; off-chain order matching, where the CDA matching engine processes approximately 3.1×10^4 order pairs per second in memory; the on-chain settlement cost of the single-pair instruction, measured at 96,707 compute units for the reported token-leg variant (approximately 48% of the initial compute budget), which opens room for batch settlement of order pairs; and the real settlement throughput, which is global-write-bound by design — serialized on a central reconciliation write — and measures roughly 0.5 transactions per second on a single validator in a single recorded run (pending repeated measurement), yet still supports roughly 450 settlements per 900-second clearing window, about an order of magnitude above the at most 40 matched pairs the tested 80-meter workload generates. All four results support the design that makes the blockchain a thin settlement layer. Measuring end-to-end latency, experimenting with batch settlement, and evaluating on real devices and networks are the next steps, and they form an important foundation for extending the work toward use at an industrial energy scale in the future.

REFERENCES

[1] W. Tushar, T. K. Saha, C. Yuen, D. Smith, and H. V. Poor, "Peer-to-Peer Trading in Electricity Networks: An Overview," *IEEE Transactions on Smart Grid*, vol. 11, no. 4, pp. 3185–3200, 2020.

[2] T. Morstyn, A. Teytelboym, and M. D. McCulloch, "Bilateral Contract Networks for Peer-to-Peer Energy Trading," *IEEE Transactions on Smart Grid*, vol. 10, no. 2, pp. 2026–2035, 2019.

[3] A. Paudel, K. Chaudhari, C. Long, and H. B. Gooi, "Peer-to-Peer Energy Trading in a Prosumer-Based Community Microgrid: A Game-Theoretic Model," *IEEE Transactions on Industrial Electronics*, vol. 66, no. 8, pp. 6087–6097, 2019.

[4] IEEE Standards Association, "IEEE Std 1547-2018: IEEE Standard for Interconnection and Interoperability of Distributed Energy Resources with Associated Electric Power Systems Interfaces." 2018.

[5] IEEE Standards Association, "IEEE Std 2030.7-2017: IEEE Standard for the Specification of Microgrid Controllers." 2018.

[6] J. Guerrero, A. C. Chapman, and G. Verbic, "Decentralized P2P Energy Trading Under Network Constraints in a Low-Voltage Network," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 5163–5173, 2019.

[7] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System." 2008.

[8] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform." 2014.

[9] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger." 2014.

[10] D. Yaga, P. Mell, N. Roby, and K. Scarfone, "Blockchain Technology Overview," technical report NIST IR 8202, 2018. doi: 10.6028/NIST.IR.8202.

[11] E. Mengelkamp, P. Staudt, J. Garttner, and C. Weinhardt, "Trading on Local Energy Markets: A Comparison of Market Designs and Bidding Strategies," *Applied Energy*, vol. 210, pp. 870–880, 2018.

[12] E. Munsing, J. Mather, and S. Moura, "Blockchains for Decentralized Optimization of Energy Resources in Microgrid Networks," in *2017 IEEE Conference on Control Technology and Applications*, 2017, pp. 2164–2171.

[13] A. Esmat, M. de Vos, Y. Ghiassi-Farrokhfal, P. Palensky, and D. Epema, "A Novel Decentralized Platform for Peer-to-Peer Energy Trading Market with Blockchain Technology,"

Applied Energy, vol. 282, p. 116123, 2021, doi: 10.1016/j.apenergy.2020.116123.

[14] J. Kang, R. Yu, X. Huang, S. Maharjan, Y. Zhang, and E. Hossain, "Enabling Localized Peer-to-Peer Electricity Trading Among Plug-in Hybrid Electric Vehicles Using Consortium Blockchains," *IEEE Transactions on Industrial Informatics*, vol. 13, no. 6, pp. 3154–3164, 2017.

[15] E. Androulaki *et al.*, "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains," in *Proceedings of the Thirteenth EuroSys Conference*, 2018, pp. 1–15. doi: 10.1145/3190508.3190538.

[16] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982, doi: 10.1145/357172.357176.

[17] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, 1999, pp. 173–186.

[18] M. Andoni *et al.*, "Blockchain Technology in the Energy Sector: A Systematic Review of Challenges and Opportunities," *Renewable and Sustainable Energy Reviews*, vol. 100, pp. 143–174, 2019, doi: 10.1016/j.rser.2018.10.014.

[19] Z. Tanis, A. Durusu, and N. Altintas, "A Comprehensive Review on Peer-to-Peer Energy Trading: Market Structure, Operational Layers, Energy Cooperatives and Multi-energy Systems," *IET Renewable Power Generation*, vol. 19, no. 1, 2025, doi: 10.1049/rpg2.70075.

[20] G. B. Bhavana, R. Anand, J. Ramprabhakar, V. P. Meena, V. K. Jadoun, and F. Benedetto, "Applications of blockchain technology in peer-to-peer energy markets and green hydrogen supply chains: a topical review," *Scientific Reports*, vol. 14, no. 1, 2024, doi: 10.1038/s41598-024-72642-2.

[21] G. B. Bhavana, R. Anand, J. Ramprabhakar, J. M. Guerrero, N. Thakkar, and A. Ambikapathy, "Comparative evaluation and simulation of blockchain consensus mechanisms for secure and scalable peer to peer energy trading in microgrids," *Scientific Reports*, vol. 15, no. 1, 2025, doi: 10.1038/s41598-025-27431-w.

[22] IEEE Standards Association, "IEEE Std 2030.8-2018: IEEE Standard for the Testing of Microgrid Controllers." 2018.

[23] SPIFFE Project, "X.509-SVID Specification." 2018.

[24] NATS.io, "JetStream." 2024.

[25] Coral, "Anchor Documentation." 2026.

[26] A. Yakovenko, "Solana: A New Architecture for a High Performance Blockchain." 2018.

[27] Solana Foundation, "Program Derived Addresses." 2026.

[28] Python Software Foundation, "Python 3.11 Documentation." 2026.

[29] D. P. Chassin, K. Schneider, and C. Gerkensmeyer, "GridLAB-D: An Open-Source Power Systems Modeling and Simulation Environment," in *2008 IEEE/PES Transmission and Distribution Conference and Exposition*, 2008, pp. 1–5. doi: 10.1109/TDC.2008.4517260.

[30] S. Ramirez, "FastAPI Documentation." 2026.

[31] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function Using NetworkX," in *Proc. 7th Python in Science Conference*, 2008, pp. 11–15.

[32] K. S. Anderson, C. W. Hansen, W. F. Holmgren, A. R. Jensen, M. A. Mikofski, and A. Driesse, "pvlib python: 2023 Project Update," *Journal of Open Source Software*, vol. 8, no. 92, p. 5994, 2023, doi: 10.21105/joss.05994.

[33] L. Thurner *et al.*, "pandapower: An Open-Source Python Tool for Convenient Modeling, Analysis, and Optimization of Electric Power Systems," *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018, doi: 10.1109/TPWRS.2018.2829021.

[34] OpenADR Alliance, "OpenADR 3.0 Definitions and Specification." 2023.

[35] S. Joshi, "Feasibility of Proof of Authority as a Consensus Protocol Model." 2021. doi: 10.48550/arXiv.2109.02480.

[36] Solana Foundation, "Tokens on Solana." 2026.

[37] Solana Foundation, "Solana Documentation." 2026.